C#Corner

Architecting Modern Applications using Microservices in ASP.NET Core Web API

Sardar Mudassar
Ali Khan

Architecting Modern Applications using Microservices in ASP.NET Core Web API

Sardar Mudassar Ali Khan

Author – Sardar Mudassar Ali Khan
Publisher – C# Corner
Editorial Team – Deepak Tewatia, Baibhav Kumar
Publishing Team – Praveen Kumar
Promotional & Media – Rohit Tomar

Dedication

Every single step that was taken toward the completion of the book was because of the strong background provided by my parents they supported me morally. Moreover, the community of C# Corner helped me a lot in getting knowledge and solutions where I was stuck. They motivate me in every situation and always ask us never to give up everything possible in the world. They held our hands firmly never letting us get down and made it possible for us to get through. This book is also dedicated to my seniors who motivated us to guide us in this regard.
The pure dedication of my book is to my parents our soul parents- the teacher and friends. Teachers always motivated us and supported us with full support whenever required.

Declaration

I Sardar Mudassar Ali hereby declare that this book contains the original work and that not any part of it has been copied from any other sources. It is further declaring that I have completed my book under the project of Free Education for Humanity and with the guidance of my teachers and my senior colleagues and this is entirely possible based on my efforts and my ideas.

Acknowledgment

It would be my pleasure and I am feeling ecstatically rapturous to pay the heartiest thanks. First, To Allah (SWT) who is most beneficent and most merciful that he enabled me and blessed me to take the pebble out of his multitude of bounties. To my dear parents, their prayers become true towards the completion of my book. I am greatly beholden to my esteemed honourable motivator and kind-hearted, teachers, friends, and colleagues who supported my idea all the way and never let me down and corporate with me in this way completely sincerely and heartedly.
Also, special thanks to those who helped me a lot in my way, but I have forgotten them to mention here.

— Sardar Mudassar Ali Khan

Table of Contents:

1

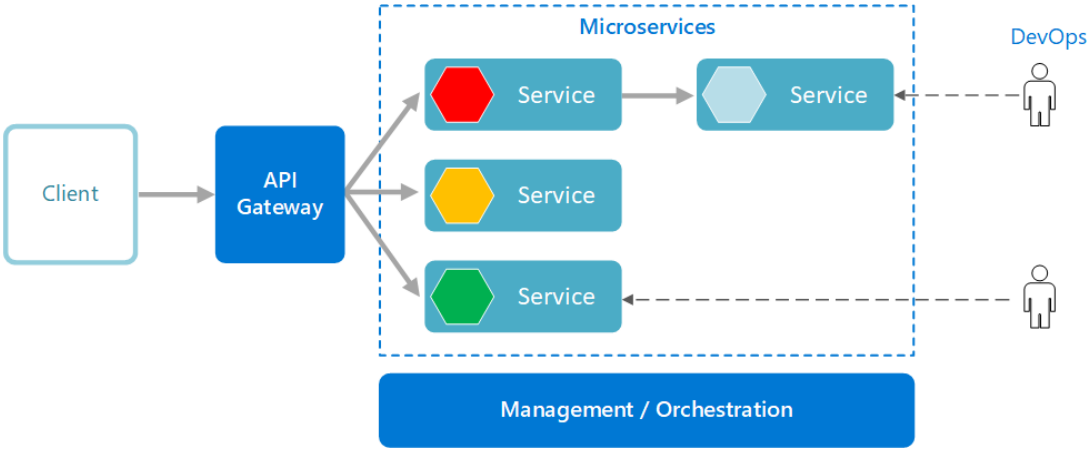
Microservices Architecture Introduction

Overview

In this chapter, we introduce microservices architecture and provide a comprehensive understanding of its principles and characteristics. We explain how microservices architecture works and compare it to monolithic architecture. The chapter covers the benefits of microservices, essential monitoring and observability practices, and the challenges associated with this architecture. We also outline best practices for implementation, present real-world use cases, and offer practical examples. Finally, we summarize the insights gained and emphasize the significance of adopting microservices architecture.

Introduction

Microservices architecture is an architectural style that structures an application as a collection of small, loosely coupled services. Each service is responsible for a specific business capability and can be developed, deployed, and scaled independently of other services. The microservices approach aims to break down monolithic applications into smaller, more manageable components that can be developed and maintained more easily.
In a microservices architecture, each service typically runs in its own process and communicates with other services through lightweight protocols, often using HTTP or messaging systems. Services are designed to be autonomous and have their own dedicated data storage, allowing them to make independent decisions and be developed using different technologies and programming languages, if they can communicate with other services effectively.



Source: https://learn.microsoft.com/

Key principles and characteristics of microservices architecture include:

Decentralized governance:

Each microservice is developed, deployed, and maintained by a small team, allowing for faster development cycles, and enabling independent decision-making within the team.

Scalability and elasticity:

Services can be scaled independently based on their specific demands. This approach allows efficient resource utilization and the ability to handle high traffic loads by scaling only the necessary services.

Resilience:

Microservices are designed to be resilient to failures. If one service goes down, the others can continue to function without being affected. This fault isolation is achieved through well-defined boundaries and communication protocols.

Flexibility and technology diversity:

Microservices enable flexibility in technology choices. Each service can be implemented using the most appropriate programming language, framework, or database for its specific requirements, promoting innovation and the use of best-suited tools for the job.

Ease of deployment and continuous delivery:

Microservices are typically deployed independently, allowing teams to release updates and new features without affecting the entire system. Continuous integration and continuous deployment practices are often employed to automate the deployment pipeline.

Independent scalability:

Services can be scaled independently, allowing resources to be allocated efficiently based on the specific needs of each service. This flexibility helps optimize performance and cost.
While microservices architecture offers numerous benefits, it also introduces challenges such as service coordination, data consistency, and distributed system complexity. These challenges need to be carefully addressed to ensure the successful implementation and operation of a microservices-based system.

What is microservices architecture used for?

Microservices architecture is used to design and build complex applications that can be divided into smaller, loosely coupled services. It offers several benefits and is commonly used for the following purposes:

Scalability:

Microservices allow individual services to be scaled independently based on their specific demands. This scalability enables efficient resource utilization and the ability to handle high traffic loads by scaling only the necessary services.

Agility and Flexibility:

Microservices enable organizations to iterate and deploy software more quickly. Services can be developed, deployed, and updated independently, which reduces dependencies and enables teams to work in parallel. It also allows for technology diversity, as each service can be implemented using the most appropriate programming language, framework, or database.

Fault Isolation:

Microservices promote fault isolation. If one service fails or experiences issues, it does not bring down the entire system. Other services can continue to function independently, ensuring the overall system's resilience.

Independent Development and Deployment:

Microservices architecture allows teams to work independently on different services, promoting autonomy and faster development cycles. Each service can be developed, tested, and deployed separately, facilitating continuous integration and continuous deployment practices.

Scalable Organization:

Microservices architecture aligns well with agile development methodologies and DevOps practices. It enables organizations to structure development teams around specific services, promoting smaller, cross-functional teams that can work autonomously and make independent decisions.

Integration and Interoperability:

Microservices can be designed to expose well-defined APIs, making it easier to integrate with other services, applications, or third-party systems. This flexibility enables organizations to adopt a modular approach, integrating new services or replacing existing ones as needed.

Legacy System Modernization:

Microservices architecture can be beneficial for modernizing and decomposing monolithic applications into smaller, more manageable services. It allows organizations to gradually replace components of the existing system with microservices, without the need for a complete system overhaul.
It's important to note that microservices architecture is not a one-size-fits-all solution and may not be suitable for all applications. The decision to adopt microservices should be based on careful consideration of the specific requirements, complexity, and scalability needs of the system at hand.

How does microservices architecture work?

Microservices architecture works by breaking down a complex application into smaller, loosely coupled services that can be developed, deployed, and scaled independently. Here are the key aspects of how microservices architecture operates:

Service Decomposition:

The first step in implementing microservices architecture is to identify the different business capabilities or domains within the application. Each domain is then encapsulated into a separate microservice. The boundaries between services are defined based on business functionality, ensuring that each service has a clear and well-defined responsibility.

Service Independence:

Each microservice operates as an independent unit, with its own codebase, database, and sometimes even its own technology stack. This independence allows teams to work on services separately, using the most appropriate tools and technologies for their specific needs.

Communication:

Microservices communicate with each other through lightweight protocols, often using HTTP or messaging systems like RabbitMQ or Apache Kafka. Services can make synchronous or asynchronous calls to exchange data and trigger actions. APIs, typically RESTful or event-driven, are used to define the interfaces and interactions between services.

Data Management:

Microservices can have their own dedicated databases, allowing them to manage their data independently. Each service is responsible for its data storage and ensures data consistency within its boundaries. Techniques like database per service, shared databases with bounded contexts, or event sourcing can be employed depending on the requirements.

Deployment and Scalability:

Microservices are typically deployed independently, either in containers (e.g., Docker) or as separate processes. This allows services to be scaled independently based on their specific

demands. Services experiencing high traffic can be scaled up, while others can remain unchanged, optimizing resource utilization.

Service Discovery and Orchestration:

As the number of microservices increases, service discovery mechanisms become crucial. Service registries or service mesh frameworks (e.g., Consul, etc., Istio) help locate and manage services dynamically. Orchestration tools like Kubernetes enable automated deployment, scaling, and management of microservices in a containerized environment.

Resilience and Fault Isolation:

Microservices architecture promotes fault isolation. If one service fails, it does not bring down the entire system. Services are designed to be resilient, with built-in mechanisms like circuit breakers, retry policies, and fallbacks to handle failures gracefully.

Monitoring and Observability:

Due to the distributed nature of microservices, monitoring and observability become crucial for understanding the system's health and performance. Logging, metrics, and tracing are essential for diagnosing issues, identifying bottlenecks, and gaining insights into the overall system behavior.

Microservices vs. monolithic architecture

Microservices architecture and monolithic architecture are two different approaches to designing and building applications. Here's a comparison of the key characteristics and differences between the two:

Monolithic Architecture:

Structure:

In a monolithic architecture, the entire application is developed as a single, unified unit. All components and functionality are tightly coupled and deployed together.

Development:

Monolithic applications are typically developed by a single team using a single technology stack. Changes and updates to the application require modifying the entire codebase.

Scalability:

Scaling a monolithic application often involves scaling the entire application, even if only certain components require additional resources.

Deployment:

Monolithic applications are deployed as a single unit. Updates and new releases require redeploying the entire application.

Communication:

Components within a monolithic application communicate through in-memory method calls or function invocations.

Data Management:

Monolithic applications typically use a shared database for all components, which can result in potential data coupling and dependencies.

Testing:

Testing a monolithic application involves comprehensive end-to-end testing, as all components are tightly integrated.

Resilience:

A failure in one component of a monolithic application can affect the entire application's availability and stability.

Microservices Architecture:

Structure:

Microservices architecture decomposes an application into smaller, loosely coupled services. Each service focuses on a specific business capability and can be developed and deployed independently.

Development:

Microservices are developed by small, autonomous teams using different technologies and programming languages. Each team can work independently on their service, enabling faster development cycles.

Scalability:

Microservices allow individual services to be scaled independently based on their specific demands. This scalability enables efficient resource utilization and the ability to handle high traffic loads by scaling only the necessary services.

Deployment:

Microservices are typically deployed independently. Updates and new releases can be rolled out for specific services without affecting the entire system.

Communication:

Microservices communicate with each other through lightweight protocols like HTTP or messaging systems. Services can make synchronous or asynchronous calls to exchange data and trigger actions.

Data Management:

Microservices can have their own dedicated databases, allowing them to manage their data independently. Each service is responsible for its data storage and ensures data consistency within its boundaries.

Testing:

Testing in a microservices architecture involves testing individual services as well as their interactions. Isolation of services enables more targeted and focused testing.

Resilience:

Microservices architecture promotes fault isolation. If one service fails, it does not bring down the entire system. Other services can continue to function independently, ensuring overall system resilience.
The choice between monolithic architecture and microservices architecture depends on various factors such as the complexity of the application, scalability requirements, team structure, and development agility. Monolithic architecture may be more suitable for smaller applications with simpler requirements, while microservices architecture offers greater flexibility, scalability, and resilience for large, complex, and rapidly evolving systems.

What are the key benefits of microservices architecture?

Microservices architecture offers several key benefits that make it a popular choice for designing and building applications. Here are some of the significant advantages of microservices architecture:

Scalability:

Microservices architecture allows individual services to be scaled independently based on their specific demands. This scalability enables efficient resource utilization and the ability to handle high traffic loads by scaling only the necessary services. It provides the flexibility to allocate resources where they are needed most, resulting in improved performance and cost-efficiency.

Agility and Faster Time-to-Market:

Microservices enable faster development cycles and deployment. Services can be developed, tested, and deployed independently by small, autonomous teams. This approach reduces dependencies and bottlenecks, allowing teams to work in parallel and deliver updates more frequently. It promotes agility, innovation, and faster time-to-market.

Technology Diversity:

Microservices architecture allows for flexibility in technology choices. Each service can be implemented using the most appropriate programming language, framework, or database for its specific requirements. This flexibility enables teams to leverage the strengths of different technologies and use the best-suited tool for each service, promoting innovation and enabling the adoption of new technologies.

Fault Isolation and Resilience:

Microservices architecture promotes fault isolation. If one service fails or experiences issues, it does not bring down the entire system. Other services can continue to function independently, ensuring overall system resilience. This fault isolation enhances the stability and availability of the system.

Independent Development and Deployment:

Microservices architecture enables independent development and deployment of services. Each service can be developed, tested, and deployed separately, without impacting other services. This independence allows teams to work autonomously and make independent decisions, resulting in faster development cycles and reduced coordination overhead.

Improved Maintainability:

Microservices architecture improves maintainability as services are smaller and more focused. It is easier to understand, modify, and test smaller codebases compared to a monolithic application. Developers can make changes to a specific service without affecting the entire system, reducing the risk of unintended consequences.

Team Scalability and Organizational Flexibility:

Microservices architecture aligns well with agile development methodologies and DevOps practices. It enables organizations to structure development teams around specific services, promoting smaller, cross-functional teams that can work autonomously and make independent decisions. This scalability in the organization allows for faster development cycles, faster onboarding of new team members, and easier management of large, complex projects.

Integration and Interoperability:

Microservices architecture facilitates integration with other services, applications, or third-party systems. Services expose well-defined APIs, making it easier to interact with them. This flexibility enables organizations to adopt a modular approach, integrate new services or replace existing ones as needed, and build systems that can evolve over time.

Microservices tools

There are various tools available that can assist in designing, developing, testing, deploying, and managing microservices-based applications. Here are some commonly used tools in the microservices ecosystem:

Service Development and Frameworks:

- **Spring Boot:** A popular Java framework that simplifies the development of microservices with features like auto-configuration, embedded servers, and dependency management.
- **Node.js:** A JavaScript runtime that is commonly used for developing lightweight and scalable microservices.
- **Express:** A web application framework for Node.js that simplifies the development of APIs and microservices.
- **Django:** A Python web framework that offers a solid foundation for building microservices in Python.
- **ASP.NET Core:** A cross-platform framework for building microservices using C# and .NET.

Service Communication and APIs:

- **RESTful APIs:** Representational State Transfer (REST) is a commonly used architectural style for building APIs that communicate between microservices.
- **gRPC:** A high-performance, open-source framework developed by Google for building remote procedure call (RPC) APIs that can be used for inter-service communication.
- **Apache Kafka:** A distributed streaming platform that can be used as a messaging system for asynchronous communication between microservices.

Containerization and Orchestration:

- **Docker:** A platform that allows you to package microservices into containers, ensuring consistent deployment across different environments.
- **Kubernetes:** An open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications, including microservices.

Service Discovery and Load Balancing:

- **Consul:** A service discovery and configuration tool that helps locate and manage services dynamically.
- **Netflix Eureka:** A service registry tool that enables service discovery and registration in a microservices environment.
- **NGINX:** A popular web server and load balancer that can be used for distributing traffic across microservices.

Monitoring and Observability:

- **Prometheus:** A monitoring and alerting toolkit that collects metrics from microservices and provides a powerful querying language for analysis.
- **Grafana:** A data visualization tool that can be used with Prometheus to create dashboards and visualize metrics from microservices.

API Gateway:

- **Netflix Zuul:** A gateway service that handles routing, load balancing, and authentication for requests made to microservices.
- **Kong:** An open-source API gateway that can be used to manage and secure APIs in a microservices architecture. These are just a few examples of tools available in the microservices ecosystem. The choice of tools depends on the specific requirements, programming languages, and technologies used in your microservices architecture.

Challenges associated with microservices architecture.

While microservices architecture offers several benefits, it also introduces certain challenges that need to be considered and addressed. Here are some common challenges associated with microservices architecture:

1. Service Coordination:

As the number of services increases, managing inter-service communication and coordination becomes more complex. Service interactions, such as synchronous and asynchronous communication, event-driven architectures, and data consistency across services, need to be carefully designed and implemented.

2. Data Management:

Data management becomes more challenging in a microservices architecture. Each service may have its own database, leading to data duplication and potential data consistency issues. Implementing strategies for data synchronization, eventual consistency, and maintaining data integrity across services is crucial.

3. Distributed System Complexity:

Microservices architecture introduces the complexity of managing a distributed system. Issues such as network latency, partial failures, message delivery guarantees, and distributed transactions need to be carefully addressed to ensure the overall system's reliability and stability.

4. Operational Overhead:

With multiple services running independently, operational tasks such as monitoring, logging, deployment, and scaling become more complex. Implementing effective DevOps practices, automation, and monitoring solutions is necessary to manage the operational overhead efficiently.

5. Service Discovery and Orchestration:

As the number of services grows, service discovery and orchestration become crucial. Tools and mechanisms for service registration, discovery, load balancing, and routing need to be implemented to ensure seamless communication and scalability.

6. Testing Complexity:

Testing a microservices architecture requires more comprehensive and sophisticated testing strategies. Apart from unit testing individual services, integration testing, end-to-end testing, and testing service interactions become important. Setting up test environments and data management for testing can be challenging.

7. Organizational and Team Alignment:

Microservices architecture requires teams to be organized around specific services or functional areas. Coordinating and aligning the efforts of multiple teams, managing inter-team dependencies, and maintaining consistent development practices across teams can be challenging.

8. Operational Monitoring and Observability:

Monitoring and gaining visibility into a distributed system can be more complex in a microservices architecture. Capturing and analyzing logs, metrics, and traces across multiple services becomes crucial for troubleshooting, performance optimization, and ensuring system health.

9. Deployment Complexity:

Deploying and managing multiple services, potentially using different technologies and versions, can be complex. Adopting containerization and orchestration tools like Docker and Kubernetes can help streamline the deployment process, but they also introduce their own learning curve and operational challenges.

10. Increased Complexity and Learning Curve:

Microservices architecture introduces additional complexity compared to a monolithic architecture. The learning curve for developers, architects, and operations teams may be steeper, requiring a thorough understanding of distributed systems, service design principles, and new tools and technologies.

Addressing these challenges requires careful planning, architectural design, and adoption of best practices. It's important to weigh the benefits and challenges of microservices architecture to ensure it aligns with the specific requirements and capabilities of your organization.

Microservices architecture examples

Microservices architecture is widely adopted by various organizations across different industries. Here are some examples of companies that have implemented microservices architecture:

1. Netflix:

Netflix is one of the well-known examples of microservices architecture. Their architecture is composed of hundreds of microservices that handle different functionalities such as user authentication, recommendation engine, content delivery, and billing. Microservices enable Netflix to scale their system, deliver personalized experiences, and continuously roll out new features.

2. Amazon:

Amazon, one of the largest e-commerce companies, utilizes microservices architecture extensively. Each service within their architecture, such as product catalog, shopping cart, payment processing, and inventory management, operates independently and communicates through well-defined APIs. This allows Amazon to handle high traffic loads, enable rapid development and deployment, and provide a seamless shopping experience.

3. Uber:

Uber, the ride-hailing platform, has adopted a microservices architecture to power its complex system. Microservices handle different aspects like user management, geolocation, pricing, ride dispatching, and payment processing. The decoupled nature of microservices allows Uber to scale its platform globally, integrate with various third-party services, and deliver real-time experiences to users.

4. Spotify:

Spotify, the popular music streaming platform, utilizes microservices architecture to deliver a personalized music experience to millions of users. Microservices handle functionalities such as user authentication, music recommendation, playlist management, and social sharing. Spotify's microservices architecture allows them to scale their platform, continuously deliver new features, and handle a vast catalog of music.

5. Airbnb:

Airbnb, the online marketplace for accommodation rentals, has embraced microservices architecture. Their architecture consists of various microservices responsible for user management, search functionality, booking, payment processing, and reviews. Microservices enable Airbnb to handle many listings, provide a seamless booking experience, and integrate with external services.

These are just a few examples of companies that have adopted microservices architecture to build scalable, resilient, and innovative systems. Microservices architecture has become a popular choice for organizations aiming to build complex, highly scalable applications that can evolve rapidly and deliver a better user experience.

Microservices best practices

When implementing microservices architecture, it is important to follow best practices to ensure the success and effectiveness of the architecture. Here are some commonly recommended best practices for microservices:

1. Single Responsibility Principle:

Each microservice should have a single responsibility, focusing on a specific business capability. This helps to keep services small, maintainable, and easily replaceable.

2. Loose Coupling:

Microservices should be loosely coupled, meaning they should have minimal dependencies on other services. Loose coupling enables independent development, deployment, and scaling of services.

3. API Design and Documentation:

Clearly define and document the APIs exposed by each microservice. Well-designed APIs facilitate communication and integration between services and make it easier for other developers to understand and use the services.

4. Decentralized Data Management:

Each microservice should have its own private database or data store, ensuring data autonomy. Decentralized data management minimizes data coupling and enables services to make autonomous data-related decisions.

5. Asynchronous Communication:

Utilize asynchronous communication patterns, such as event-driven architecture or message queues, to decouple services and enable scalability. Asynchronous communication allows services to process events or messages at their own pace, improving system resilience and responsiveness.

6. Fault Isolation and Resilience:

Design services to be resilient to failures. Implement fault isolation mechanisms so that if one service fails, it does not bring down the entire system. Implement retry strategies, circuit breakers, and fallback mechanisms to handle service failures gracefully.

7. Continuous Integration and Deployment:

Adopt CI/CD (Continuous Integration/Continuous Deployment) practices to automate the build, test, and deployment processes. This helps ensure rapid and reliable delivery of changes to microservices.

8. Containerization and Orchestration:

Use containerization technologies like Docker to package and deploy microservices consistently across different environments. Container orchestration platforms like Kubernetes provide capabilities for automated scaling, service discovery, and deployment management.

9. Monitoring and Observability:

Implement robust monitoring and observability practices to gain insights into the performance, health, and behavior of microservices. Use tools like distributed tracing, logging, and metrics collection to effectively monitor and debug the system.

10. Team Collaboration and Autonomy:

Encourage cross-functional teams with ownership of specific microservices. Foster collaboration, communication, and knowledge sharing across teams to ensure a cohesive and coordinated approach to development and operations.

11. Security and Access Control:

Implement security measures such as authentication, authorization, and encryption to protect microservices and sensitive data. Apply security best practices at the service level and enforce access control through API gateways or service mesh.

12. Testing Strategies:

Implement comprehensive testing strategies, including unit testing, integration testing, and end-to-end testing. Consider using contract testing to verify the compatibility between services. Implement automated testing to ensure the reliability and quality of microservices.

13. Incremental Iterative Development:

Adopt an iterative approach to developing and evolving microservices. Start with a minimum viable product and gradually add new features and functionalities. This enables faster time-to-market, continuous feedback, and the ability to adapt to changing requirements.
These best practices serve as guidelines to design, develop, and operate microservices effectively. It's important to consider the specific needs and constraints of your project and adapt these practices accordingly.

Use Cases of Microservices Architecture

Microservices architecture is applicable to a wide range of use cases, particularly in complex and scalable systems. Here are some common use cases where microservices architecture is beneficial:

- **E-commerce Platforms:** Microservices architecture can be used in e-commerce platforms to handle various functionalities such as product catalog, shopping cart, order management, payment processing, user reviews, and recommendation engines. Microservices allow independent scaling of different components, facilitating high traffic loads, personalized experiences, and seamless integrations with external systems.
- **Media Streaming Services:** Services like Netflix, Hulu, and Spotify leverage microservices architecture to deliver media content to millions of users. Microservices enable efficient content delivery, personalized recommendations, user management, and billing. This architecture facilitates scalability, fault tolerance, and rapid feature deployment.
- **Travel and Hospitality Applications:** Microservices architecture is well-suited for travel and hospitality applications, including online booking platforms, hotel reservation systems, and travel management systems. Different microservices can handle functions like booking management, search and filtering, user profiles, payment processing, and

reviews. Microservices enable flexible integration with external services like airlines, hotels, and payment gateways.

- **Financial Services and Banking:** Microservices architecture is applicable to financial services, banking applications, and payment systems. Services can be built to handle functionalities such as account management, transaction processing, fraud detection, risk assessment, and compliance. Microservices allow for easy integration with legacy systems, scalability to handle high transaction volumes, and improved security.
- **Internet of Things (IoT):** IoT applications often involve a network of interconnected devices and sensors generating vast amounts of data. Microservices architecture can handle the challenges of IoT systems by providing services for data ingestion, real-time processing, analytics, device management, and integration with external systems. Microservices enable scalability, flexibility, and support for various types of devices and protocols.
- **Social Media Platforms:** Social media platforms, like Facebook and Twitter, can leverage microservices architecture to handle diverse functionalities such as user profiles, posts, timelines, notifications, messaging, and content moderation. Microservices enable efficient handling of high user concurrency, personalized experiences, and integrations with external APIs and services.
- **Gaming and Entertainment:** Microservices architecture is suitable for gaming and entertainment applications that require real-time interactions, multiplayer capabilities, and content delivery. Different microservices can handle matchmaking, game logic, player management, leaderboards, virtual currency, and content distribution. Microservices provide scalability, fault tolerance, and the ability to roll out new game features or updates quickly.
- **Healthcare and Telemedicine:** Microservices architecture can be applied to healthcare applications, electronic health records (EHR), telemedicine platforms, and health monitoring systems. Microservices can handle functionalities like patient management, appointment scheduling, data analysis, secure communication, and integration with medical devices. Microservices enable flexibility, interoperability, and compliance with healthcare standards. These are just a few examples of how microservices architecture can be applied across various industries and use cases. The flexibility, scalability, and independent development and deployment capabilities of microservices make it a suitable choice for building complex, distributed systems.

Conclusion About Microservices Architecture

In conclusion, microservices architecture has emerged as a popular approach for designing and building complex, scalable, and maintainable software systems. It offers a set of principles and practices that enable the development of loosely coupled and independently deployable services.

Microservices architecture brings several benefits, including:

Scalability: Microservices allow individual services to scale independently based on demand, enabling systems to handle high traffic and accommodate growth more effectively.

Flexibility and Agility: Microservices promote independent development and deployment of services, enabling faster iterations, continuous delivery, and the ability to adopt new technologies and frameworks.

Fault Isolation: Services in a microservices architecture are decoupled, allowing failures in one service to be isolated and not impact the entire system, leading to improved fault tolerance and resilience.

Easier Maintenance and Evolution: Each microservice has a well-defined responsibility, making it easier to understand, modify, and maintain. Microservices can be updated or replaced without impacting other services.

Technology Diversity: Microservices architecture allows different services to be built using different technologies, programming languages, and frameworks, enabling teams to choose the most suitable technology for each specific service. However, adopting microservices architecture also comes with challenges that need to be addressed, such as service coordination, data management, distributed system complexity, and operational overhead. These challenges require careful planning, architectural design, and the adoption of best practices to ensure the success of microservices-based systems. It is important to consider the specific requirements, complexity, and capabilities of your project before deciding to adopt microservices architecture. While microservices offer benefits, they may not be suitable for every application or organization. It is crucial to evaluate the trade-offs and determine if the benefits outweigh the associated complexities. Ultimately, the success of microservices architecture depends on proper design, implementation, testing, and operational practices. By following best practices, leveraging appropriate tools and technologies, and considering the specific needs of your project, you can harness the power of microservices to build scalable, resilient, and agile software systems.

2

Microservices Design Patterns Used in Software Industry

Overview

In this chapter, we explore the Microservices Design Pattern and explore various patterns including Service Registry, API Gateway, Circuit Breaker, Bulkhead, Event Sourcing, Command Query Responsibility Segregation, Saga, Choreography vs Orchestration, Database Sharding, Service Mesh, Sidecar Pattern, and Immutable Infrastructure, concluding with a summary of key insights and best practices.

Microservices Design Pattern Introduction

Explore the foundational elements of microservices architecture through a comprehensive understanding of crucial design patterns. From the fundamental Service Registry and API Gateway, enabling seamless communication, to resilience-focused patterns like Circuit Breaker and Bulkhead, each pattern is a key building block for scalable and fault-tolerant systems. Dive into Event Sourcing and CQRS for efficient data handling, Saga for managing complex workflows, and the intricacies of choreography versus orchestration. Discover Database Sharding for optimized storage, Service Mesh for streamlined communication, and the Sidecar pattern for modular enhancements. Finally, grasp the concept of Immutable Infrastructure for consistent and reproducible deployments. Unravel the power of these design patterns to craft robust and efficient microservices architectures that scale with ease and withstand challenges.

Service Registry

The Service Registry acts as a centralized database where individual microservices register their locations and current availability. This registration enables other services within the ecosystem to dynamically discover and communicate with each other. Implementations such as Consul, etcd, or Netflix Eureka are frequently employed for this purpose, providing tools for service registration and discovery. The primary advantages of utilizing a Service Registry include the ability to add or remove services without impacting others, enabling seamless load balancing across various service instances, and supporting failover mechanisms, which enhance system resilience by redirecting traffic in case of service failures. Overall, the Service Registry fundamentally streamlines communication and coordination among microservices within a distributed architecture.

Description: A central repository where microservices register their location and availability, enabling dynamic discovery and communication.

Implementation: Tools like Consul, etcd, or Netflix Eureka are commonly used for service registration and discovery.

Advantages: Allows services to be added or removed without affecting other services, facilitates load balancing, and supports failover mechanisms.

API Gateway

An API Gateway is a centralized entry point for clients to access multiple services within a microservices architecture. It serves as a single point of contact for external clients, aggregating various APIs, and providing a unified interface to interact with these services.

Key functions of an API Gateway include:

- **Routing:** Directing incoming requests from clients to the appropriate microservice based on the requested endpoint or functionality.
- **Authentication and Authorization:** Handling user authentication and ensuring that only authorized users or systems can access specific services or endpoints.
- **Load Balancing:** Distributing incoming requests across multiple instances of a service to ensure efficient resource utilization and scalability.
- **Caching:** Storing frequently accessed data or responses to reduce latency and improve overall system performance.
- **Protocol Translation:** Converting incoming requests from clients into formats or protocols that are compatible with the internal services.
- **Monitoring and Analytics:** Collecting data on API usage, performance metrics, and error tracking to facilitate monitoring and optimization of the system.

By consolidating these functionalities into a single component, the API Gateway simplifies client access, enhances security by centralizing authentication, and enables better management and control of the microservices architecture.

Description: Acts as a single-entry point for clients, providing a unified interface to multiple microservices.

Functionality: Handles authentication, routing, load balancing, caching, and API composition.

Benefits: Simplifies client access, improves security by centralizing authentication, and assists in versioning and monitoring.

Circuit Breaker

A Circuit Breaker is a design pattern used in distributed systems, including microservices architectures, to enhance resilience and prevent cascading failures. It functions similarly to an electrical circuit breaker: when there's a fault or failure in a service, the Circuit Breaker "opens," preventing further requests from reaching the failing service for a specified duration.
The primary purpose of a Circuit Breaker is to manage the flow of requests and protect the system from continuously attempting to use a service that's experiencing issues. When the Circuit Breaker detects that a service is failing (due to timeouts, errors, or other predefined metrics), it temporarily stops forwarding requests to that service.

Key features of a Circuit Breaker:

- **Monitoring:** Constantly observes the health and responsiveness of services it's connected to.
- **Thresholds:** Defines thresholds for errors, timeouts, or other metrics that trigger the Circuit Breaker to open.
- **States:** Typically operates in three states: closed (normal operation), open (service failure detected), and half-open (testing if the service has recovered).
- **Fallback Mechanism:** Provides a fallback response or alternative behaviour when the Circuit Breaker is open, allowing the system to gracefully handle service unavailability.

Benefits of using a Circuit Breaker:

- **Fault Isolation:** Prevents the failure of one service from affecting others by stopping the propagation of requests.
- **Resilience:** Improves the overall stability of the system by minimizing the impact of service failures.
- **Performance:** Reduces the load on a failing service, preventing system overload, and maintaining responsiveness.
  In essence, the Circuit Breaker pattern enhances the fault tolerance of a system by introducing a mechanism to handle and recover from service failures, promoting stability and reliability in distributed architectures.

Purpose: Prevents the continuous flow of requests to a failing service, thus avoiding system overload.

Mechanism: Monitors for failures and, upon detecting a threshold, stops sending requests temporarily or redirects to a fallback mechanism.

Advantages: Enhances system resilience by isolating failures and prevents cascading failures across services.

4. Bulkhead

The Bulkhead pattern, borrowed from naval architecture, is a design principle applied in software engineering and microservices architecture to enhance system resilience and prevent failures from propagating across different components.

In a literal sense, bulkheads on ships are partitions that create separate compartments to prevent the entire vessel from sinking if one section gets breached. Similarly, in software, the Bulkhead pattern involves isolating components or resources into separate compartments or pools to contain failures and limit their impact on other parts of the system.

The main concept involves segregating various resources, such as threads, connections, or services, into distinct compartments, ensuring that if one of these resources fails or becomes overwhelmed, it doesn't affect the functionality of other resources. By providing boundaries and containment, the Bulkhead pattern aims to enhance fault tolerance and system stability.

For instance, in a microservices architecture, isolating services into different pools or containers can prevent failures in one service from causing a domino effect and affecting the performance or availability of other services. It's akin to creating boundaries that limit the spread of problems, making it easier to manage failures and maintain overall system reliability.

Implementing the Bulkhead pattern involves careful consideration of resource allocation, setting limits, and defining boundaries to ensure that failures or overloads in one part of the system do not compromise the integrity or functionality of the entire system. Overall, the Bulkhead pattern plays a crucial role in building robust and resilient software architectures, especially in distributed systems.

Definition: Divides components of a system into separate pools to prevent failures in one part from affecting the entire system.

Implementation: Can be achieved by running services in separate containers or using thread pools for isolation.

Advantages: Improves fault tolerance, ensuring that failures in one part of the system don’t bring down the entire system.

Event Sourcing

Event Sourcing is a design pattern used in software development, particularly in building complex and scalable systems, where the state of an application is determined by a sequence of events rather than the current state itself.

In traditional systems, the current state of an application is stored and updated directly. However, in Event Sourcing, every change to the state of the application is captured as an immutable event. These events are stored in an event log or append-only data store, forming a chronological record of actions or occurrences that have affected the application's state.

By storing events rather than the current state, Event Sourcing allows the entire history of changes to be retained. This history can be replayed to reconstruct the current state at any point in time. This ability to replay events provides several advantages:

- **Auditability and Compliance:** It enables detailed auditing as every state transition is recorded as an event, facilitating traceability and compliance with regulations.
- **Temporal Queries and Analysis:** The complete history of events allows querying the state of the system at any specific time, supporting temporal queries and analysis.

- **Debugging and Rollbacks:** It simplifies debugging by allowing developers to trace back through events to identify the source of issues. Additionally, it enables easy rollbacks to previous states by replaying events.
- **Scalability and Decoupling:** Events are a natural way to distribute work across different components or microservices within a system, allowing for scalability and loosely coupled architectures.

However, implementing Event Sourcing introduces complexity, especially when dealing with event storage, event versioning, and event replay. It requires careful design and consideration of how events are captured, stored, and utilized within the system.

Event Sourcing offers a powerful mechanism to manage state changes, providing a historical record of events that can be leveraged for various purposes, including auditing, analytics, and reconstructing system states.

Concept: Stores the state of an application as a sequence of events rather than the current state, enabling reconstruction of past states and auditing.

Usage: Helps in scaling, maintaining history, and enables complex data analysis based on historical events.

Benefits: Increases reliability, supports rollback, and provides a detailed

Command Query Responsibility Segregation (CQRS)

Command Query Responsibility Segregation (CQRS) is a design pattern that advocates separating the operations that read data (queries) from those that modify data (commands) within an application or system.
In traditional architectures, the same model is often used for both read and write operations. However, CQRS proposes using separate models—one optimized for reading data and another for writing data. This separation enables each model to be tailored to its specific purpose, optimizing performance and scalability.

Key elements of CQRS:

- **Commands:** Represent actions that change the state of the system (e.g., creating, updating, or deleting data). Commands are responsible for modifying the application's state.
- **Queries:** Involve retrieving data without modifying the application's state. Queries provide a way to read data from the system. By decoupling read and write operations, CQRS allows each model to be optimized independently based on its unique requirements:
- **Write Model (Command):** Optimized for handling complex business logic, validation, and data modifications. It's focused on ensuring data consistency and integrity.
- **Read Model (Query):** Optimized for efficient and scalable data retrieval. This model is denormalized and tailored to suit specific query requirements, improving read performance.

CQRS can be implemented in various ways, such as using different databases for read and write operations, employing separate services or components for handling commands and queries, or utilizing specialized data structures for queries.

While CQRS offers benefits like performance optimization, scalability, and flexibility in handling read and write operations independently, it also introduces complexity due to the need for synchronization between the two models and maintaining consistency between them.

CQRS is a powerful pattern that enables the creation of systems that efficiently handle complex operations by separating the responsibilities of reading and writing data.

Principle: Segregates read and write operations, using separate models to optimize performance for each operation type.

Implementation: Uses different data models and storage for read and write operations, often asynchronously synchronized.

Advantages: Optimizes read and write operations independently, improving scalability and performance.

Saga Pattern

The Saga Pattern is a design pattern used in distributed systems, particularly in microservices architectures, to manage long-lived transactions or workflows that span multiple services. It helps ensure data consistency across these distributed operations.

In complex business processes, a single transaction might involve multiple steps across different services. The Saga Pattern addresses the challenges of maintaining consistency in such distributed transactions by breaking them down into a series of smaller, more manageable steps called "saga steps."

Key aspects of the Saga Pattern:

- **Atomic Steps:** Each saga step represents a single unit of work within the larger transaction. These steps are designed to be individually reversible or compensatable.
- **Compensation Actions:** For any failed or partially completed saga steps, compensation actions are defined to undo the effects of the preceding steps and restore system consistency.
- **Coordinator or Orchestrator:** A central entity, often referred to as a saga coordinator or orchestrator, manages the execution of saga steps. It coordinates the sequence of steps and handles both successful completion and failure scenarios.
- **Eventual Consistency:** The Saga Pattern embraces the concept of eventual consistency, meaning that while individual steps may execute successfully, the entire transaction might not be immediately consistent. However, compensation actions are triggered to maintain overall system integrity.

By breaking down a complex transaction into smaller, individually reversible steps and incorporating compensating actions, the Saga Pattern enables systems to maintain consistency and recover from failures or partial successes. This approach helps prevent distributed transactions from causing inconsistencies across multiple services.

Implementing the Saga Pattern requires careful planning, especially in designing and managing saga steps, defining compensating actions for each step, and handling potential failures and timeouts. Additionally, it demands robust communication and coordination mechanisms between services to ensure proper execution of the saga.

The Saga Pattern is valuable in scenarios where traditional ACID (Atomicity, Consistency, Isolation, Durability) transactions are challenging or impossible to implement due to the distributed nature of the system. It provides a means to handle long-running transactions across multiple services while maintaining system integrity and resilience.

Purpose: Manages long-running transactions or workflows by breaking them into a series of smaller, more manageable steps called "saga steps."

Functionality: Orchestrates the series of steps, often including compensating actions to handle failures.

Benefits: Ensures consistency in distributed transactions and handles failures gracefully.

Choreography vs. Orchestration

Choreography and Orchestration are two approaches used in designing the coordination and communication between multiple services in a distributed system, especially within microservices architectures.

Choreography:

- **Decentralized Communication:** In choreography, services communicate directly with each other through events they produce or consume.
- **Autonomous Services:** Each service is responsible for reacting to events it observes and triggering actions based on those events.
- **Loose Coupling:** Services are loosely coupled as they interact based on predefined events without a central coordinator.
- **Complexity Distribution:** While it offers more autonomy to services, it might make system-wide understanding and management more challenging as the interactions can be distributed across services.

Orchestration:

- **Centralized Control:** In orchestration, a central controller or orchestrator coordinates the interactions between services.
- **Defined Workflow:** The orchestrator defines the sequence of steps or actions that need to be executed by different services.
- **Clear Coordination:** Offers a clear and centralized view of the workflow, making it easier to manage and monitor.
- **Tighter Control:** Provides more control but can create a single point of failure and could lead to a more tightly coupled system.

Choosing between choreography and orchestration often depends on the complexity of the system and the level of control and visibility required:

Choreography is suitable for scenarios where services are more autonomous, and a decentralized approach to communication is preferred. It works well when services need to react to events independently.

Orchestration is suitable when there's a need for a centralized view and control over the workflow. It's beneficial for managing complex, multi-step processes involving multiple services, providing a clearer overview of the entire process.

Both approaches have their merits and trade-offs, and the choice between them depends on factors such as system complexity, communication patterns, and the level of coordination and control needed in managing interactions between services.

Choreography: Services communicate directly through events they produce or consume, promoting decentralized interactions.

Orchestration: Involves a central controller that manages interactions between services, determining the flow of communication.

Selection: Depends on complexity, clarity, and control needed in managing interactions between services.

Database Sharding

Sharding is a database partitioning technique used to horizontally divide and store data across multiple servers in a distributed system. This method aims to improve performance, scalability, and efficiency by spreading the dataset over several shards (smaller databases).

Here's an overview of the key aspects:

- **Data Distribution:** Sharding involves breaking up a large database into smaller, more manageable parts called shards. Each shard contains a subset of the data and is stored on a separate server or node within a network.
- **Shard Key:** The shard key is a crucial element that determines how data is distributed across shards. It's often a field or a set of fields in a database record. Properly choosing the shard key is essential for even distribution of data among shards and optimizing query performance.
- **Scalability:** Sharding provides horizontal scalability, allowing the database to handle increased load by adding more shards and distributing the data across them. This makes it easier to scale the system as the amount of data or the number of users grows. Performance: With sharding, queries can potentially be faster since they can be executed on multiple shards simultaneously. However, poorly chosen shard keys or complex query patterns can lead to performance issues.
- **Complexity and Maintenance:** Sharding introduces complexity in database architecture and operations. It requires careful planning and implementation to ensure data consistency, efficient querying, and fault tolerance. Maintenance tasks like backup, recovery, and shard rebalancing are also more complex in a sharded environment.
- **Fault Tolerance:** Sharding can provide increased fault tolerance. If one shard or server fails, other shards can continue operating independently, reducing the risk of complete system downtime.
- **Consistency and Transactions:** Maintaining consistency across shards can be challenging, especially in distributed transactions that involve multiple shards. Ensuring data consistency often requires additional effort and consideration in a sharded environment.

Different databases and systems have their own approaches to sharding. Some databases, like MongoDB and Cassandra, have built-in support for sharding, offering tools and features to manage sharded environments more effectively. It's important to carefully design and plan the shard architecture based on specific requirements and anticipated growth to reap the benefits of sharding while mitigating its complexities.

Definition: Splits a large database into smaller shards to distribute data, improve performance, and enable horizontal scaling.

Approach: Can be done based on various criteria like range-based, hash-based, or key-based sharding.

Advantages: Enhances scalability and performance by distributing data across multiple databases.

10. Service Mesh

A service mesh is a dedicated infrastructure layer built to facilitate communication between microservices in a distributed architecture. It's designed to manage the complexities of service-to-service communication in modern applications.

Microservices:

Modern applications are often broken down into smaller, independent services known as microservices. Each microservice handles specific functions, promoting modularity and scalability in application development.

Challenges:

As the number of microservices grows, managing their communication becomes complex. Challenges include:

- **Service Discovery:** Microservices need to find and communicate with each other.
- **Load Balancing:** Traffic distribution among services for optimal performance.
- **Security:** Encryption, authentication, and authorization between services.
- **Monitoring and Observability:** Understanding and managing the health and performance of services.
- **Resilience and Fault Tolerance:** Handling failures and maintaining application stability.

Enter Service Mesh:

A service mesh addresses these challenges by providing a dedicated infrastructure layer responsible for handling communication between services. It typically involves:

- **Proxy-Based Communication:** Each service instance is accompanied by a lightweight proxy (like Envoy, Linkerd, Istio) that manages incoming and outgoing traffic.
- **Service Discovery and Load Balancing:** Proxies assist in service discovery and intelligently distribute traffic among instances.
- **Security and Policies:** Encryption, authentication, and authorization policies are enforced at the proxy level.
- **Observability and Monitoring:** Proxies collect metrics, logs, and tracing information for visibility into the communication between services.
- **Resilience Features:** Service meshes often provide mechanisms for retry logic, timeouts, and circuit breaking to ensure system robustness.

Benefits:

- **Simplified Communication:** Developers can focus on coding services without worrying about network-related complexities.
- **Uniform Policies:** Security, monitoring, and other policies can be uniformly applied and managed.
- **Enhanced Observability:** Better insights into service interactions for debugging and optimization.
- **Improved Resilience:** Mechanisms to handle failures and maintain system stability. Service meshes have become a crucial part of cloud-native applications, aiding in managing the intricacies of microservices-based architectures efficiently.
- **Functionality:** Manages communication between microservices, handling tasks like service discovery, load balancing, encryption, and authentication.
- **Implementation:** Tools like Istio, Linkerd, or Envoy are used to create a service mesh.
- **Benefits:** Offloads communication-related functionalities from application code, simplifying service-to-service communication.

11. Sidecar Pattern

The Sidecar Pattern is an architectural design pattern used in software development. It involves attaching additional responsibilities or functionalities to an application by deploying it as a separate process or container, known as the "sidecar." This approach allows the main application to focus on its primary function while delegating ancillary tasks to the attached sidecar. Imagine a scenario where an application needs functionalities like logging, monitoring, security, or even language translation. Instead of integrating these functionalities directly into the main application codebase, you can create separate modules or services for each of these tasks and deploy them alongside the primary application as sidecars.

Key aspects of the Sidecar Pattern include:

- **Modularity:** Each functionality or service (sidecar) is encapsulated and operates independently of the main application.
- **Separation of Concerns:** The primary application's core logic remains clean and focused, as it offloads secondary tasks to the sidecar components.
- **Flexibility and Scalability:** Sidecars can be added, updated, or removed without impacting the core application, providing flexibility and scalability.
- **Isolation:** Sidecars can run in their own containers or processes, ensuring isolation and fault tolerance. They can also communicate with the main application via defined interfaces or protocols.

For instance, in a microservices architecture, each microservice can have its own sidecar handling tasks such as logging, monitoring, authentication, or encryption. This separation enables teams to develop and maintain these functionalities independently, easing the overall management and evolution of the system.

However, implementing the Sidecar Pattern introduces complexity in managing multiple components and their interactions, potentially leading to increased resource consumption due to additional processes or containers running alongside the main application.

The Sidecar Pattern offers a way to extend functionalities without directly modifying the core application, making it a valuable architectural choice in various distributed systems and microservices-based environments.

Explanation: Attaches auxiliary services to the main application, running in separate containers but closely related and providing additional functionalities like logging, monitoring, or security.

Advantages: Simplifies the addition of functionalities without directly impacting the main application, aiding in modularization.

12. Immutable Infrastructure

Immutable infrastructure refers to a type of infrastructure design where components, such as servers or containers, are replaced rather than modified or updated after their deployment. In this approach, once a component is created and deployed, it remains unchanged throughout its lifecycle. Any updates or changes result in the creation of a new, entirely fresh component, rather than modifying an existing one.

This methodology offers several benefits:

- **Consistency and Reliability:** Immutable infrastructure ensures that all instances are identical, reducing configuration drift and potential errors that might arise from inconsistent environments.

- **Security:** Since components aren't altered after deployment, there's a reduced risk of unauthorized changes or vulnerabilities introduced through modifications.
- **Easier Rollbacks:** If an issue arises with a new deployment, reverting to a previous version is straightforward by redeploying the older, known-working version.
- **Scalability:** Scaling is simplified as new instances are created with updated configurations, rather than modifying existing ones, enabling easier horizontal scaling.

However, adopting an immutable infrastructure approach requires tools and processes that support this paradigm, such as containerization technologies like Docker, orchestration tools like Kubernetes, version control systems, and automation frameworks. Additionally, it might necessitate a mindset shift in how software updates and infrastructure changes are managed, as well as consideration for storage and data persistence strategies.

Concept: Treats infrastructure components as unchangeable, replacing or recreating them instead of modifying existing ones.

Benefits: Ensures consistency and reliability by reducing configuration drift, facilitating easier rollbacks, and improving reproducibility.

Each design pattern addresses specific challenges in microservices architecture, providing strategies and best practices to build resilient, scalable, and maintainable systems.

Conclusion

Design patterns in microservices architecture offer a toolkit of solutions to address the complexities and challenges of building distributed systems. By implementing these patterns, developers can create more resilient, scalable, and maintainable applications. Here's a brief summary of their impact:

- **Flexibility and Scalability:** Design patterns enable services to be added, removed, or updated independently without disrupting the entire system, fostering flexibility and scalability.
- **Resilience and Fault Isolation:** Patterns like circuit breakers, bulkheads, and the saga pattern ensure that failures in one part of the system don’t lead to catastrophic failures across the entire architecture, enhancing resilience and fault isolation.
- **Efficient Communication:** Service registries, API gateways, service meshes, and choreography/orchestration patterns facilitate efficient communication between microservices, reducing complexity and streamlining interactions.
- **Data Management:** Techniques like database sharding and CQRS optimize data handling, improving performance, and enabling better scalability.
- **Monitoring and Maintenance:** Sidecar patterns and immutable infrastructure practices simplify monitoring, logging, and maintenance tasks, aiding in better management of the system.

These design patterns are instrumental in mitigating the challenges posed by microservices architecture. They provide a structured approach to building robust systems that can adapt, scale, and maintain operational integrity in the dynamic world of distributed computing. Understanding and appropriately applying these patterns contribute significantly to the success of microservices-based applications.

3

Building Microservices Using 3 Tier Architecture

Overview

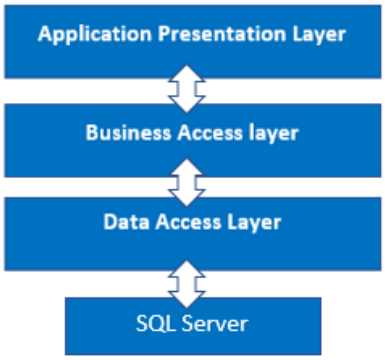
In this chapter, we introduce the 3-tier architecture and its Project Structure, detailing the Presentation Layer (PL), Business Access Layer (BAL), and Data Access Layer (DAL). We guide you on Adding References to the Project, explore Contacts, Data, Migrations, Models, and Repositories within the DAL, and discuss Services in the BAL. We highlight the Advantages and Disadvantages of 3-Tier Architecture and conclude with the Complete GitHub Project URL. Prepare for a comprehensive guide to effectively implementing and utilizing 3-tier architecture in your projects.

Introduction

In this chapter, we'll look at three-tier architecture and how to incorporate the Data Access Layer and Business Access Layer into a project, as well as how these layers interact.

Project Structure

We use a three-tier architecture in this project, with a data access layer, a business access layer, and an application presentation layer.



Presentation Layer (PL)

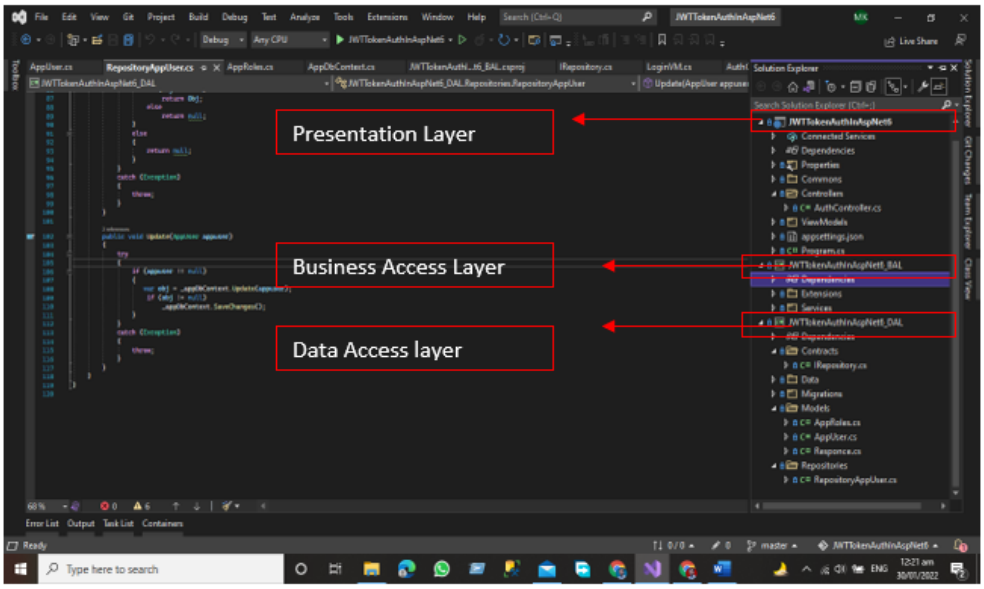
The Presentation layer is the top-most layer of the 3-tier architecture, and its major role is to display the results to the user, or to put it another way, to present the data that we acquire from the business access layer and offer the results to the front-end user.

Business Access Layer (BAL)

The logic layer interacts with the data access layer and the presentation layer to process the activities that lead to logical decisions and assessments. This layer's primary job is to process data between other layers.

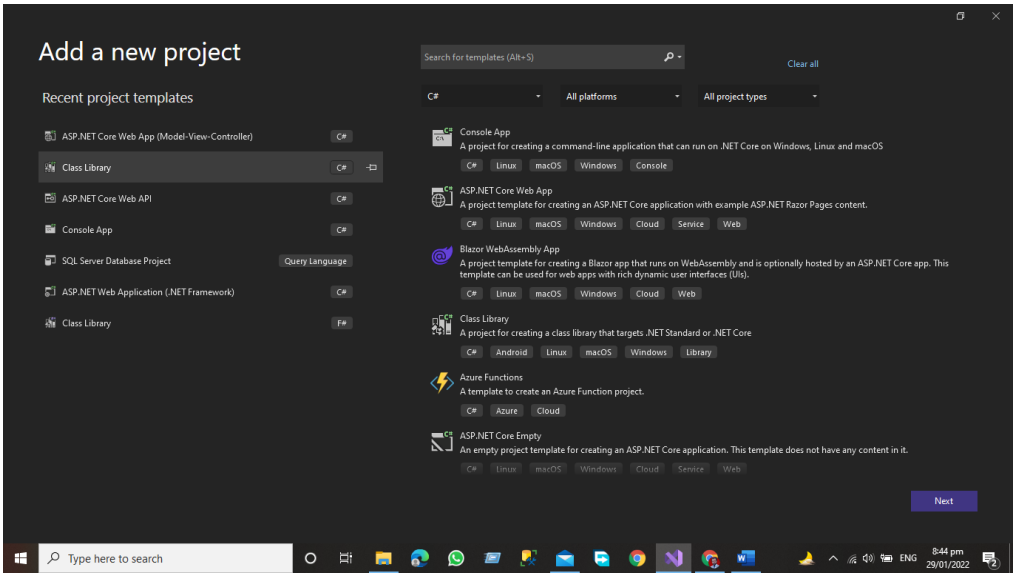
Data Access Layer (DAL)

The main function of this layer is to access and store the data from the database and the process of the data to business access layer data goes to the presentation layer against user request.

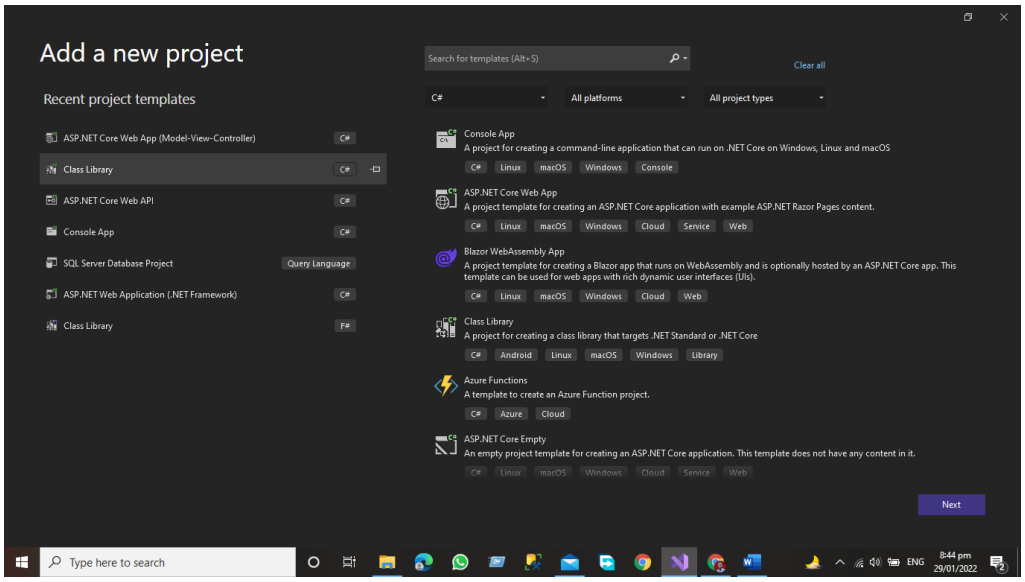


Steps to Follow for configuring these layers.

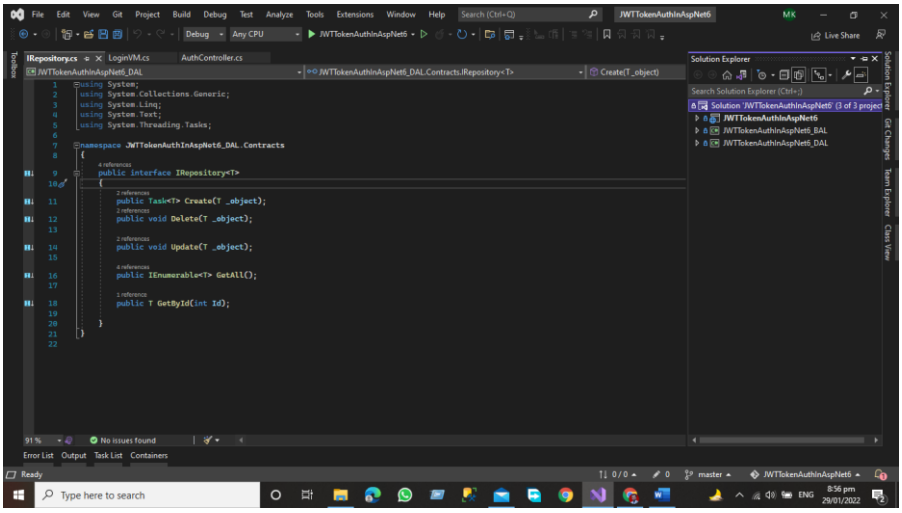
- Add the Class Library project of Asp.net for Data Access Layer
- Right Click on the project and then go to the add the new project window and then add the Asp.net Core class library project.



- After Adding the Data Access layer project now, we will add the Business access layer folder
- Add the Class library project of Asp.Net Core for Business Access
- Right Click on the project and then go to the add the new project window and then add the Asp.net Core class library project.



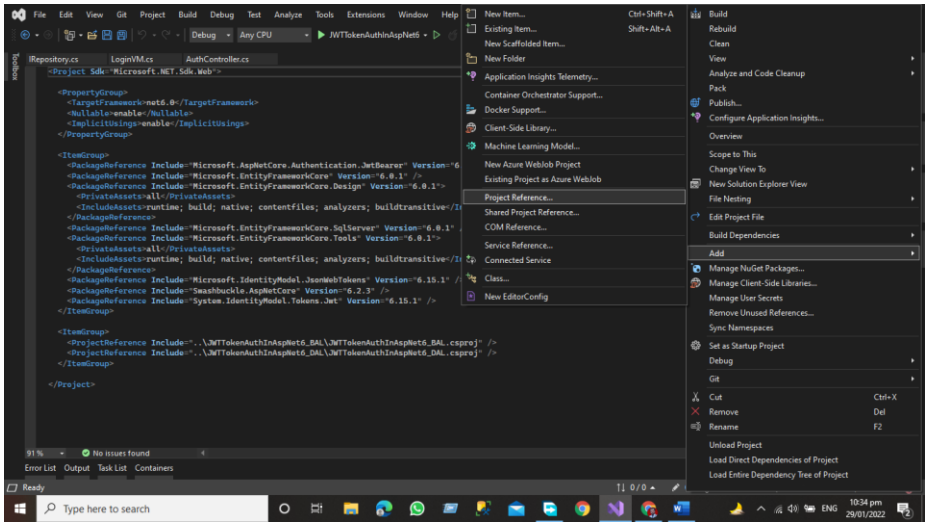
After adding the business access layer and data access layer, we must first add the references of the Data Access layer and Business Access layer, so that the project structure can be seen.



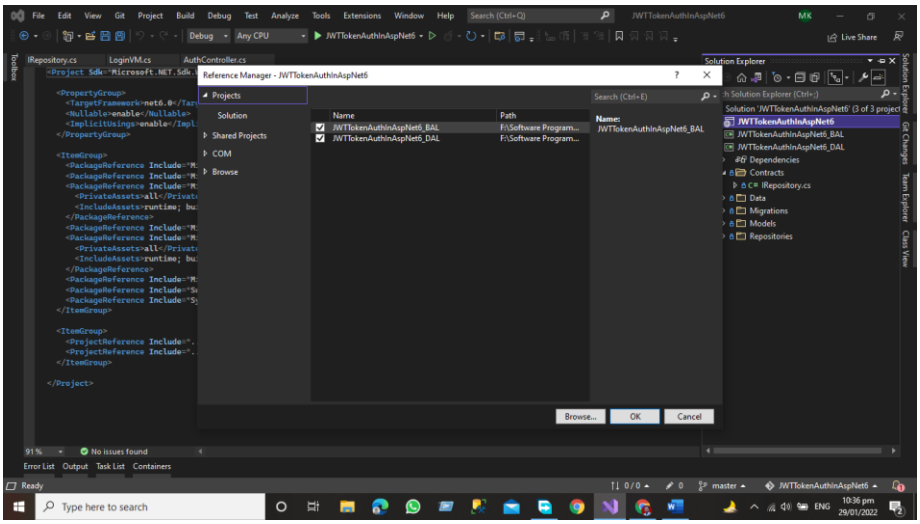
As you can see, our project now has a Presentation layer, Data Access Layer, and Business Access layer.

Add the References to the Project

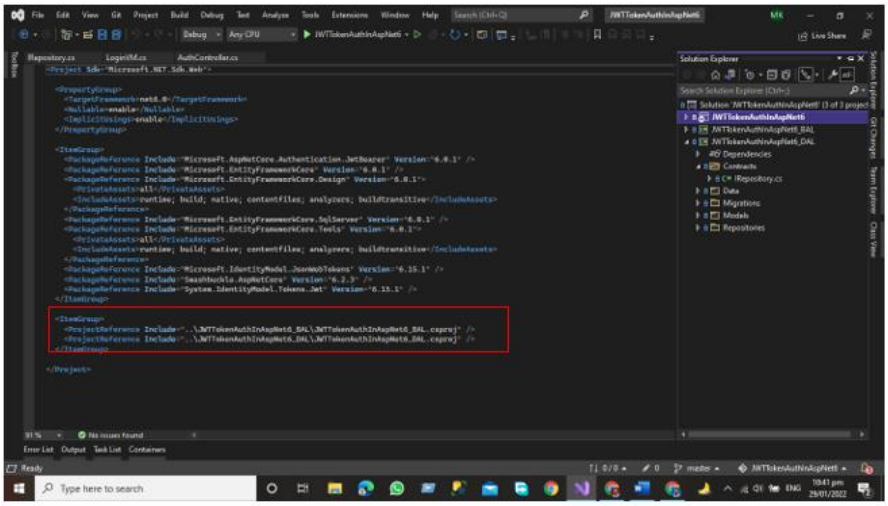
Now, in the Presentation Layer, add the references to the Data Access Layer and the Business Access Layer. Right Click on the Presentation layer and then click on add => project Reference



Add the References by Checking both the checkboxes.

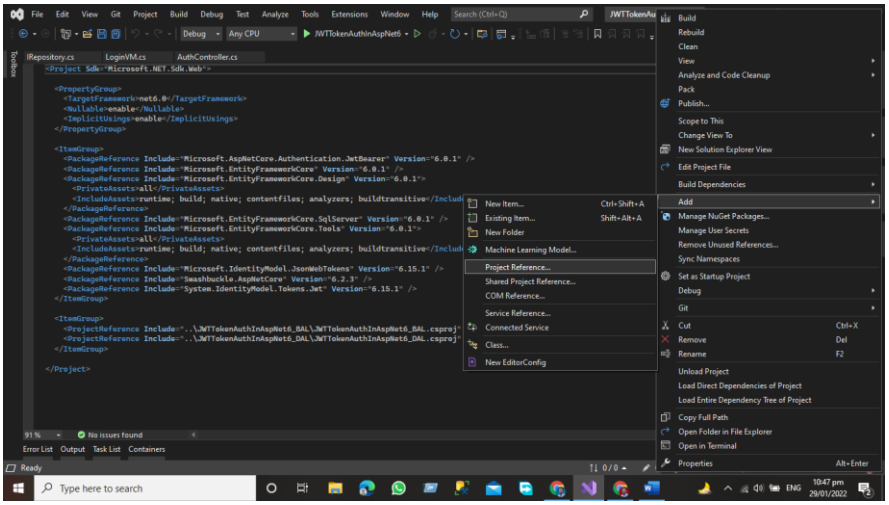


References have been added for both DAL and BAL

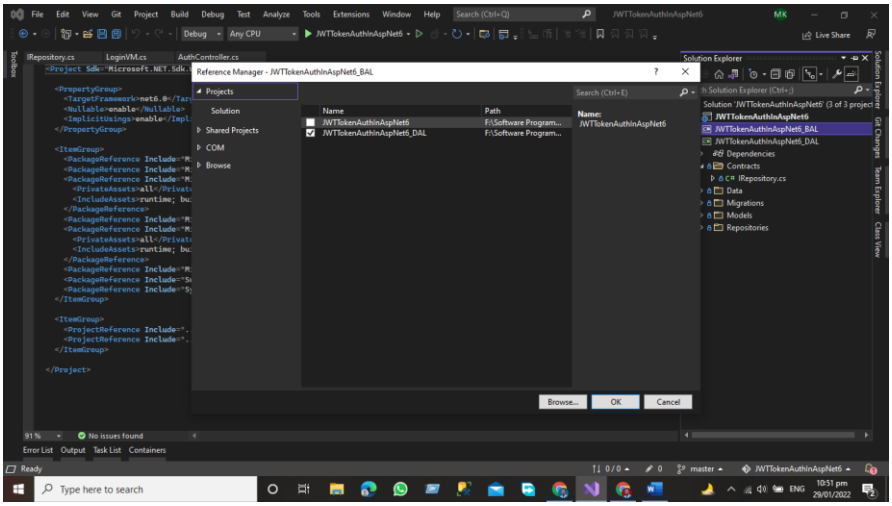


Now we'll add the Data Access layer project references to the Business layer.

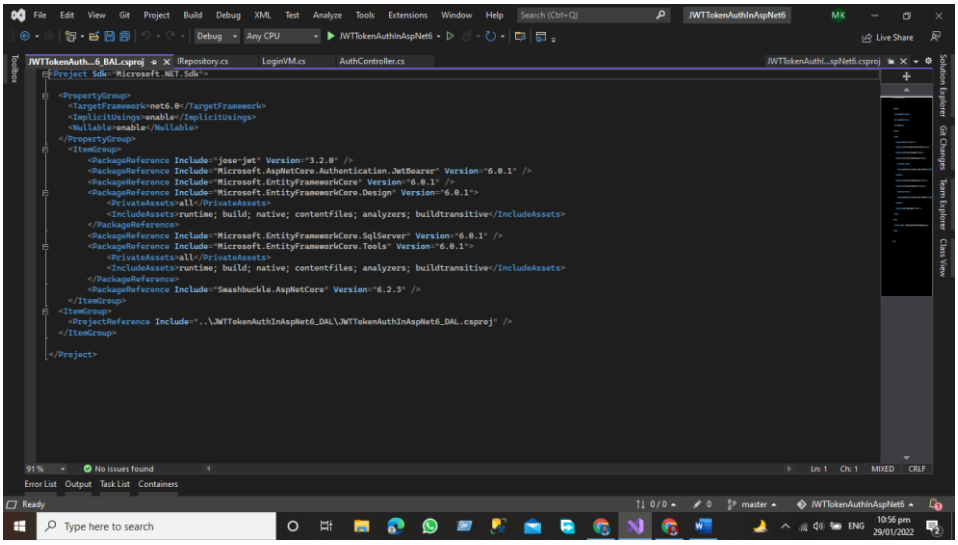
- Right Click on the Business access layer and then Click Add Button => Project References



Remember that we have included the DAL and BAL references in the project, so don't add the Presentation layer references again in the business layer. We only need the DAL Layer references in the business layer

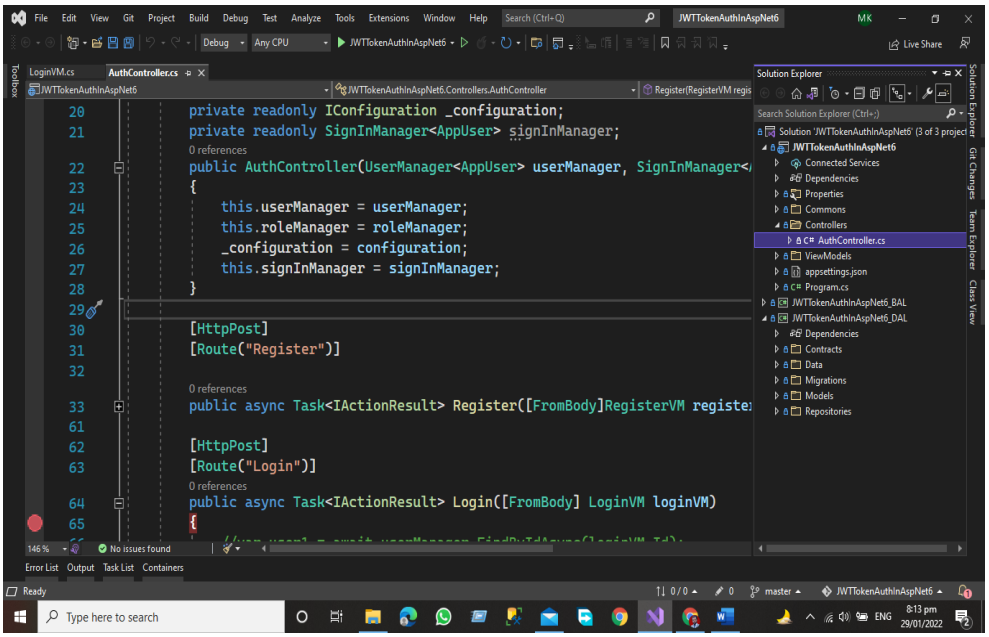


Project References of DAL Has been Added to BAL Layer



Data Access Layer

The Data Access Layer contains methods that assist the Business Access Layer in writing business logic, whether the functions are linked to accessing or manipulating data. The Data Access Layer's major goal is to interface with the database and the Business Access Layer in our project.



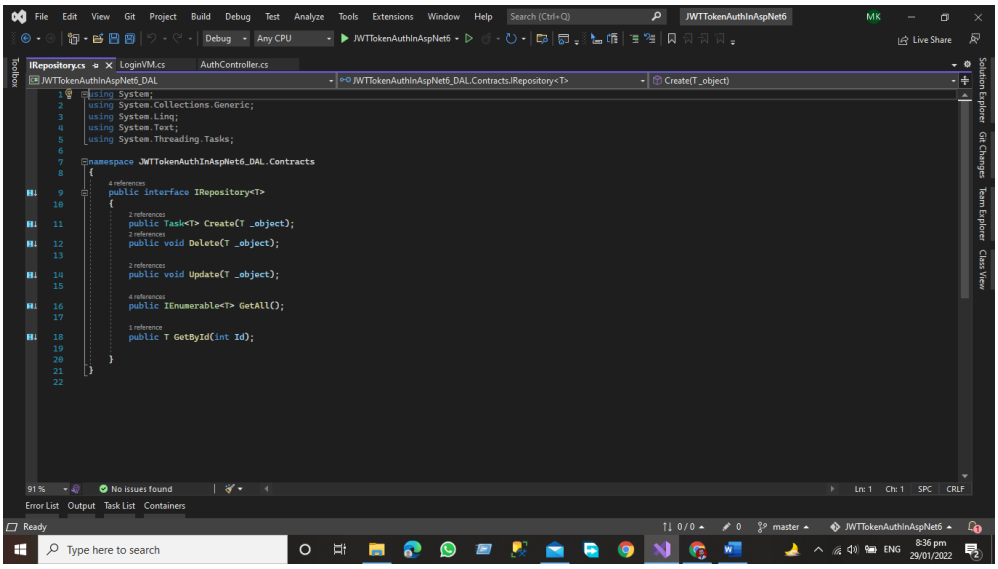
the structure will be like this. In this Layer we have the Following Folders Add these folders to your Data access layer

- Contacts
- Data
- Migrations
- Models
- Repositories

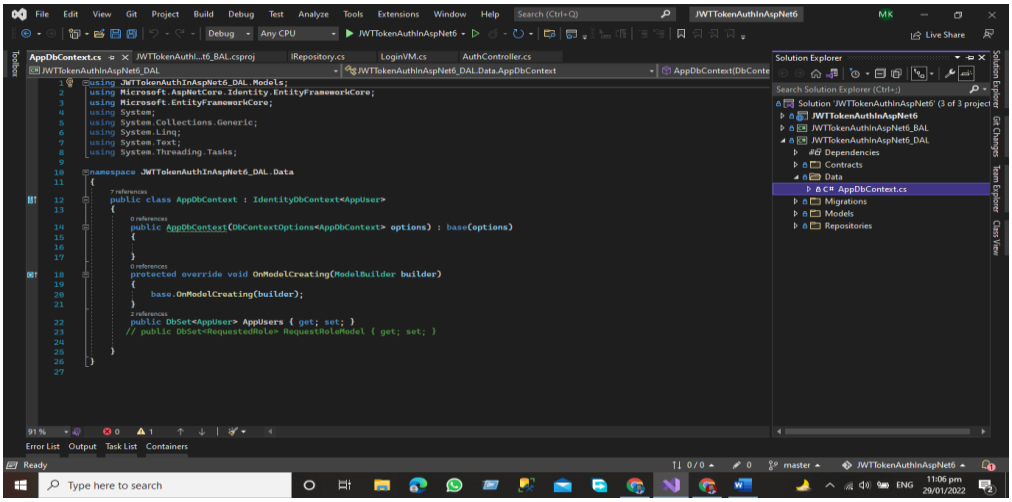
Contacts

In the Contract Folder, we define the interface that has the following function that performs the desired functionalities with the database like

```
using System;
using System.Collections.Generic;
using System. Linq;
using System. Text;
using System.Threading.Tasks;

namespace JWTTokenAuthInAspNet6_DAL.Contracts
{
    public interface IRepository<T>
    {
        public Task<T> Create(T _object);
        public void Delete(T _object);

        public void Update(T _object);
        public IEnumerable<T> GetAll();
        public T GetById(int Id);

    }
}
```

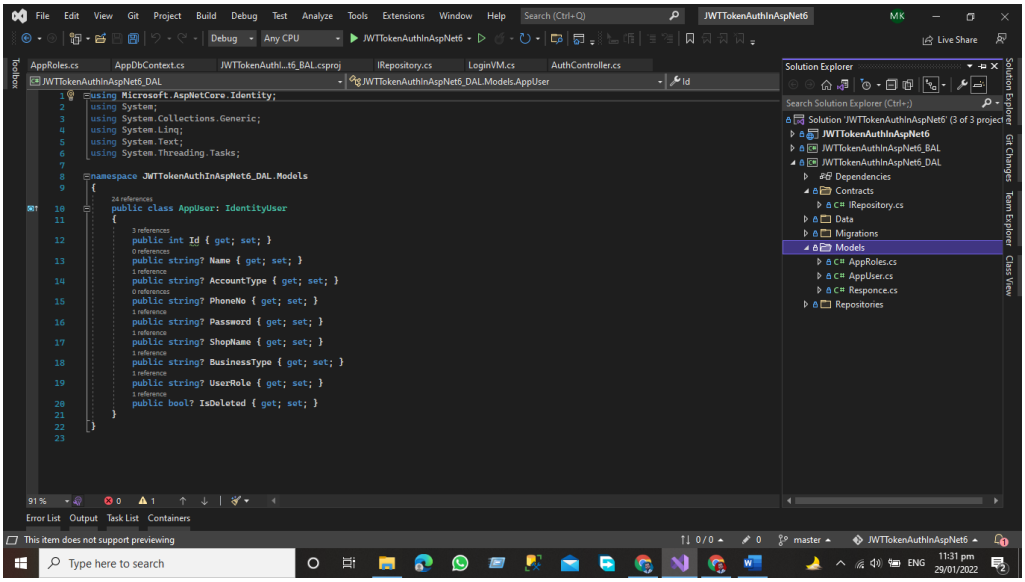


Data: In the Data folder, we have Db Context Class this class is very important for accessing the data from the database.



Migrations: The Migration Folder contains information on all the migrations we performed during the construction of this project. The Migration Folder contains information on all the migrations we performed during the construction of this project.

Models: Our application models, which contain domain-specific data, and business logic models, which represent the structure of the data as public attributes, business logic, and methods, are stored in the Model folder.

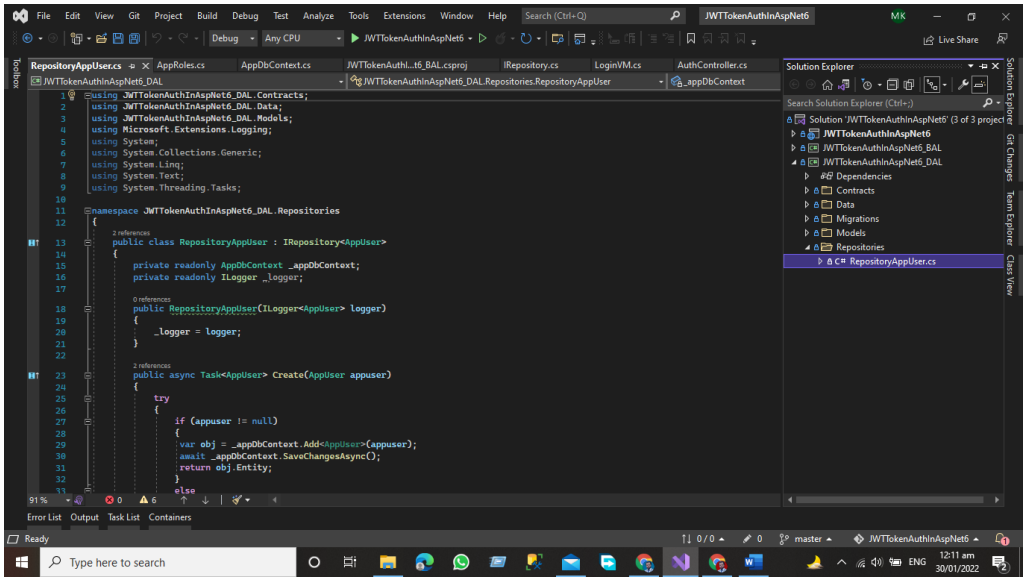


```
using Microsoft.AspNetCore.Identity;
using System;
using System.Collections.Generic;
using System. Linq;
using System. Text;
using System.Threading.Tasks;

namespace JWTTokenAuthInAspNet6_DAL.Models
{
    public class AppUser: IdentityUser
    {
        public int Id { get; set; }
        public string? Name { get; set; }
```

```
        public string? AccountType { get; set; }
        public string? PhoneNo { get; set; }
        public string? Password { get; set; }
        public string? ShopName { get; set; }
        public string? BusinessType { get; set; }
        public string? UserRole { get; set; }
        public bool? IsDeleted { get; set; }
    }
}
```

Repositors

In the Repository folder, we add the repository classes against each model. We write the CRUD function that communicates with the database using the entity framework. We add the repository class that inherits our Interface that is present in our contract folder.



```
using JWTTokenAuthInAspNet6_DAL.Contracts;
using JWTTokenAuthInAspNet6_DAL.Data;
using JWTTokenAuthInAspNet6_DAL.Models;
using Microsoft.Extensions.Logging;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace JWTTokenAuthInAspNet6_DAL.Repositories
{
    public class RepositoryAppUser : IRepository<AppUser>
    {
        private readonly AppDbContext _appDbContext;
        private readonly ILogger _logger;

        public RepositoryAppUser(ILogger<AppUser> logger)
        {
```

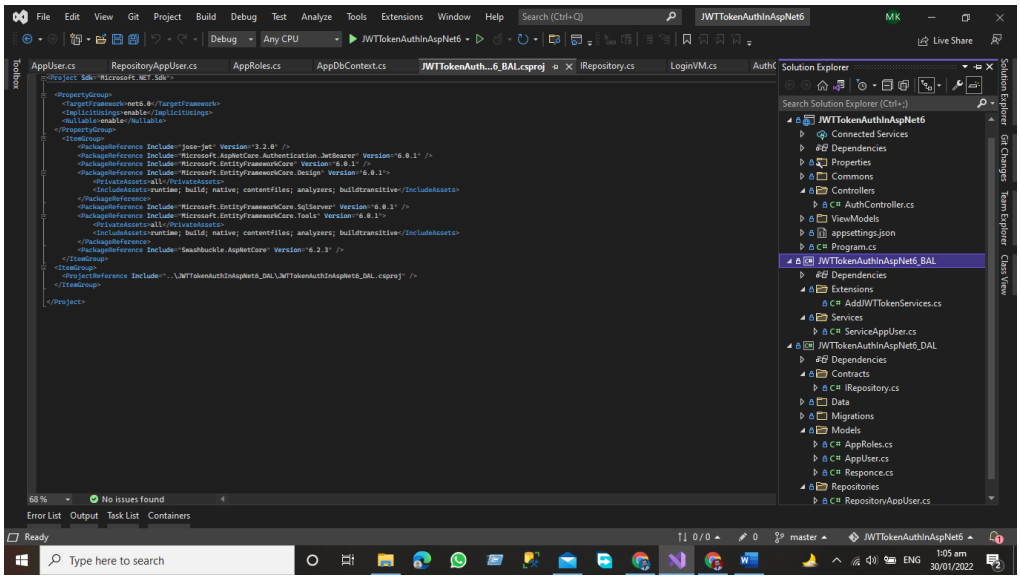
```
            _logger = logger;
        }

        public async Task<AppUser> Create(AppUser appuser)
        {
            try
            {
                if (appuser != null)
                {
                 var obj = _appDbContext.Add<AppUser>(appuser);
                 await _appDbContext.SaveChangesAsync();
                 return obj.Entity;
                }
                else
                {
                    return null;
                }
            }
            catch (Exception)
            {
                throw;
            }
        }

        public void Delete(AppUser appuser)
        {
            try
            {
                if(appuser!=null)
                {
                    var obj = _appDbContext.Remove(appuser);
                    if (obj!=null)
                    {
                        _appDbContext.SaveChangesAsync();
                    }
                }
            }
            catch (Exception)
            {
                throw;
            }
        }

        public IEnumerable<AppUser> GetAll()
        {
            try
            {
                var obj = _appDbContext.AppUsers.ToList();
```

```
                if (obj != null)
                    return obj;
                else
                    return null;
            }
            catch (Exception)
            {
                throw;
            }
        }

        public AppUser GetById(int Id)
        {
            try
            {
                if(Id!=null)
                {
                  var Obj = _appDbContext.AppUsers.FirstOrDefault(x =>
x.Id == Id);
                    if (Obj != null)
                        return Obj;
                    else
                        return null;
                }
                else
                {
                    return null;
                }
            }
            catch (Exception)
            {
                throw;
            }
        }

        public void Update(AppUser appuser)
        {
            try
            {
                if (appuser != null)
                {
                   var obj = _appDbContext.Update(appuser);
                    if (obj != null)
                        _appDbContext.SaveChanges();
                }
            }
            catch (Exception)
            {
```

```
                throw;
            }
        }
    }
}
```

Business Access Layer

The business layer communicates with the data access layer and the presentation layer logic layer process the actions that make the logical decision and evaluations the main function of this layer is to process the data between surrounding layers. Our Structure of the project in the Business Access Layer will be like this.



In this layer, we will have two folders

- Extensions Method Folder
- And Services Folder

In the business access layer, we have our services that communicate with the surrounding layers like the data layer and Presentation Layer.

Services

Services define the application's business logic. We develop services that interact with the data layer and move data from the data layer to the presentation layer and from the presentation layer to the data access layer.

Example

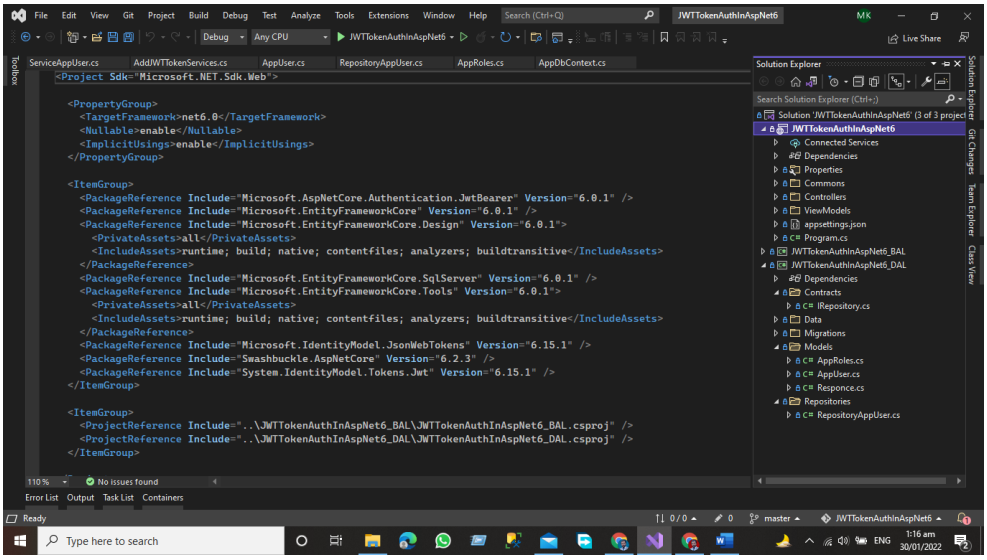
```
using JWTTokenAuthInAspNet6_DAL.Contracts;
using JWTTokenAuthInAspNet6_DAL.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace JWTTokenAuthInAspNet6_BAL.Services
```

```
{
    public class ServiceAppUser
    {
        public readonly IRepository<AppUser> _repository;
        public ServiceAppUser(IRepository<AppUser> repository)
        {
            _repository = repository;
        }
        //Create Method
        public async Task<AppUser> AddUser(AppUser appUser)
        {
            try
            {
                if (appUser == null)
                {
                    throw new ArgumentNullException(nameof(appUser));
                }
                else
                {
                    return await _repository.Create(appUser);
                }
            }
            catch (Exception)
            {

                throw;
            }
        }

        public void DeleteUser(int Id)

        {
            try
            {
                if (Id != 0)
                {
                    var obj = _repository.GetAll().Where(x => x.Id ==
Id).FirstOrDefault();
                    _repository.Delete(obj);
                }
            }
            catch (Exception)
            {
                throw;
            }
        }

        public void UpdateUser(int Id)
```

```
        {
            try
            {
                if (Id != 0)
                {
                    var obj = _repository.GetAll().Where(x => x.Id ==
Id).FirstOrDefault();
                    if (obj != null)
                        _repository.Update(obj);
                }
            }
            catch (Exception)
            {

                throw;
            }

        }

        public IEnumerable<AppUser> GetAllUser()
        {
            try
            {
               return _repository.GetAll().ToList();
            }
            catch (Exception)
            {

                throw;
            }
        }


    }
}
```

Presentation layer

The top-most layer of the 3-Tier architecture is the Presentation layer the main function of this layer is to give the results to the user or in simple words to present the data that we get from the business access layer and give the results to the front-end user.
In the presentation layer, we have the default template of the asp.net core application here OUR structure of the application will be like this.



The presentation layer is responsible to give the results to the user against every HTTP request. Here we have a controller that is responsible for handling the HTTP request and communicates with the business layer against each HTTP request get the get data from the associated layer and then present it to the User.

Advantages of 3-Tier Architecture

- It makes the logical separation between the presentation layer, business layer, and data layer.
- Migration to a new environment is fast.
- In This Model, each tier is independent so we can set the different developers on each tier for the fast development of applications.
- Easy to maintain and understand large-scale applications.
- The business layer is in between the presentation layer and data layer the application is more secure because the client will not have direct access to the database.
- The process data that is sent from the business layer is validated at this level.
- The Posted data from the presentation layer will be validated at the business layer before Inserting or Updating the Database
- Database security can be provided at BAL Layer
- Define the business logic one in the BAL Layer and then share it among the other components at the presentation layer.
- Easy to Appy for Object-oriented Concepts
- Easy to update the data provided quires at DAL Layer

Dis-Advantages of 3-Tier Architecture

- It takes a lot of time to build a small part of the application.
- Need a Good understanding of Object-Oriented programming.

Conclusion

In this chapter, we have learned about 3-tier Architecture and how the three-layer exchange data between them how we can add the Data Access Layer and Business access layer in the project, and studied how these layers communicate with each other.

- **Presentation Layer (PL):** The top-most layer of the 3-Tier architecture is the Presentation layer the main function of this layer is to give the results to the user or in

simple words to present the data that we get from the business access layer and give the results to the front-end user.

- **Business Access Layer (BAL):** The logic layer communicates with the data access layer and the presentation layer logic layer process the actions that make the logical decision and evaluations the main function of this layer is to process the data between surrounding layers.
- **Data Access Layer (DAL):** The main function of this application is to access and store the data from the database and the process of the data to business access layer data goes to the presentation layer against user request.

Complete Auth Microservices Using Three Tier Architecture Git Hub Project URL

Click here for complete project access

4

Building Microservices Using Onion Architecture

Overview

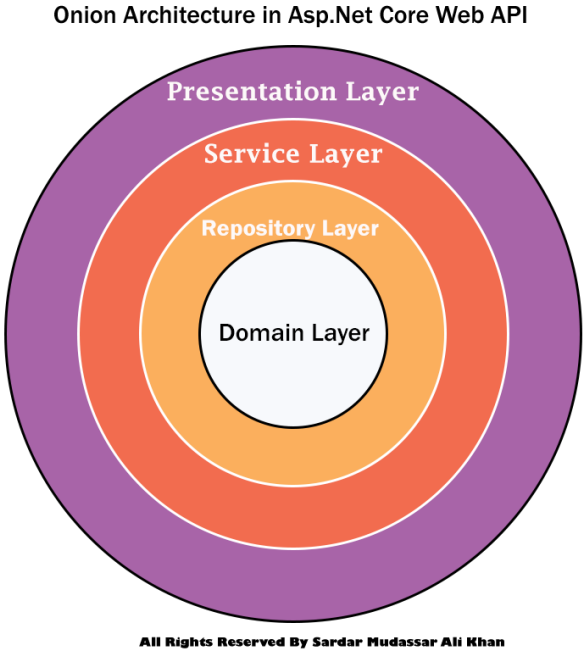
In this chapter, we explore Onion Architecture and explore its key Layers: Domain, Repository, Service, and Presentation. We discuss its Advantages and guide you through the Implementation and Project Structure. Detailed insights include the Models Folder, Data Folder, and various services like Department Service and Student Service. We also cover Dependency Injection, modifications to Program.cs, and the role of Controllers. This chapter equips you with the knowledge to effectively implement Onion Architecture in your projects.

Introduction

In this chapter, we will cover the onion architecture using the Asp.Net 6 Web API. Onion architecture term introduces by Jeffrey Palermo in 2008 this architecture provides us a better way to build applications using this architecture our applications are better testable maintainable and dependable on infrastructures like databases and services. Onion architecture solves common problems like coupling and separation of concerns.

What is Onion architecture

In onion architecture, we have the domain layer, repository layer, service layer, and presentation layer. Onion architecture solves the problem that we face during the enterprise applications like coupling and separations of concerns. Onion architecture also solves the problem that we confronted in three-tier architecture and N-Layer architecture. In Onion architecture, our layer communicates with each other using interfaces.



Layers in Onion Architecture

Onion architecture uses the concept of the layer, but it is different from N-layer architecture and 3-Tier architecture.

Domain Layer

This layer lies in the center of the architecture where we have application entities which are the application model classes or database model classes using the code first approach in the application development using Asp.net core these entities are used to create the tables in the database.

Repository Layer

The repository layer act as a middle layer between the service layer and model objects we will maintain all the database migrations and database context Object in this layer. We will add the

interfaces that consist the of data access pattern for reading and writing operations with the database.

Service Layer

This layer is used to communicate with the presentation and repository layer. The service layer holds all the business logic of the entity. In this layer services interfaces are kept separate from their implementation for loose coupling and separation of concerns.

Presentation Layer

In the case of the API Presentation layer that presents us the object data from the database using the HTTP request in the form of JSON Object. But in the case of front-end applications, we present the data using the UI by consuming the APIS.

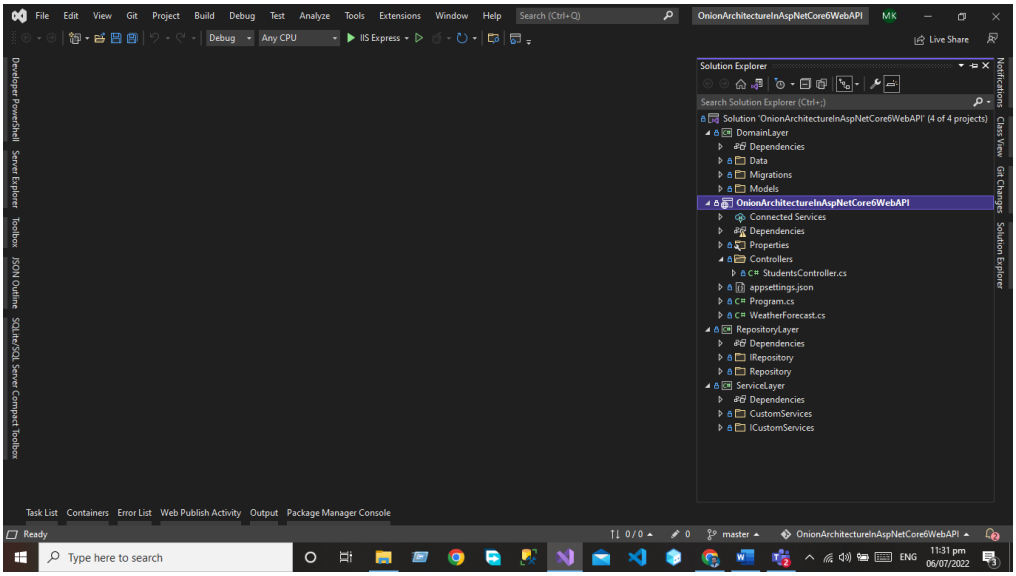
Advantages of Onion Architecture

- Onion architecture provides us with the batter maintainability of code because code depends on layers.
- It provides us batter testability for unit tests we can write the separate test cases in layers without affecting the other module in the application.
- Using the onion architecture our application is loosely coupled because our layers communicate with each other using the interface.
- Domain entities are the core and center of the architecture and have access to databases and UI Layer.
- A Complete implementation would be provided to the application at run time.
- The external layer never depends on the external layer.

Implementation of Onion Architecture

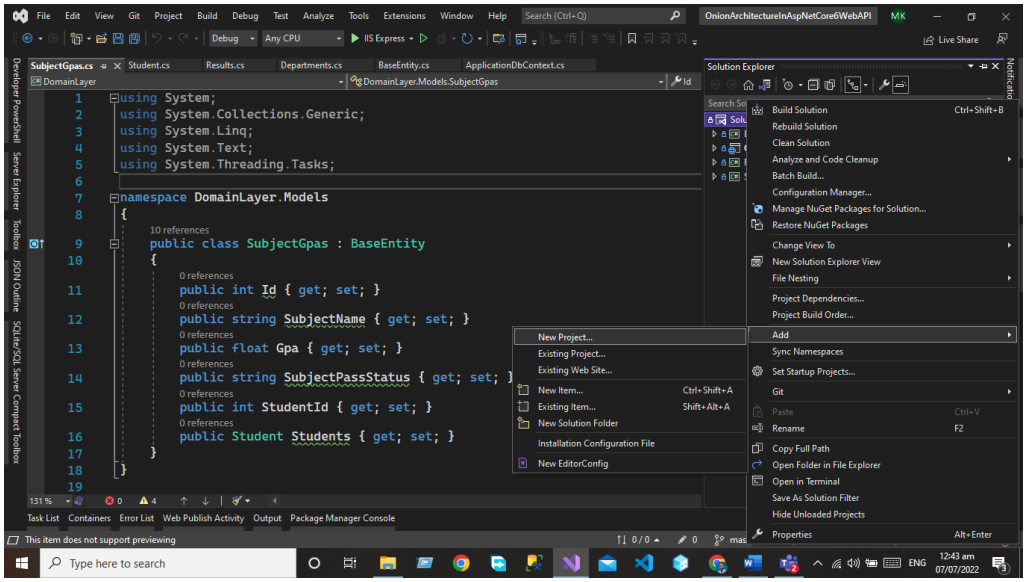
Now we are going to develop the project using the Onion Architecture
First, you need to create the Asp.net Core web API project using visual studio. After creating the project, we will add our layer to the project after adding all the layers our project structure will look like this.

Project Structure.

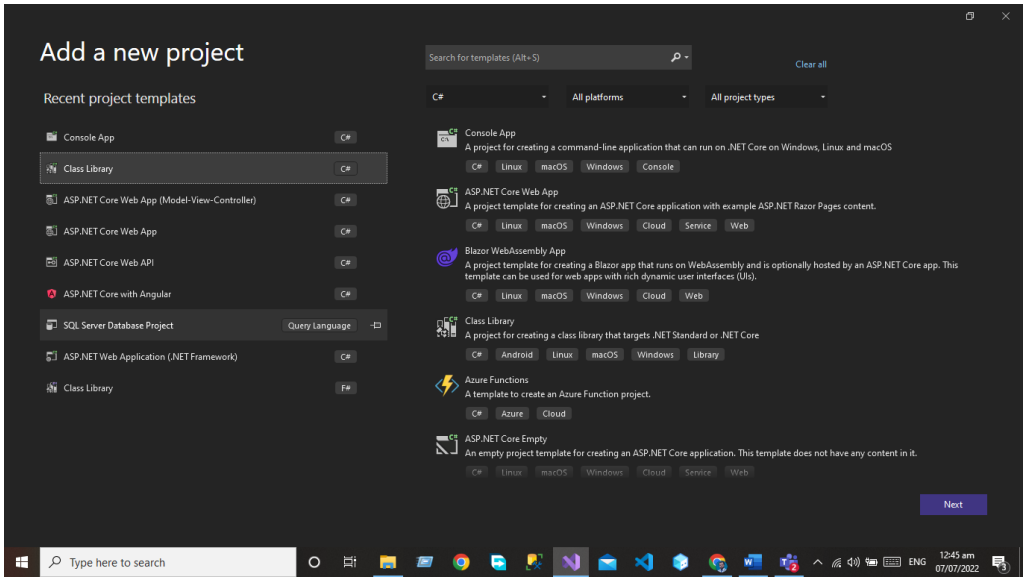


Domain Layer

This layer lies in the center of the architecture where we have application entities which are the application model classes or database model classes using the code first approach in the application development using Asp.net core these entities are used to create the tables in the database. For the Domain layer, we need to add the library project to our application.
Write click on the Solution and then click on add option.



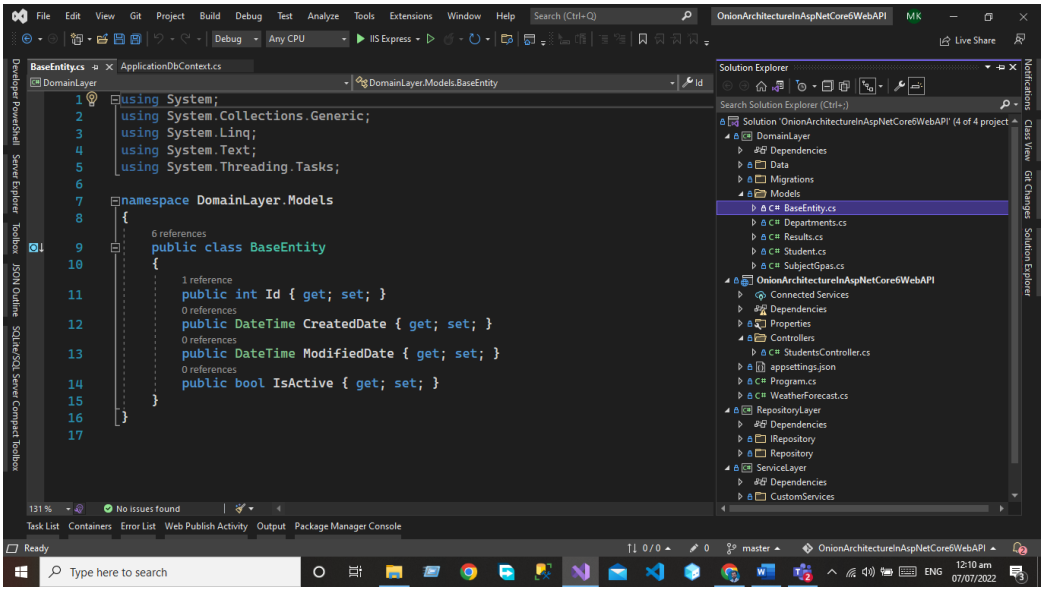
Add the library project to your solution



Name this project as Domain Layer

Models Folder

First, you need to add the Models folder that will be used to create the database entities. In the Models folder, we will create the following database entities.



Base Entity

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DomainLayer.Models
{
    public class BaseEntity
    {
        public int Id { get; set; }
        public DateTime CreatedDate { get; set; }
        public DateTime ModifiedDate { get; set; }
        public bool IsActive { get; set; }
    }
}
```

Department

Department entity will extend the base entity

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DomainLayer.Models
{
    public class Departments :BaseEntity
    {
        public int Id { get; set; }
```

```
        public string DepartmentName { get; set; }
        public int StudentId { get; set; }
        public Student Students { get; set; }
    }
}
```

Result

Result Entity will extend the base entity

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DomainLayer.Models
{
    public class Resultss : BaseEntity
    {
        [Key]
        public int Id { get; set; }
        public string? ResultStatus { get; set; }
        public int StudentId { get; set; }
        public Student Students { get; set; }
    }
}
```

Students

Student Entity will extend the base entity.

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DomainLayer.Models
{
    public class Student : BaseEntity
    {
        [Key]
        public int Id { get; set; }
        public string? Name { get; set; }
        public string? Address { get; set; }
        public string? Emial { get; set; }
        public string? City { get; set; }
        public string? State { get; set; }
```

```
        public string? Country { get; set; }
        public int? Age { get; set; }
        public DateTime? BirthDate { get; set;}

    }
}
```

SubjectGpa

Subject GPA Entity will extend the Base Entity

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DomainLayer.Models
{
    public class SubjectGpas : BaseEntity
    {
        public int Id { get; set; }
        public string SubjectName { get; set; }
        public float Gpa { get; set; }
        public string SubjectPassStatus { get; set; }
        public int StudentId { get; set; }
        public Student Students { get; set; }
    }
}
```

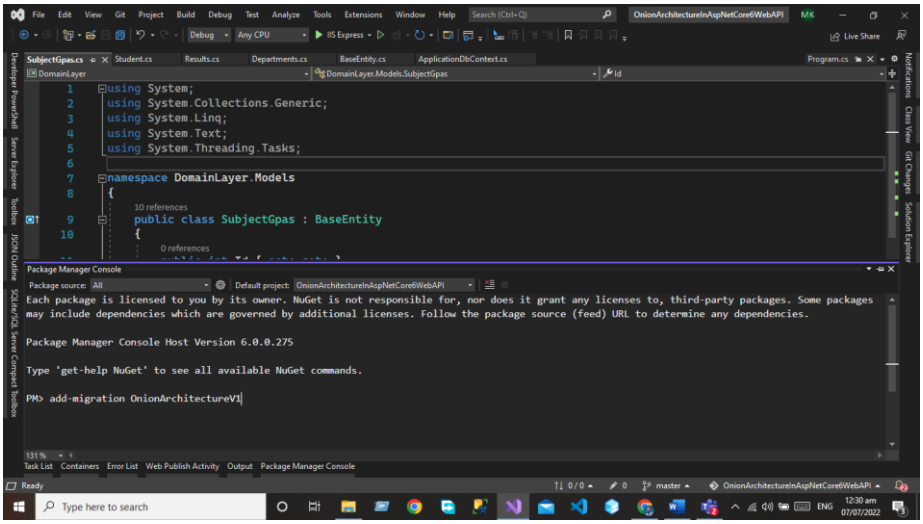
Data Folder

Add the Data in the domain that is used to add the database context class. The database context class is used to maintain the session with the underlying database using which you can perform the CRUD operation.
Write click on the application and create the class ApplicationDbContext.cs

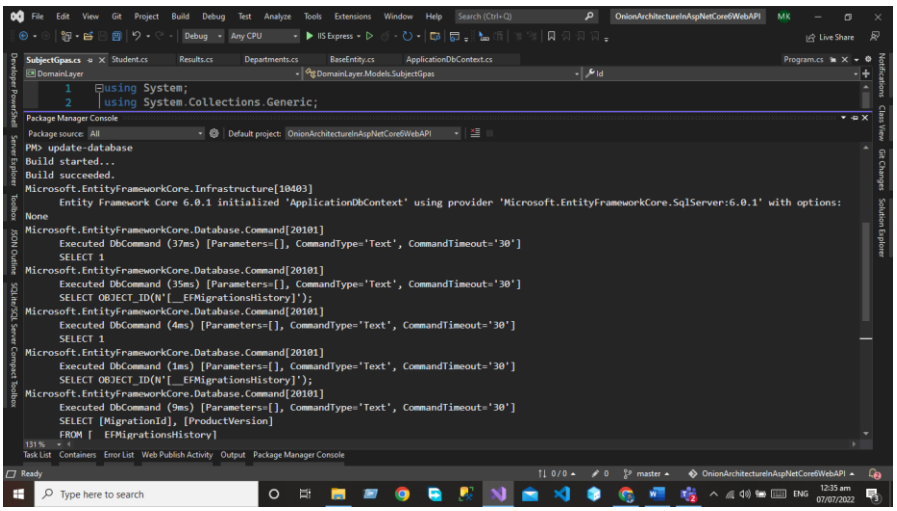
```
using DomainLayer.Models;
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DomainLayer.Data
{
    public class ApplicationDbContext : DbContext
    {
```

```
        public
ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options)
        {

        }
        protected override void OnModelCreating(ModelBuilder builder)
        {
            base.OnModelCreating(builder);
        }
        public DbSet<Student> Students { get; set; }
        public DbSet<Departments> Departments { get; set; }
        public DbSet<SubjectGpas> SubjectGpas { get; set; }
        public DbSet<Resultss> Results { get; set; }
    }
}
```

After Adding the DbSet properties we need to add the migration using the package manager console and run the command Add-Migration. add-migration OnionArchitectureV1

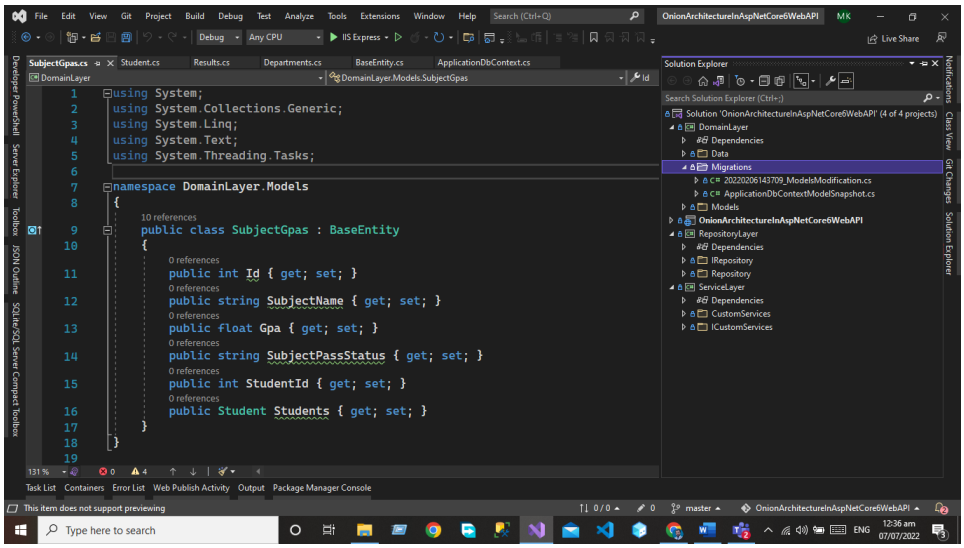


After executing the commands now, you need to update the database by executing the update-database Command.



Migration Folder

After executing both commands now, you can see the migration folder in the domain layer.

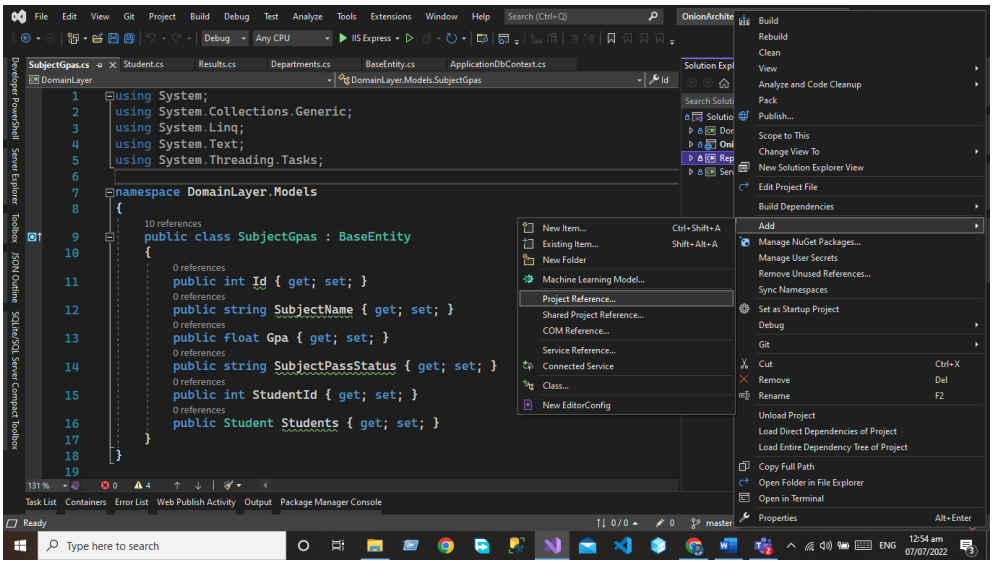


Repository Layer

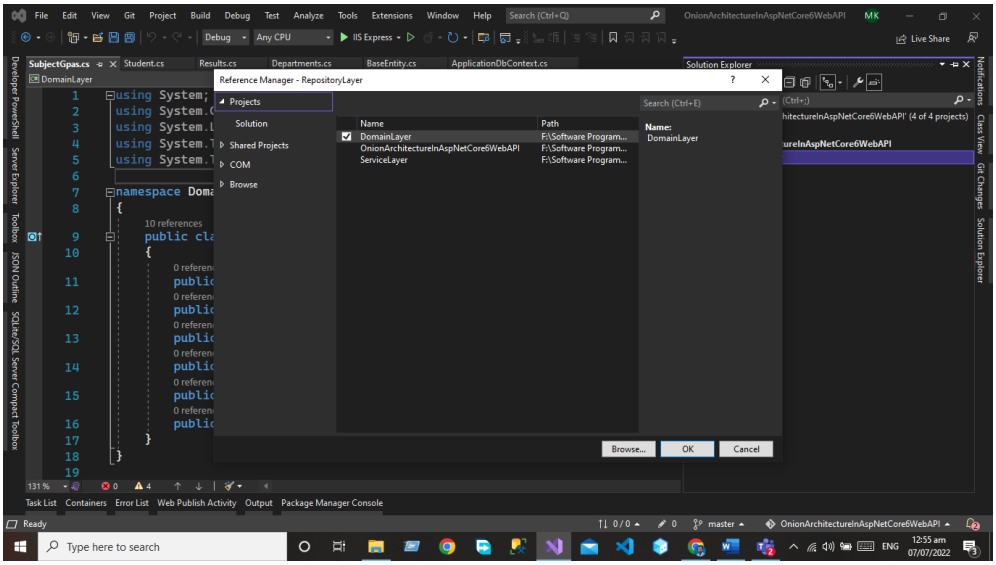
The repository layer act as a middle layer between the service layer and model objects we will maintain all the database migrations and database context Object in this layer. We will add the interfaces that consist the of data access pattern for reading and writing operations with the database.

Now we will work on Repository Layer we will follow the same project as we did for the Domain layer add the library project in your applicant and give a name to that project Repository layer.

But here we need to add the project reference of the Domain layer in the repository layer. Write click on the project and then click the Add button after that we will add the project references in the Repository layer.



Click on project reference now and select the Domain layer.



Now we need to add the two folders.

- IRepository
- Repository

IRepository

IRepsitory folder contains the generic interface using this interface we will create the CRUD operation for our all entities. Generic Interface will extend the Base-Entity for using some properties. The Code of the generic interface is given below.

```
using DomainLayer.Models;

namespace RepositoryLayer.IRepository
{
    public interface IRepository<T> where T: BaseEntity
    {
        IEnumerable<T> GetAll();
        T Get(int Id);
        void Insert(T entity);
        void Update(T entity);
        void Delete(T entity);
        void Remove(T entity);
        void SaveChanges();
    }
}
```

Repository

Now we create the Generic repository that extends the Generic IRepository Interface. The Code of the generic repository is given below.

```
using DomainLayer.Data;
using DomainLayer.Models;
using Microsoft.EntityFrameworkCore;
using RepositoryLayer.IRepository;
```

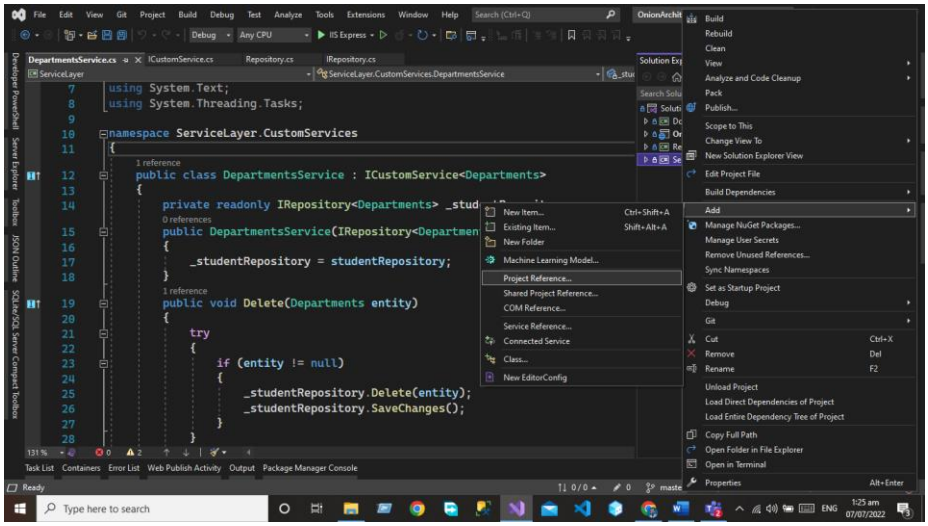
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace RepositoryLayer.Repository
{
    public class Repository<T> : IRepository<T> where T : BaseEntity
    {
        #region property
        private readonly ApplicationDbContext _applicationDbContext;
        private DbSet<T> entities;
        #endregion

        #region Constructor
        public Repository(ApplicationDbContext applicationDbContext)
        {
            _applicationDbContext = applicationDbContext;
            entities = _applicationDbContext.Set<T>();
        }
        #endregion

        public void Delete(T entity)
        {
            if (entity == null)
            {
                throw new ArgumentNullException("entity");
            }
            entities.Remove(entity);
            _applicationDbContext.SaveChanges();
        }
        public T Get(int Id)
        {
            return entities.SingleOrDefault(c => c.Id == Id);
        }

        public IEnumerable<T> GetAll()
        {
            return entities.AsEnumerable();
        }
        public void Insert(T entity)
        {
            if (entity == null)
            {
                throw new ArgumentNullException("entity");
            }
             entities.Add(entity);
```

```
            _applicationDbContext.SaveChanges();
        }
        public void Remove(T entity)
        {
            if (entity == null)
            {
                throw new ArgumentNullException("entity");
            }
            entities.Remove(entity);
        }

        public void SaveChanges()
        {
            _applicationDbContext.SaveChanges();
        }

        public void Update(T entity)
        {
            if (entity == null)
            {
                throw new ArgumentNullException("entity");
            }
            entities.Update(entity);
            _applicationDbContext.SaveChanges();
        }

    }
}
```

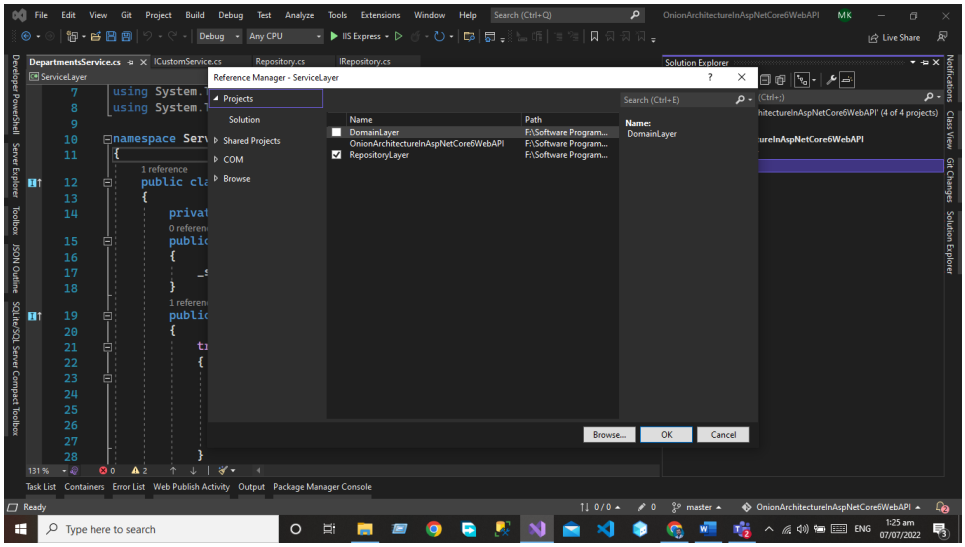
We will use this repository in our service layer.

Service Layer

This layer is used to communicate with the presentation and repository layer. The service layer holds all the business logic of the entity. In this layer services interfaces are kept separate from their implementation for loose coupling and separation of concerns. Now we need to add a new project to our solution that will be the service layer we will follow the same process for adding the library project in our application but here we need some extra work after adding the project we need to add the reference of the Repository Layer.



Now our service layer contains the reference of the repository layer.



In the Service layer, we will create the two folders.

- ICustomServices
- CustomServices

ICustom Service

Now in the ICustomServices folder, we will create the ICustomServices Interface this interface holds the signature of the method we will implement these methods in the customs service code of the ICustomServices Interface given below.

```
using DomainLayer.Models;
using System;
using System.Collections.Generic;
using System. Linq;
using System. Text;
using System.Threading.Tasks;

namespace ServiceLayer.ICustomServices
{
```

```
    public interface ICustomService<T> where T : class
    {
        IEnumerable<T> GetAll();
        T Get(int Id);
        void Insert(T entity);
        void Update(T entity);
        void Delete(T entity);
        void Remove(T entity);

    }
}
```

Custom Service

In the custom service folder, we will create the custom service class that inherits the ICustomService interface code of the custom service class given below. We will write the crud for our all entities. For every service, we will write the CRUD operation using our generic repository.

```
Department Service
using DomainLayer.Models;
using RepositoryLayer.IRepository;
using ServiceLayer.ICustomServices;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ServiceLayer.CustomServices
{
    public class DepartmentsService : ICustomService<Departments>
    {
        private readonly IRepository<Departments> _studentRepository;
        public DepartmentsService(IRepository<Departments>
studentRepository)
        {
            _studentRepository = studentRepository;
        }
        public void Delete(Departments entity)
        {
            try
            {
                if (entity != null)
                {
                    _studentRepository.Delete(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
```

```
            {

                throw;
            }
        }

        public Departments Get(int Id)
        {
            try
            {
                var obj = _studentRepository.Get(Id);
                if (obj != null)
                {
                    return obj;
                }
                else
                {
                    return null;
                }

            }
            catch (Exception)
            {

                throw;
            }
        }

        public IEnumerable<Departments> GetAll()
        {
            try
            {
                var obj = _studentRepository.GetAll();
                if (obj != null)
                {
                    return obj;
                }
                else
                {
                    return null;
                }
            }
            catch (Exception)
            {

                throw;
            }
        }
```

```
public void Insert(Departments entity)
{
    try
    {
        if (entity != null)
        {
            _studentRepository.Insert(entity);
            _studentRepository.SaveChanges();
        }
    }
    catch (Exception)
    {

        throw;
    }
}

public void Remove(Departments entity)
{
    try
    {
        if (entity != null)
        {
            _studentRepository.Remove(entity);
            _studentRepository.SaveChanges();
        }
    }
    catch (Exception)
    {

        throw;
    }
}
public void Update(Departments entity)
{
    try
    {
        if (entity != null)
        {
            _studentRepository.Update(entity);
            _studentRepository.SaveChanges();
        }
    }
    catch (Exception)
    {

        throw;
```

```
            }
        }
    }
}
```

Student Service

```
using DomainLayer.Models;
using RepositoryLayer.IRepository;
using ServiceLayer.ICustomServices;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ServiceLayer.CustomServices
{
    public class StudentService : ICustomService<Student>
    {
        private readonly IRepository<Student> _studentRepository;
        public StudentService(IRepository<Student> studentRepository)
        {
            _studentRepository = studentRepository;
        }
        public void Delete(Student entity)
        {
            try
            {
                if(entity!=null)
                {
                    _studentRepository.Delete(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }

        public  Student Get(int Id)
        {
            try
            {
                var obj = _studentRepository.Get(Id);
                if(obj!=null)
                {
```

```
                return obj;
            }
            else
            {
                return null;
            }

        }
        catch (Exception)
        {

            throw;
        }
    }

    public IEnumerable<Student> GetAll()
    {
        try
        {
            var obj = _studentRepository.GetAll();
            if (obj != null)
            {
                return obj;
            }
            else
            {
                return null;
            }
        }
        catch (Exception)
        {

            throw;
        }
    }

    public void Insert(Student entity)
    {
        try
        {
            if (entity != null)
            {
                _studentRepository.Insert(entity);
                _studentRepository.SaveChanges();
            }
        }
        catch (Exception)
        {
```

```
                throw;
            }
        }

        public void Remove(Student entity)
        {
            try
            {
                if(entity!=null)
                {
                  _studentRepository.Remove(entity);
                  _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }
        public void Update(Student entity)
        {
            try
            {
                if(entity!=null)
                {
                    _studentRepository.Update(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }
    }
}
```

Result Service

```
using DomainLayer.Models;
using RepositoryLayer.IRepository;
using ServiceLayer.ICustomServices;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
```

```
using System.Threading.Tasks;

namespace ServiceLayer.CustomServices
{
    public class ResultService : ICustomService<Resultss>
    {
        private readonly IRepository<Resultss> _studentRepository;
        public ResultService(IRepository<Resultss> studentRepository)
        {
            _studentRepository = studentRepository;
        }
        public void Delete(Resultss entity)
        {
            try
            {
                if (entity != null)
                {
                    _studentRepository.Delete(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }

        public Resultss Get(int Id)
        {
            try
            {
                var obj = _studentRepository.Get(Id);
                if (obj != null)
                {
                    return obj;
                }
                else
                {
                    return null;
                }

            }
            catch (Exception)
            {

                throw;
            }
```

```
        }

        public IEnumerable<Resultss> GetAll()
        {
            try
            {
                var obj = _studentRepository.GetAll();
                if (obj != null)
                {
                    return obj;
                }
                else
                {
                    return null;
                }
            }
            catch (Exception)
            {

                throw;
            }
        }

        public void Insert(Resultss entity)
        {
            try
            {
                if (entity != null)
                {
                    _studentRepository.Insert(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }

        public void Remove(Resultss entity)
        {
            try
            {
                if (entity != null)
                {
                    _studentRepository.Remove(entity);
                    _studentRepository.SaveChanges();
```

```
                }
            }
            catch (Exception)
            {

                throw;
            }
        }
        public void Update(Resultss entity)
        {
            try
            {
                if (entity != null)
                {
                    _studentRepository.Update(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }
    }
}
```

Subject GPA Service

```
using DomainLayer.Models;
using RepositoryLayer.IRepository;
using ServiceLayer.ICustomServices;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ServiceLayer.CustomServices
{
    public class SubjectGpasService : ICustomService<SubjectGpas>
    {
        private readonly IRepository<SubjectGpas> _studentRepository;
        public SubjectGpasService(IRepository<SubjectGpas>
studentRepository)
        {
            _studentRepository = studentRepository;
        }
        public void Delete(SubjectGpas entity)
```

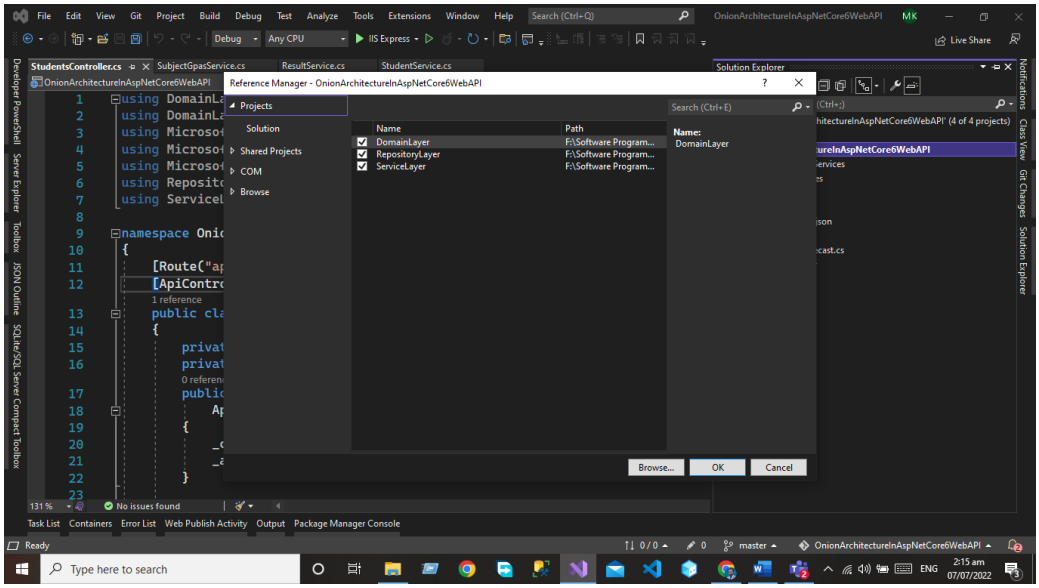
```
        {
            try
            {
                if (entity != null)
                {
                    _studentRepository.Delete(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }

        public SubjectGpas Get(int Id)
        {
            try
            {
                var obj = _studentRepository.Get(Id);
                if (obj != null)
                {
                    return obj;
                }
                else
                {
                    return null;
                }

            }
            catch (Exception)
            {

                throw;
            }
        }

        public IEnumerable<SubjectGpas> GetAll()
        {
            try
            {
                var obj = _studentRepository.GetAll();
                if (obj != null)
                {
                    return obj;
                }
                else
```

```
                {
                    return null;
                }
            }
            catch (Exception)
            {

                throw;
            }
        }

        public void Insert(SubjectGpas entity)
        {
            try
            {
                if (entity != null)
                {
                    _studentRepository.Insert(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }

        public void Remove(SubjectGpas entity)
        {
            try
            {
                if (entity != null)
                {
                    _studentRepository.Remove(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }
        public void Update(SubjectGpas entity)
        {
            try
            {
```

```
                if (entity != null)
                {
                    _studentRepository.Update(entity);
                    _studentRepository.SaveChanges();
                }
            }
            catch (Exception)
            {

                throw;
            }
        }
    }
}
```

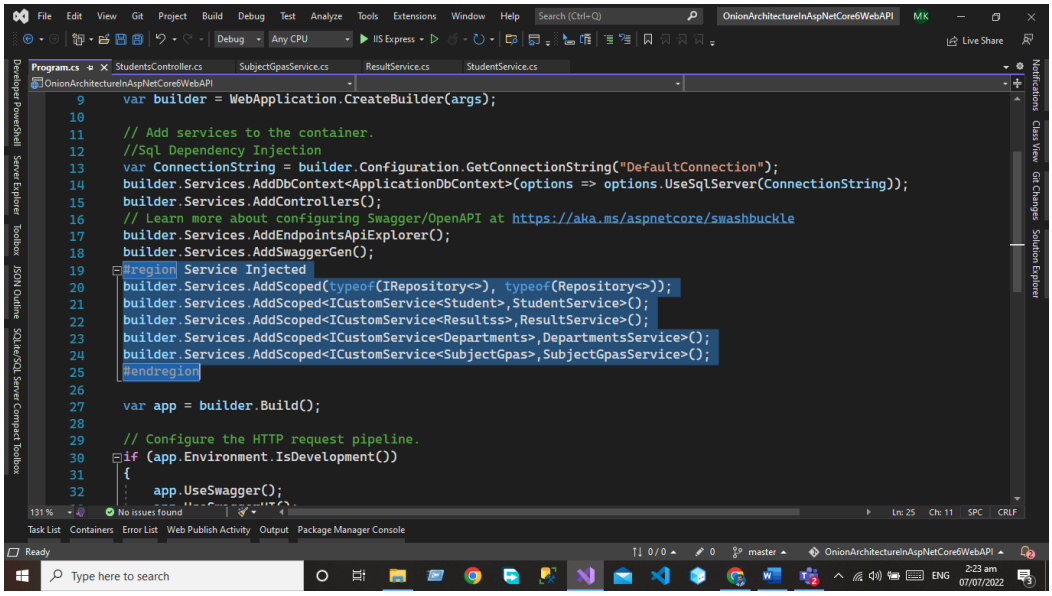
Presentation Layer

The presentation layer is our final layer that presents the data to the front-end user on every HTTP request.
In the case of the API presentation layer that presents us the object data from the database using the HTTP request in the form of JSON Object. But in the case of front-end applications, we present the data using the UI by consuming the APIS.
The presentation layer is the default Asp.net core web API project Now we need to add the project references of all the layers as we did before



Dependency Injection

Now we need to add the dependency Injection of our all services in the program.cs class



Modify Program.cs File

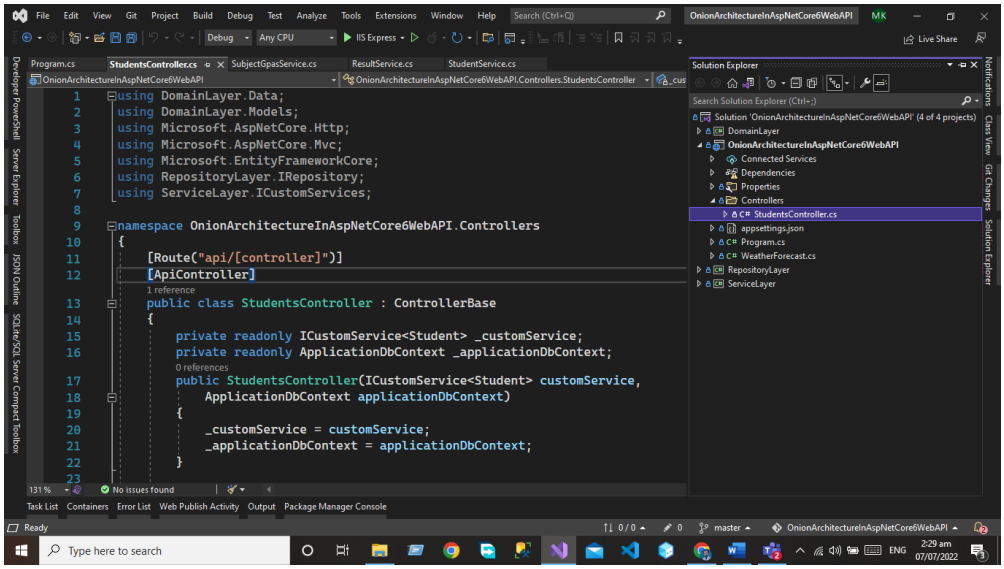
The Code of the Startup Class is Given below

```
using DomainLayer.Data;
using DomainLayer.Models;
using Microsoft.EntityFrameworkCore;
using RepositoryLayer.IRepository;
using RepositoryLayer.Repository;
using ServiceLayer.CustomServices;
using ServiceLayer.ICustomServices;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
//Sql Dependency Injection
var ConnectionString =
builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(ConnectionString));
builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
#region Service Injected
builder.Services.AddScoped(typeof(IRepository<>),
typeof(Repository<>));
builder.Services.AddScoped<ICustomService<Student>,StudentService>();
builder.Services.AddScoped<ICustomService<Resultss>,ResultService>();
builder.Services.AddScoped<ICustomService<Departments>,DepartmentsServi
ce>();
```

```
builder.Services.AddScoped<ICustomService<SubjectGpas>,SubjectGpasServi
ce>();
#endregion

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

Controllers

Controllers are used to handle the HTTP request. Now we need to add the student controller that will interact will our service layer and display the data to the users.



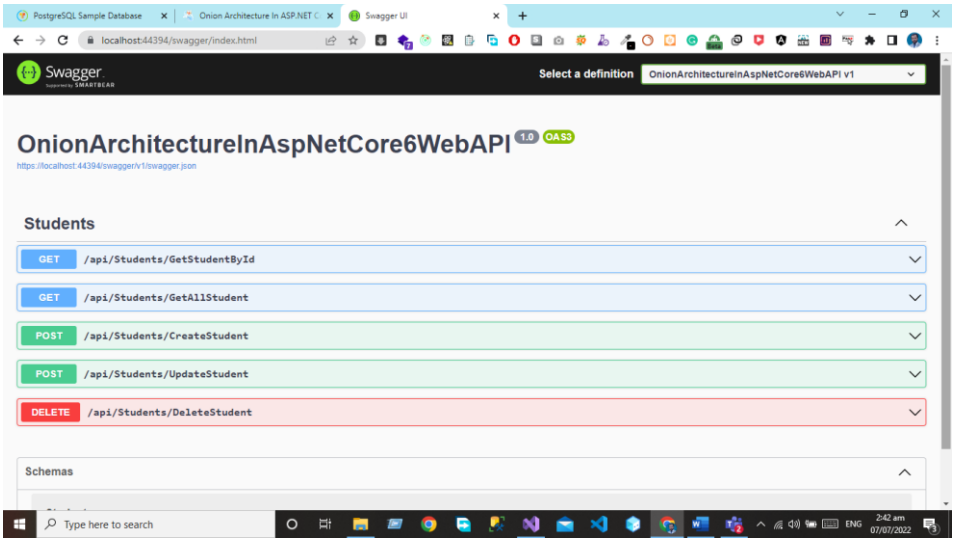
The Code of the Controller is given below.

```
using DomainLayer.Data;
using DomainLayer.Models;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using RepositoryLayer.IRepository;
using ServiceLayer.ICustomServices;
```

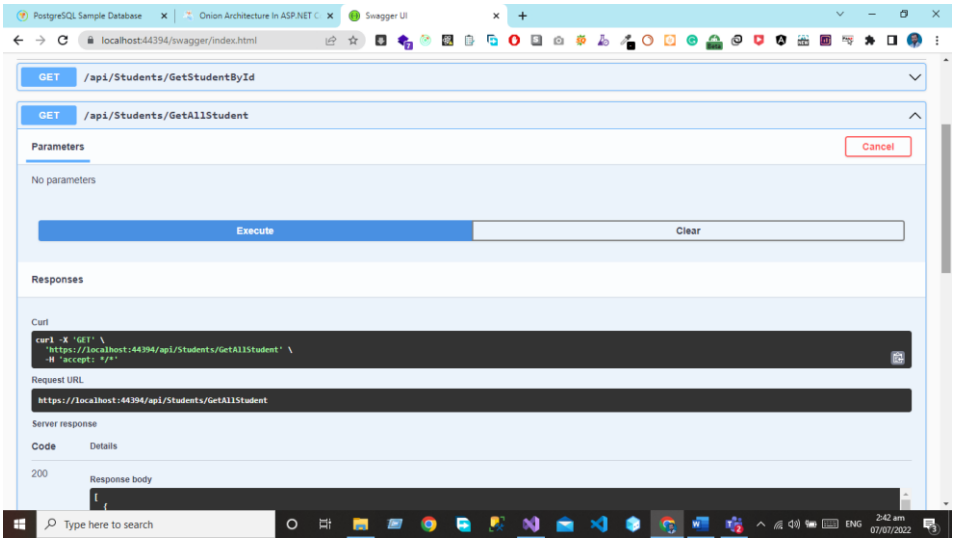
```
namespace OnionArchitectureInAspNetCore6WebAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly ICustomService<Student> _customService;
        private readonly ApplicationDbContext _applicationDbContext;
        public StudentsController(ICustomService<Student>
customService,
            ApplicationDbContext applicationDbContext)
        {
            _customService = customService;
            _applicationDbContext = applicationDbContext;
        }

        [HttpGet(nameof(GetStudentById))]
        public IActionResult GetStudentById(int Id)
        {
            var obj = _customService.Get(Id);
            if (obj == null)
            {
                return NotFound();
            }
            else
            {
                return Ok(obj);
            }
        }
        [HttpGet(nameof(GetAllStudent))]
        public IActionResult GetAllStudent()
        {
            var obj = _customService.GetAll();
            if(obj == null)
            {
                return NotFound();
            }
            else
            {
                return Ok(obj);
            }
        }

        [HttpPost(nameof(CreateStudent))]
        public IActionResult CreateStudent(Student student)
        {
            if (student!=null)
```

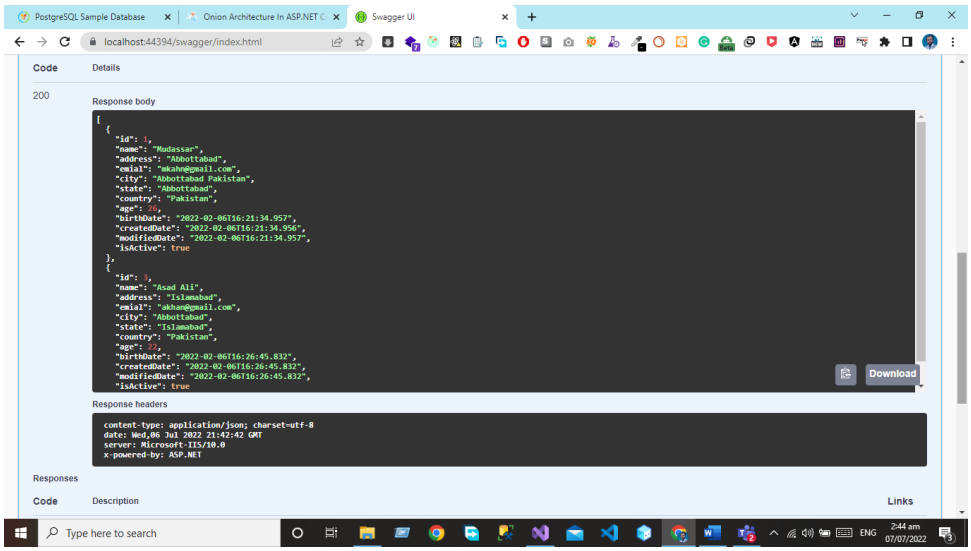
```
            {
              _customService.Insert(student);
               return Ok("Created Successfully");
            }
            else
            {
                return BadRequest("Somethingwent wrong");
            }
        }

        [HttpPost(nameof(UpdateStudent))]
        public IActionResult UpdateStudent(Student student)
        {
            if(student!=null)
            {
                _customService.Update(student);
                return Ok("Updated SuccessFully");
            }
            else
            {
                return BadRequest();
            }

        }

        [HttpDelete(nameof(DeleteStudent))]
        public IActionResult DeleteStudent(Student student)
        {
            if(student!=null)
            {
                _customService.Delete(student);
                return Ok("Deleted Successfully");
            }
            else
            {
                return BadRequest("Something went wrong");
            }
        }
    }
}
```

Output: Now we will run the project and will see the output using the swagger.



Now we can see when we hit the GetAllStudent Endpoint we can see the data of students from the database in the form of JSON projects.





Conclusion

In this chapter, we have implemented the Onion architecture using the Entity Framework and Code First approach. We have now the knowledge of how the layer communicates with each other's in onion architecture and how we can write the Generic code for the Interface repository and services. Now we can develop our project using onion architecture for API Development OR MVC Core Based projects.

Complete GitHub project URL

Click here for complete project

5

Building Microservice Using Clean Architecture

Overview

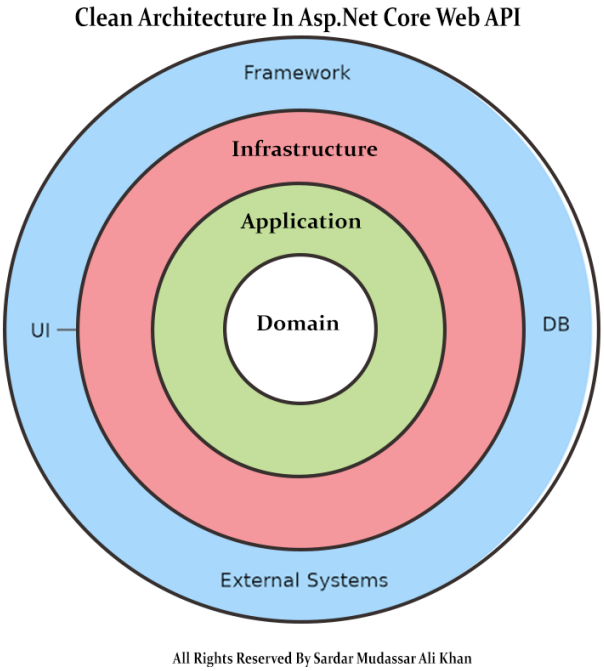
In this chapter, we introduce Clean Architecture, covering its core principles and Layers: Domain, Application, Infrastructure, and Presentation. We explore its Advantages, guide you through Implementation, and detail setup for Extensions Folder, Startup.cs, and Helpers. We provide Code Examples for Auto Mapper, DTOs, and Controllers, including the Basket Controller and Product Controller. Finally, we present the Complete Project Code to help you effectively apply Clean Architecture in your projects.

Introduction

In this chapter, we will cover clean architecture practically. Clean architecture is the software architecture that helps us to keep the entire application code under control. The main goal of the clean architecture is the code/logic, which is unlikely to change, and must be written without any direct dependency this means that if I want to change my development framework OR User Interface (UI) of the system the core of the system should not be changed. It means that our external dependencies are completely replaceable.

What is Clean Architecture

Clean architecture has a domain layer, Application Layer, Infrastructure Layer, and Framework Layer. The domain and application layer are always the center of the design and are known as the core of the system. The core will be independent of the data access and infrastructure concerns. And we can achieve this goal by using the Interfaces and abstraction within the core system but implementing them outside of the core system.



Layer In Clean Architecture

Clean architecture has a domain layer, Application Layer, Infrastructure Layer, and Presentation Layer. The domain and application layer are always the center of the design and are known as the core of the system. In Clean architecture, all the dependencies of the application are Independent /Inwards, and the Core system has no dependencies on any other layer of the system. So, in the future, if we want to change the UI/ OR framework of the system we can do it easily because our all-other dependencies of the system are not dependent on the core of the system.

Domain Layer

The domain layer in the clean architecture contains the enterprise logic Like the Entities and their specifications This layer lies in the center of the architecture where we have application entities which are the application model classes or database model classes using the code first approach in the application development using Asp.net core these entities are used to create the tables in the database.

Application Layer

The application layer contains the business logic all the business logic will be written in this layer. In this layer services interfaces are kept separate from their implementation for loose coupling and separation of concerns.

Infrastructure Layer

In the infrastructure layer, we have model objects we will maintain all the database migrations and database context Objects in this layer. In this layer, we have the repositories of all the domain model objects.

Presentation Layer

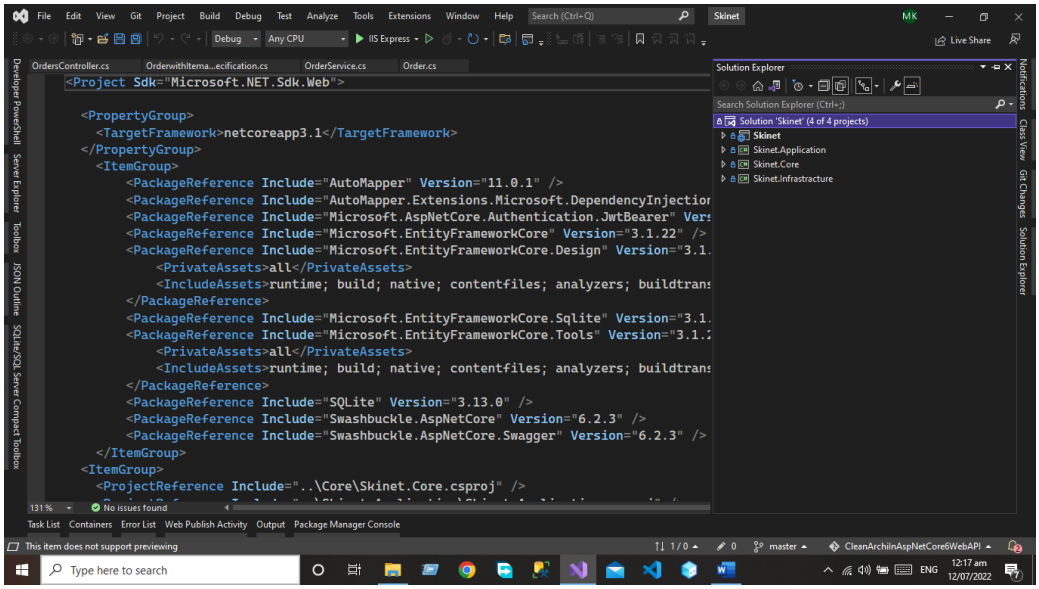
In the case of the API Presentation layer that presents us the object data from the database using the HTTP request in the form of JSON Object. But in the case of front-end applications, we present the data using the UI by consuming the APIS.

Advantages of Clean Architecture

- The immediate implementation you can implement this architecture with any programming language.
- The domain and application layer are always the center of the design and are known as the core of the system that why the core of the system is not dependent on external systems.
- This architecture allows you to change the external system without affecting the core of the system.
- In a highly testable environment, you can test your code quickly and easily.
- You can create a highly scalable and quality product.

Implementation of Clean Architecture

Now we are going to implement the clean architecture. First, you need to create the Asp.net Core API Project using the visual studio after that we will add the layer into our solution so after adding all the layers in the system our project structure will be like this.



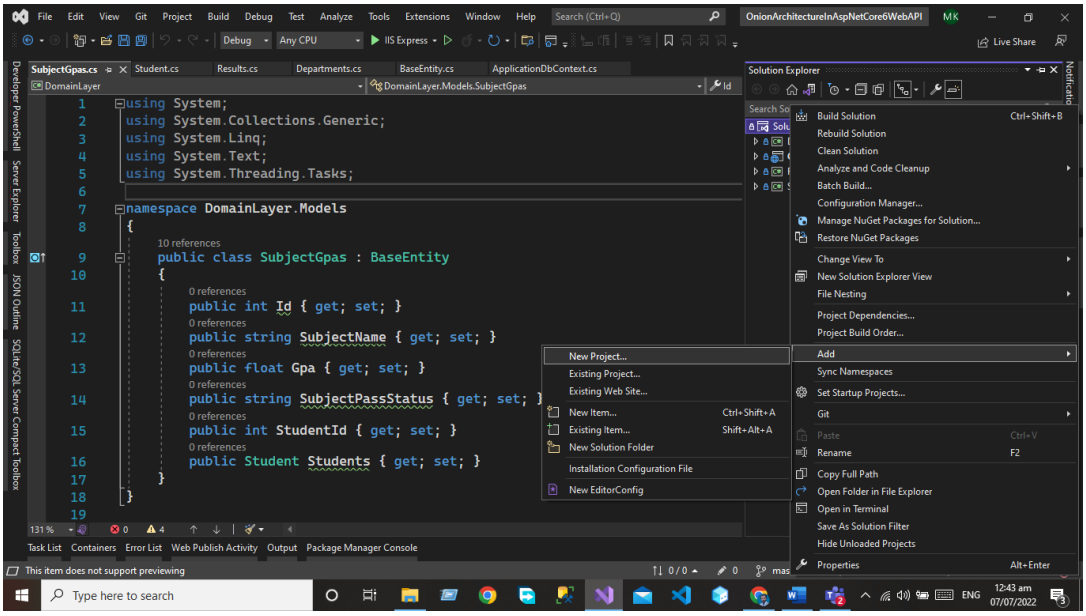
Now you can see that our project structure will be like in the above picture.
Let's Implement the layer practically.

Domain Layer

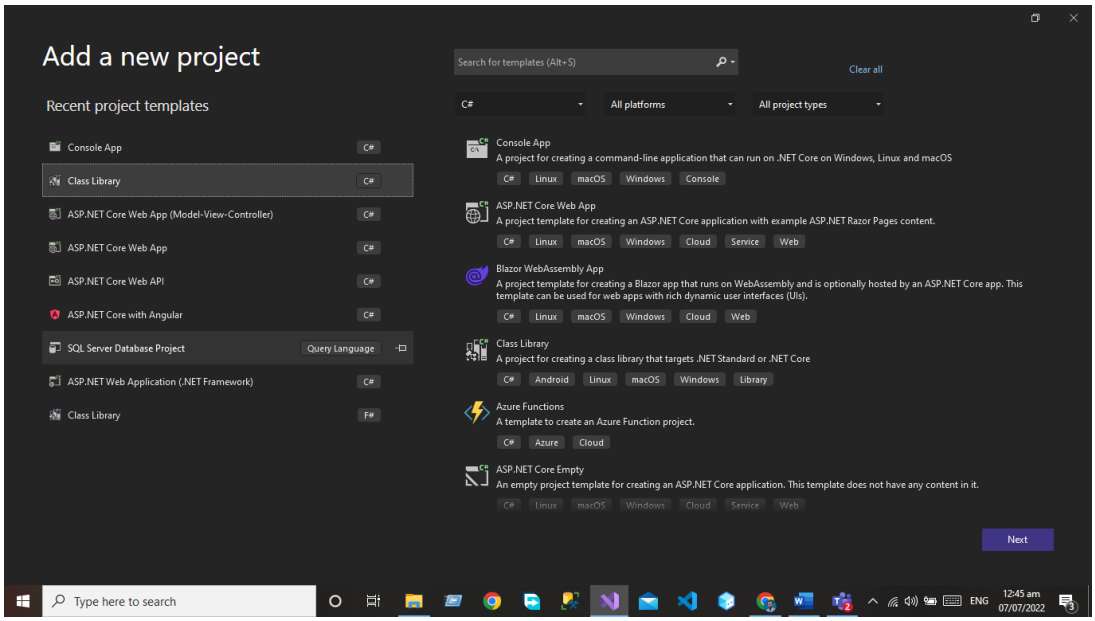
The domain layer in the clean architecture contains the enterprise logic Like the Entities and their specifications This layer lies in the center of the architecture where we have application entities which are the application model classes or database model classes using the code first approach in the application development using Asp.net core these entities are used to create the tables in the database.

Implementation of Domain layer

First, you need to add the library project to your system so let's add the library project to your system. Write click on the Solution and then click on add option.



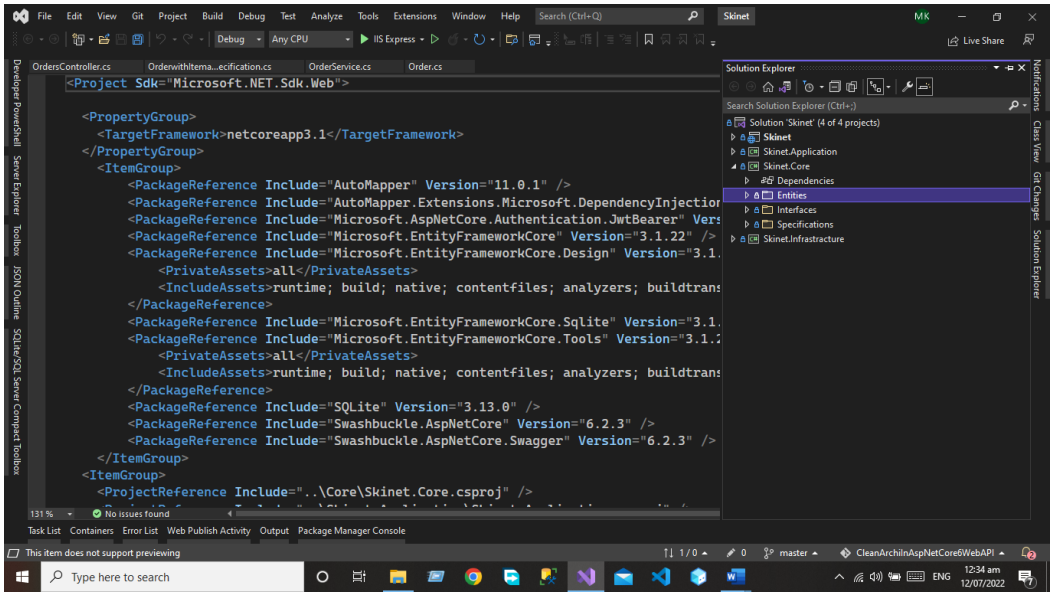
Add the library project to your solution.



Name this project as Domain Layer

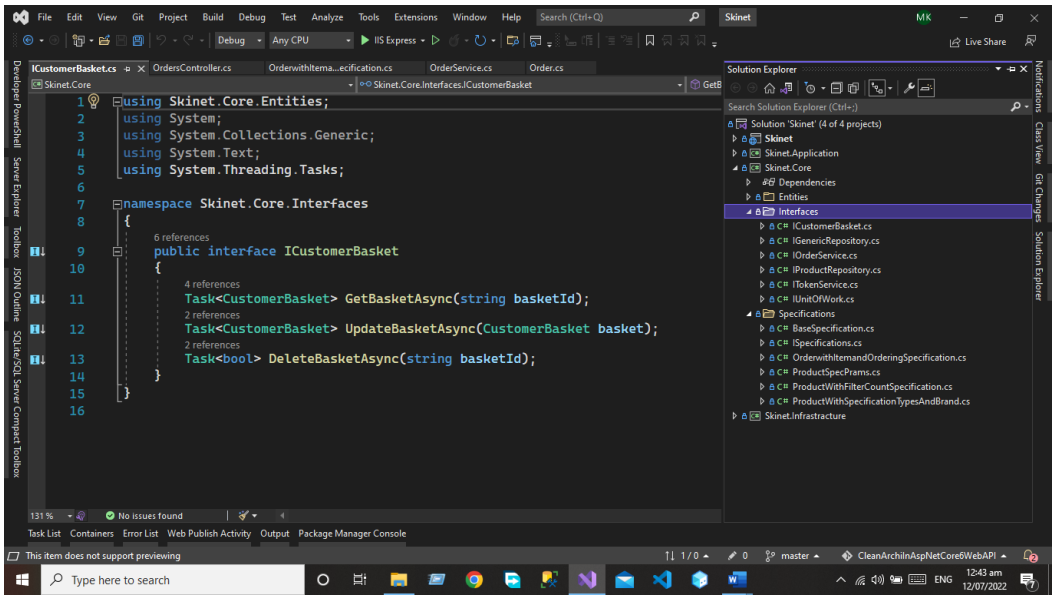
Entities Folder

First, you need to add the Models folder that will be used to create the database entities. In the Models folder, we will create the following database entities.



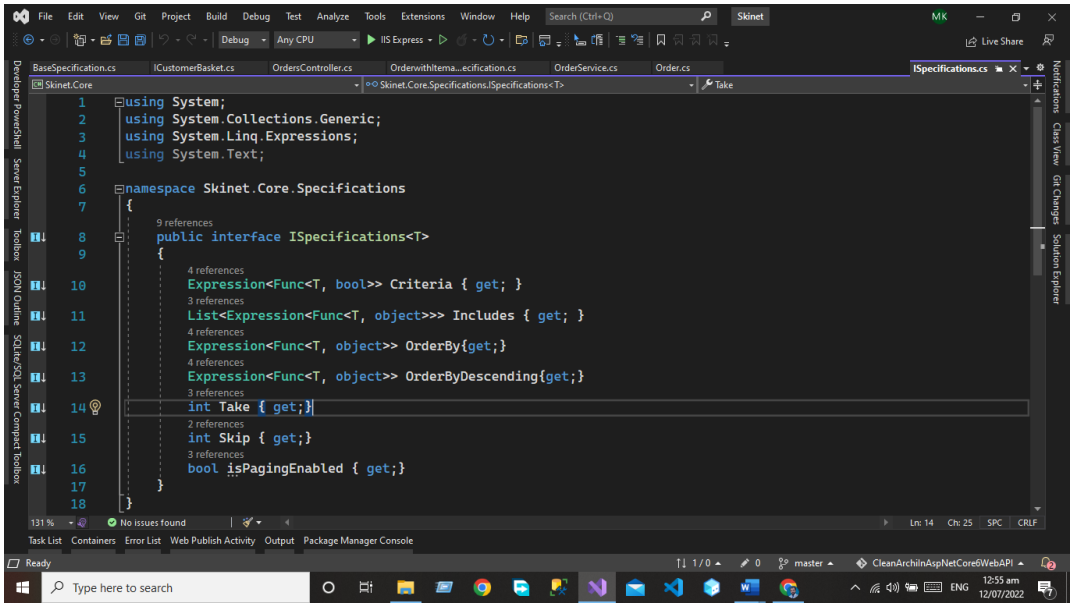
Interface Folder

This folder is used to add the interfaces of the entities that you want to add the specific methods in your Interface.



Specification Folder

This folder is used to add all the specifications lets us take the example if you want the result of the API in ascending OR in descending Order OR want the result in the specific criteria OR want the result in the form of pagination then you need to add the specification class. Let's take the example



Note: Complete Code of the Project will be available on My GitHub

Code Example of Specification Class

```
using System;
using System.Collections.Generic;
using System.Linq.Expressions;
using System.Text;

namespace Skinet.Core.Specifications
{
    public interface ISpecifications<T>
    {
        Expression<Func<T, bool>> Criteria { get; }
        List<Expression<Func<T, object>>> Includes { get; }
        Expression<Func<T, object>> OrderBy{get;}
        Expression<Func<T, object>> OrderByDescending{get;}
        int Take { get;}
        int Skip { get;}
        bool isPagingEnabled { get;}
    }
}
```

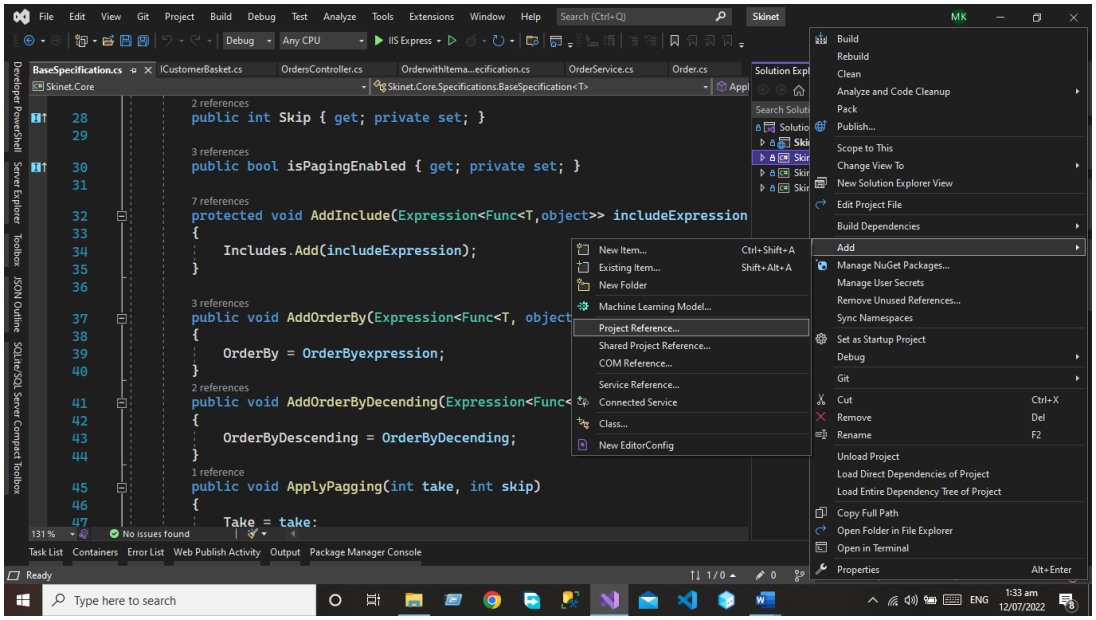
Base Specification Class

```
using System;
using System.Collections.Generic;
using System.Linq.Expressions;
using System.Text;

namespace Skinet.Core.Specifications
{
    public class BaseSpecification<T> : ISpecifications<T>
    {
```

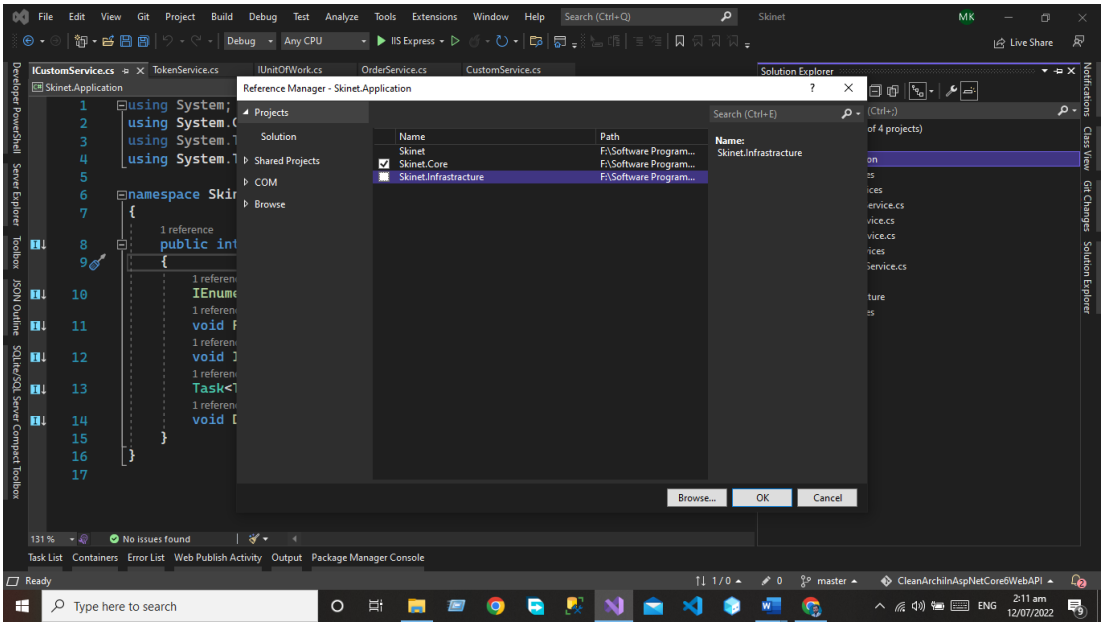
```
        public Expression<Func<T, bool>> Criteria { get; }
        public BaseSpecification()
        {

        }
        public BaseSpecification(Expression<Func<T, bool>> Criteria)
        {
            this.Criteria = Criteria;
        }
        public List<Expression<Func<T, object>>> Includes { get; }
        = new List<Expression<Func<T, object>>>();

        public Expression<Func<T, object>> OrderBy { get; private set;
}

        public Expression<Func<T, object>> OrderByDescending { get;
private set; }

        public int Take { get; private set; }

        public int Skip { get; private set; }

        public bool isPagingEnabled { get; private set; }

        protected void AddInclude(Expression<Func<T,object>>
includeExpression)
        {
            Includes.Add(includeExpression);
        }

        public void AddOrderBy(Expression<Func<T, object>>
OrderByexpression)
        {
            OrderBy = OrderByexpression;
        }
        public void AddOrderByDecending(Expression<Func<T, object>>
OrderByDecending)
        {
            OrderByDescending = OrderByDecending;
        }
        public void ApplyPagging(int take, int skip)
        {
            Take = take;
            //Skip = skip;
            isPagingEnabled = true;
        }
    }
}
```

Application Layer

The application layer contains the business logic all the business logic will be written in this layer. In this layer services interfaces are kept separate from their implementation for loose coupling and separation of concerns.
Now we will add the application layer to our application.
Now we will work on the application Layer we will follow the same process as we did for the Domain layer add the library project in your applicant and give a name to that project Repository layer.
But here we need to add the project reference of the Domain layer in the application layer. Write click on the project and then click the Add button after that we will add the project references in the application layer.



Now select the core project to add the project reference of the domain layer.



Click the OK button After that our project reference will be added to our system.

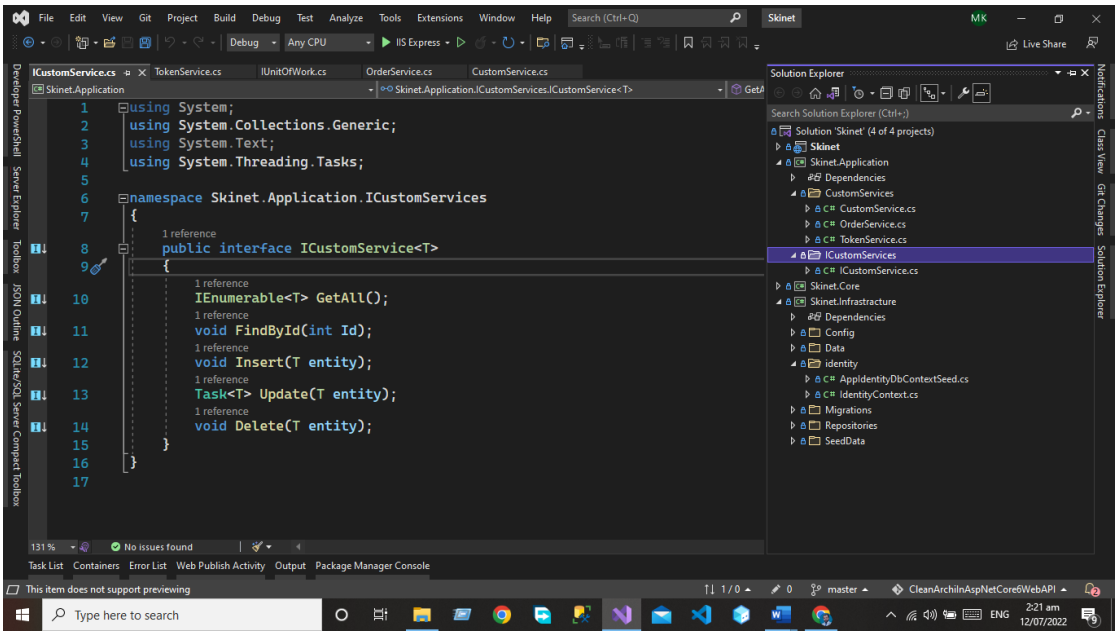
Now let's Create the Desired Folders in our Projects.

I Custom Services

Let's add the I Custom services folder in our application in this folder we will add the I Custom Service Interfaced that will be Inherited by all the services we will add in our Customer Service folder.
Let's add some services to our project.

Code Of the Custom Service

Let us add the Token service that inherits the token service interface from the Core.



Code Of ICustom Interface

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;

namespace Skynet.Application.ICustomServices
{
    public interface ICustomService<T>
    {
        IEnumerable<T> GetAll();
        void FindById(int Id);
        void Insert(T entity);
        Task<T> Update(T entity);
        void Delete(T entity);
    }
}
```

Code of the ICustomerBasket

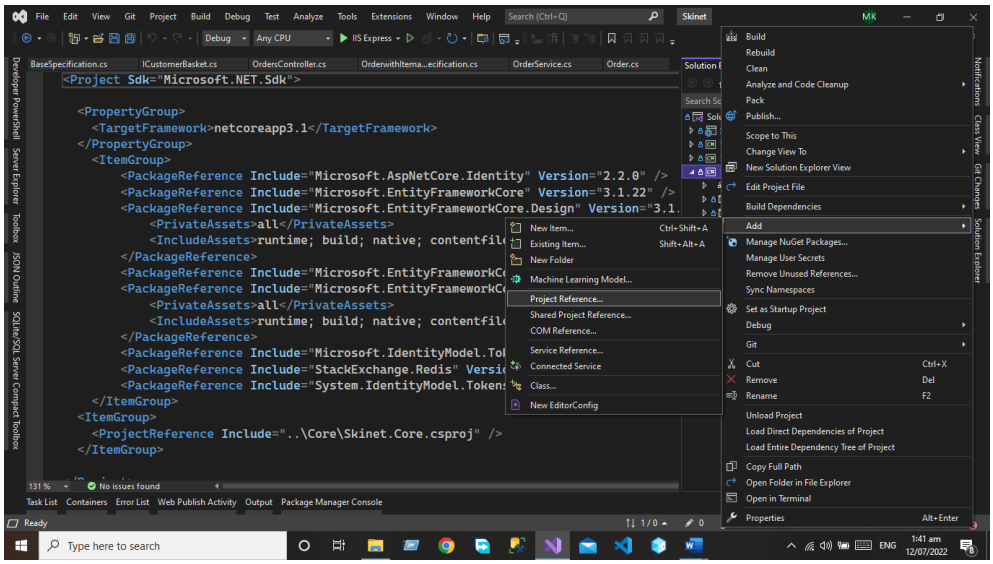
```
using Skinet.Core.Entities;
using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;

namespace Skinet.Core.Interfaces
{
    public interface ICustomerBasket
    {
        Task<CustomerBasket> GetBasketAsync(string basketId);
        Task<CustomerBasket> UpdateBasketAsync(CustomerBasket basket);
        Task<bool> DeleteBasketAsync(string basketId);
    }
}
```

Custom Services Folder

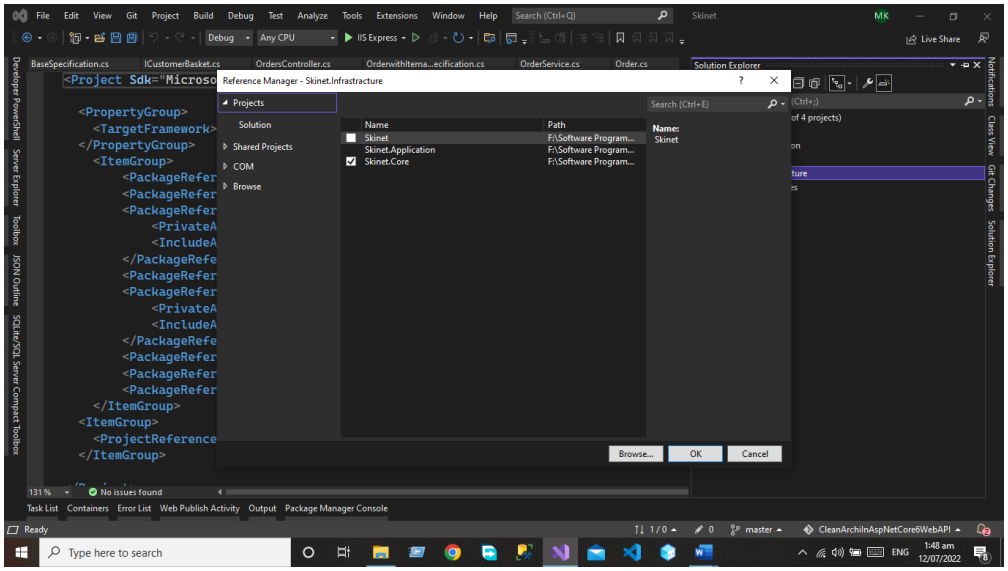
This folder will be used to add the custom services to our system and lets us create some custom services for our project so that our concept will be clear about Custom services. All the custom services will inherit the I Custom services interface using that interface we will add the CRUD Operation in our system.

Infrastructure Layer

In the infrastructure layer, we have model objects we will maintain all the database migrations and database context Objects in this layer. In this layer, we have the repositories of all the domain model objects.
Now we will add the infrastructure layer to our application.
Now we will work on the infrastructure Layer we will follow the same process as we did for the Domain layer add the library project in your applicant and give a name to that project infrastructure layer. But here we need to add the project reference of the Domain layer in the infrastructure layer. Write click on the project and then click the Add button after that we will add the project references in the infrastructure layer.



Now select the Core for adding the domain layer reference in our project.



Implementation of Infrastructure Layer

Let's implement the infrastructure layer in our system.
First, you need to add the Data folder to your infrastructure layer.

Data Folder

Add the Data folder in the infrastructure layer that is used to add the database context class.
The database context class is used to maintain the session with the underlying database using which you can perform the CRUD operation.
In our project, we will add the store context class that will handle the session with our database.

Code of the Database Context Class

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
using Skinet.Core.Entities;
using Skinet.Core.Entities.OrderAggregate;
using System;
using System.Linq;
using System.Reflection;

namespace Skinet.Infrastracture.Data
{
    public class StoreContext : DbContext
    {
        public StoreContext(DbContextOptions<StoreContext> options) :
base(options)
        {

        }
        public DbSet<Products> Products { get; set; }
        public DbSet<ProductType> ProductTypes { get; set; }
        public DbSet<ProductBrand> ProductBrands { get; set; }
```

```
        public DbSet<Order> Orders { get; set; }
        public DbSet<DeliveryMethod> DeliveryMethods { get; set; }
        protected override void OnModelCreating(ModelBuilder
modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

modelBuilder.ApplyConfigurationsFromAssembly(Assembly.GetExecutingAssem
bly());
            if(Database.ProviderName
=="Microsoft.EntityFramework.Sqlite")
            {
                foreach (var entity in
modelBuilder.Model.GetEntityTypes())
                {
                    var properties = entity.ClrType.GetProperties()
                        .Where(p => p.PropertyType == typeof(decimal));
                    var dateandtimepropertise =
entity.ClrType.GetProperties()
                        .Where(t => t.PropertyType ==
typeof(DateTimeOffset));
                    foreach (var property in properties)
                    {

modelBuilder.Entity(entity.Name).Property(property.Name)
                            .HasConversion<double>();
                    }
                    foreach (var property in dateandtimepropertise)
                    {

modelBuilder.Entity(entity.Name).Property(property.Name)
                            .HasConversion(new
DateTimeOffsetToBinaryConverter());
                    }
                }
            }
        }


    }
}
```

Repositories Folder

The repositories folder is used to add the repositories of the domain classes because we are going to implement the repository pattern in our solution.
Note: Complete Code of the Project will be available on My GitHub
Let's use add some repositories to our solution so that our concept about the repositories will be clear.

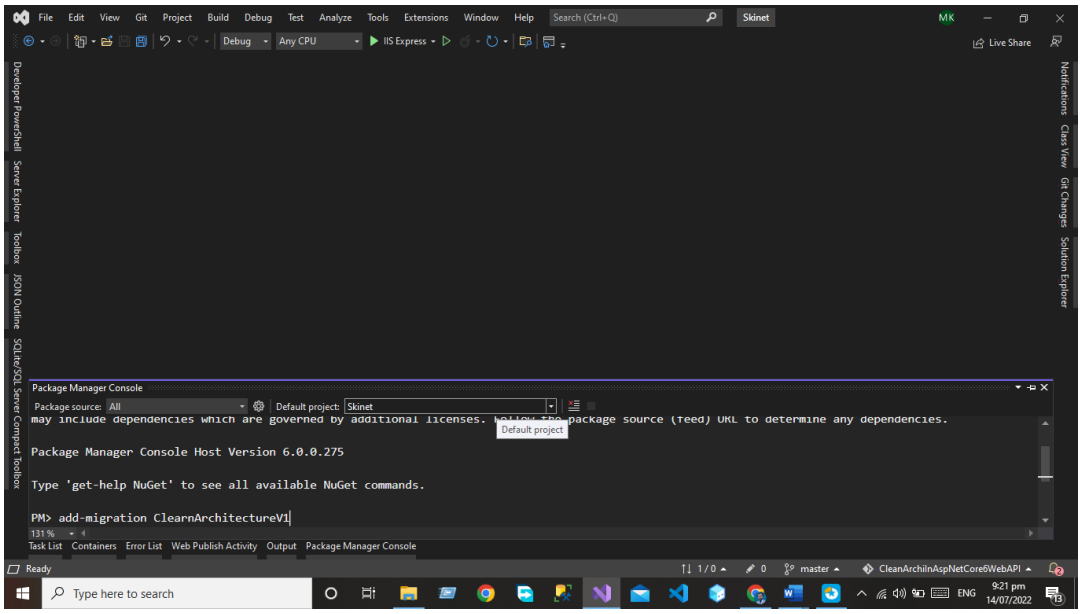
Basket Repository

Let's create the basket as an example to clarify the concept of repositories. That will inherit the Interface of IBasket Interface from the core layer.

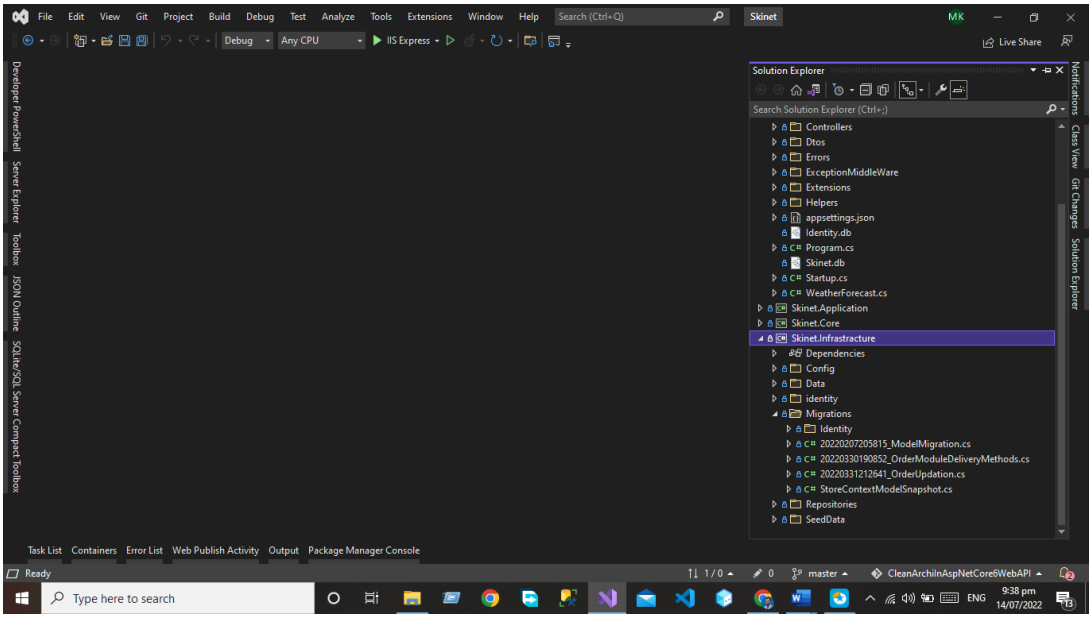
```
using Skinet.Core.Entities;
using Skinet.Core.Interfaces;
using StackExchange.Redis;
using System;
using System.Collections.Generic;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;

namespace Skinet.Infrastracture.Repositories
{
    public class BasketRepository : ICustomerBasket
    {
        private readonly IDatabase _database;
        public BasketRepository(IConnectionMultiplexer radis)
        {
            _database= radis.GetDatabase();
        }
        public async Task<bool> DeleteBasketAsync(string basketId)
        {
            return await _database.KeyDeleteAsync(basketId);
        }

        public async Task<CustomerBasket> GetBasketAsync(string
basketId)
        {
            var data = await _database.StringGetAsync(basketId);
            return data.IsNullOrEmpty ? null :
JsonSerializer.Deserialize<CustomerBasket>(data);
        }

        public async Task<CustomerBasket>
UpdateBasketAsync(CustomerBasket basket)
        {
            var created = await _database.StringSetAsync(basket.Id,

JsonSerializer.Serialize(basket),TimeSpan.FromDays(15));
            if (!created)
            {
                return null;
            }
            return await GetBasketAsync(basket.Id);
        }
    }
}
```

Migrations

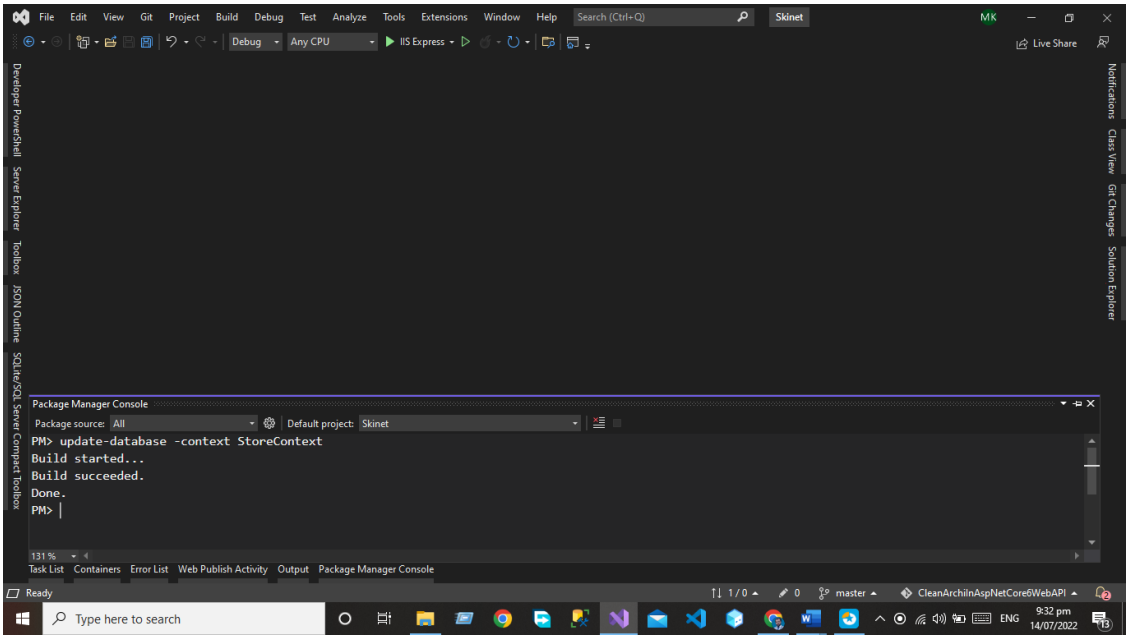
After Adding the DbSet properties we need to add the migration using the package manager console and run the command Add-Migration.
add-migration ClearnArchitectureV1



After executing the Add-Migration Command we have the following auto-generated classes in our migration folder.



After executing the commands now, you need to update the database by executing the update-database Command



Presentation Layer

The presentation layer is our final layer that presents the data to the front-end user on every HTTP request.
In the case of the API presentation layer that presents us the object data from the database using the HTTP request in the form of JSON Object. But in the case of front-end applications, we present the data using the UI by consuming the APIS.

Extensions Folder

Extensions Folder is used for extension methods / Classes we extend functionality C# extension method is a static method of a static class, where the "this" modifier is applied to the first parameter. The type of the first parameter will be the type that is extended. Extension methods are only in scope when you explicitly import the namespace into your source code with a using directive.
Let's Create the static ApplicationServicesExtensions class where we will create the extension method for registering all services we have created during the entire project
Now we need to add the dependency Injection of our all services in the Extensions Classes and then we will use these extensions classes and methods in our startup class.

Code of Extension Method

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.DependencyInjection;
using Skinet.Application.CustomServices;
using Skinet.Core.Interfaces;
using Skinet.Errors;
using Skinet.Infrastructure.Data;
using Skinet.Infrastructure.Repositories;
using Skinet.Infrastructure.SeedData;
using System.Linq;

namespace Skinet.Controllers.Extensions
{
    public static class ApplicationServicesExtensions
```

```
    {
        public static IServiceCollection AddApplicationServices(this
IServiceCollection services)
        {
            services.AddScoped<ITokenService, TokenService>();
            services.AddScoped<StoreContext, StoreContext>();
            services.AddScoped<StoreContextSeed, StoreContextSeed>();
            services.AddScoped<IProductRepository,
ProductRepository>();
            services.AddScoped<ICustomerBasket, BasketRepository>();
            services.AddScoped<IUnitOfWork, UnitOfWork>();
            services.AddScoped<IOrderService, OrderService>();
            services.AddScoped(typeof(IGenericRepository<>),
typeof(GenericRepository<>));
            services.Configure<ApiBehaviorOptions>(options =>
              options.InvalidModelStateResponseFactory = ActionContext
=>
              {
                  var error = ActionContext.ModelState
                              .Where(e => e.Value.Errors.Count > 0)
                              .SelectMany(e => e.Value.Errors)
                              .Select(e => e.ErrorMessage).ToArray();
                  var errorresponce = new APIValidationErrorResponce
                  {
                      Errors = error
                  };
                  return new BadRequestObjectResult(error);
              }
            );
            return services;
        }
    }
}
```

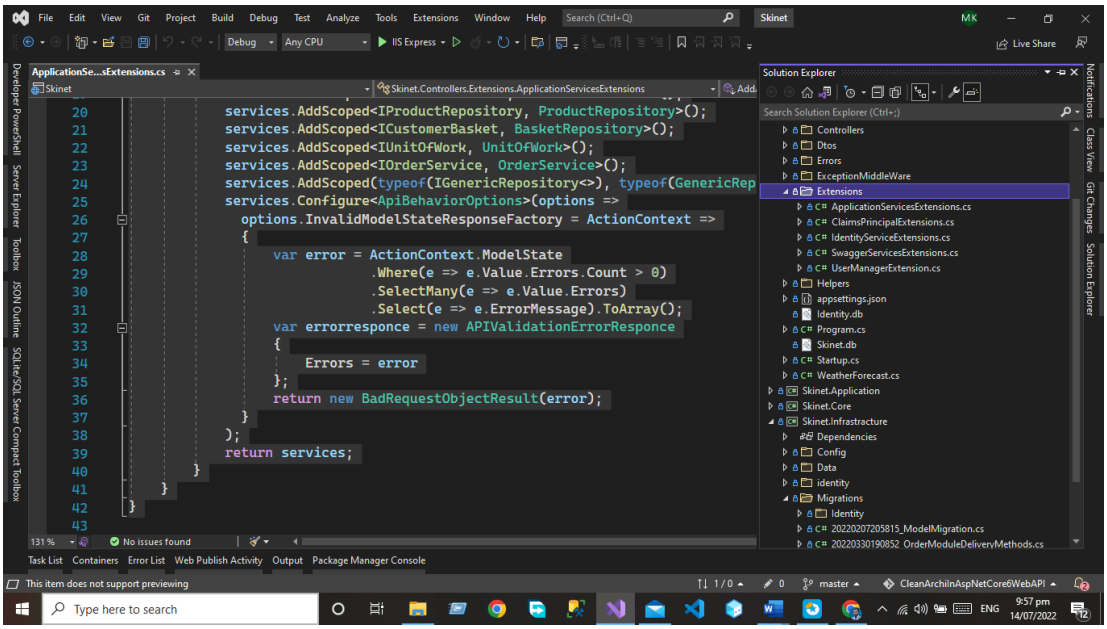
Modify The Startup.cs Class

In Startup.cs Class we will use our extensions method that we have created in extensions folders.

```
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Skinet.Core.Entities;
using Skinet.Core.Interfaces;
using Skinet.Infrastructure.Data;
using Skinet.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Skinet.Infrastructure.SeedData;
```

```
using Skinet.Helpers;
using Skinet.ExceptionMiddleWare;
using Microsoft.AspNetCore.Mvc;
using System.Linq;
using Skinet.Errors;
using Microsoft.OpenApi.Models;
using Skinet.Controllers.Extensions;
using StackExchange.Redis;
using Skinet.Infrastracture.identity;
using Skinet.Extensions;

namespace Skinet
{
    public class Startup
    {
        public IConfiguration Configuration { get; }

        public Startup(IConfiguration configuration)
        {
            Configuration = configuration;
        }
        // This method gets called by the runtime. Use this method to
add services to the container.
        public void ConfigureServices(IServiceCollection services)
        {
            services.AddDbContext<StoreContext>(options =>

options.UseSqlite(Configuration.GetConnectionString("DefaultConnection"
)));
            services.AddDbContext<IdentityContext>(options =>

options.UseSqlite(Configuration.GetConnectionString("DefaultIdentityCon
nection")));
            services.AddSingleton<IConnectionMultiplexer>(c =>
            {
                var configuration = ConfigurationOptions.
                Parse(Configuration.GetConnectionString("Radis"),
true);
                return ConnectionMultiplexer.Connect(configuration);
            });

            services.AddAutoMapper(typeof(MappingProfiles));
            services.AddControllers();
            services.AddApplicationServices();
            services.AddSwaggerDocumentation();
            services.AddIdentityService(Configuration);
            services.AddCors(options =>
            {
```

```
                options.AddPolicy("CorsPolicy", policy =>
                {

policy.AllowAnyHeader().AllowAnyMethod().WithOrigins("*");
                });
            });
        }

        // This method gets called by the runtime. Use this method to
configure the HTTP request pipeline.
        public void Configure(IApplicationBuilder app,
IWebHostEnvironment env)
        {
            app.UseMiddleware<ExceptionMiddle>();
            app.UseStatusCodePagesWithReExecute("/error/{0}");
            app.UseCors("CorsPolicy");
            app.UseHttpsRedirection();
            app.UseSwaggerGen();
            app.UseRouting();
            app.UseStaticFiles();
            app.UseAuthentication();
            app.UseAuthorization();

            app.UseEndpoints(endpoints =>
            {
                endpoints.MapControllers();
            });
        }
    }
}
```

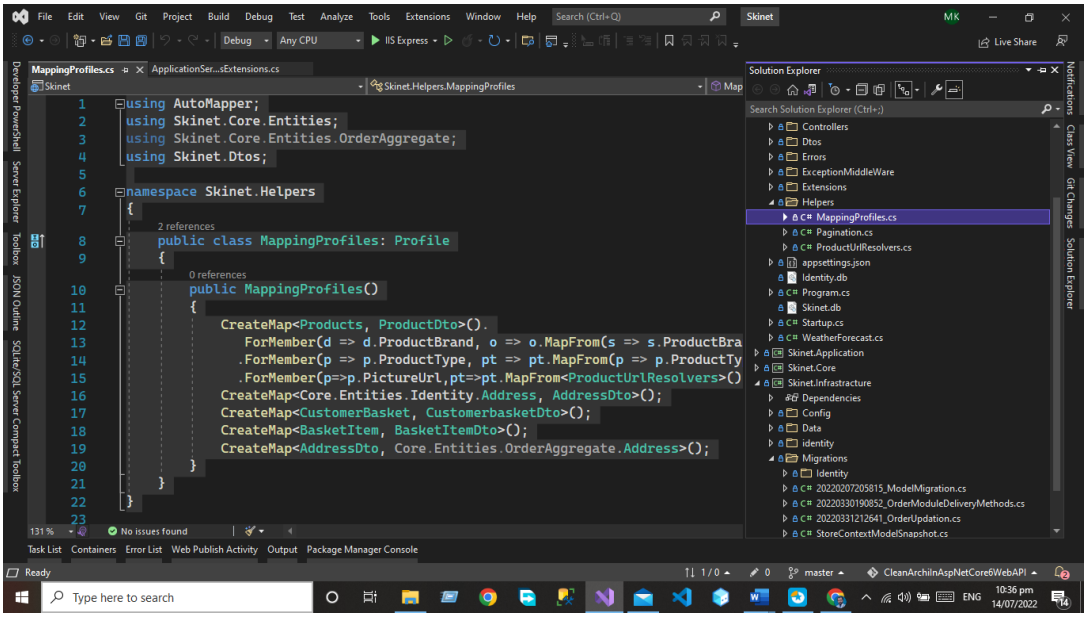


Helpers

Helpers' folder here we create the auto mapper profiles for mapping the entities with each other.

Code Example of Auto Mapper

```
using AutoMapper;
using Skinet.Core.Entities;
using Skinet.Core.Entities.OrderAggregate;
using Skinet.Dtos;

namespace Skinet.Helpers
{
    public class MappingProfiles: Profile
    {
        public MappingProfiles()
        {
            CreateMap<Products, ProductDto>().
               ForMember(d => d.ProductBrand, o => o.MapFrom(s =>
s.ProductBrand.Name))
              .ForMember(p => p.ProductType, pt => pt.MapFrom(p =>
p.ProductType.Name))

.ForMember(p=>p.PictureUrl,pt=>pt.MapFrom<ProductUrlResolvers>());
            CreateMap<Core.Entities.Identity.Address, AddressDto>();
            CreateMap<CustomerBasket, CustomerbasketDto>();
            CreateMap<BasketItem, BasketItemDto>();
            CreateMap<AddressDto,
Core.Entities.OrderAggregate.Address>();
        }
    }
}
```



Data Transfer Object Folder (DTOs)

DTO stands for data transfer object. As the name suggests, a DTO is an object made to transfer data. We create the data transfer object class for mapping the incoming request data.

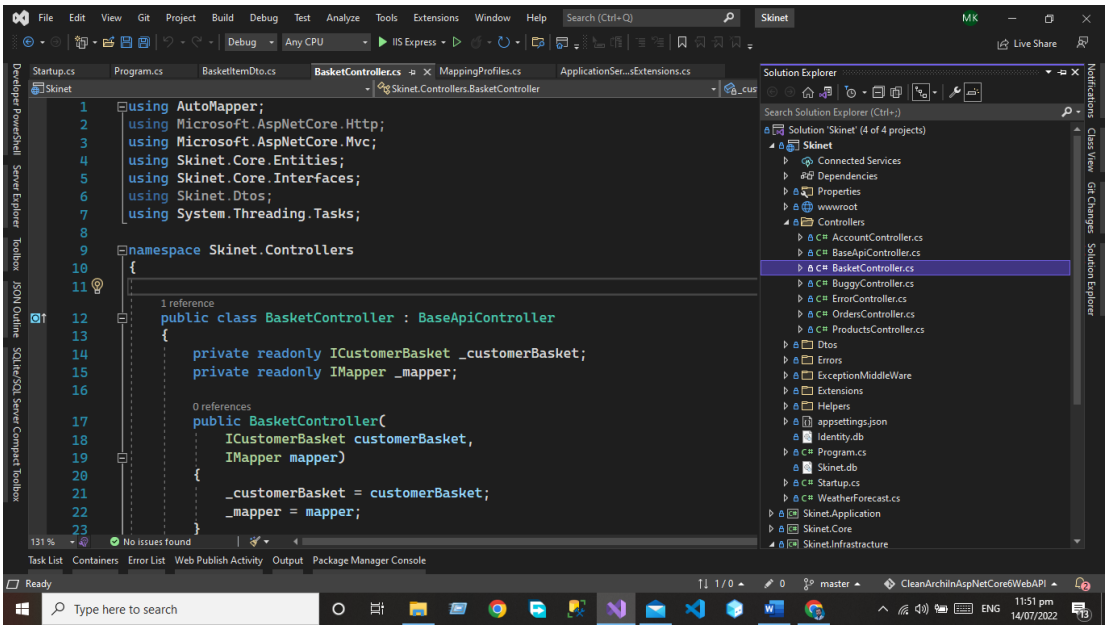
Code of DTOs

Let's us Create the Basket DTOs for clearing the concept about

```
using System.ComponentModel.DataAnnotations;

namespace Skinet.Dtos
{
    public class BasketItemDto
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public int Quantity { get; set; }
        public string PictureUrl { get; set; }
        public string Brand { get; set; }
        public string Type { get; set; }
    }
}
```

Controllers

Controllers are used to handle the HTTP request. Now we need to add the student controller that will interact will our service layer and display the data to the users.
In this chapter, we will focus on Basket controllers because in the whole chapter we talk about Basket.



Code of Basket Controller

```
using AutoMapper;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Skinet.Core.Entities;
using Skinet.Core.Interfaces;
using Skinet.Dtos;
using System.Threading.Tasks;

namespace Skinet.Controllers
{
    public class BasketController : BaseApiController
    {
        private readonly ICustomerBasket _customerBasket;
        private readonly IMapper _mapper;

        public BasketController(
            ICustomerBasket customerBasket,
            IMapper mapper)
        {
            _customerBasket = customerBasket;
            _mapper = mapper;
        }
        [HttpGet(nameof(GetBasketElement))]
        public async Task<ActionResult<CustomerBasket>>
GetBasketElement([FromQuery]string Id)
        {
            var basketelements = await
_customerBasket.GetBasketAsync(Id);
            return Ok(basketelements??new CustomerBasket(Id));
        }
        [HttpPost(nameof(UpdateProduct))]
        public async Task<ActionResult<CustomerBasket>>
UpdateProduct(CustomerBasket product)
        {
            //var customerbasket = _mapper.Map<CustomerbasketDto,
CustomerBasket>(product);
            var data =  await
_customerBasket.UpdateBasketAsync(product);
            return Ok(data);
        }
        [HttpDelete(nameof(DeleteProduct))]
        public async Task DeleteProduct(string Id)
        {
            await _customerBasket.DeleteBasketAsync(Id);
        }
    }
}
```

Let’s us Take another example of Product Controller

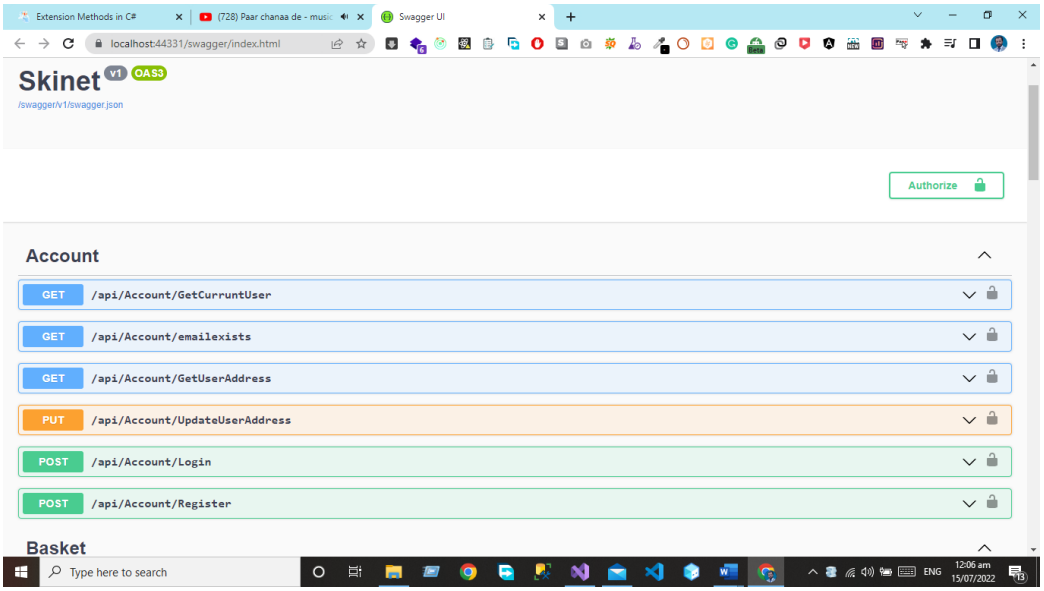
Code of the Product Controller

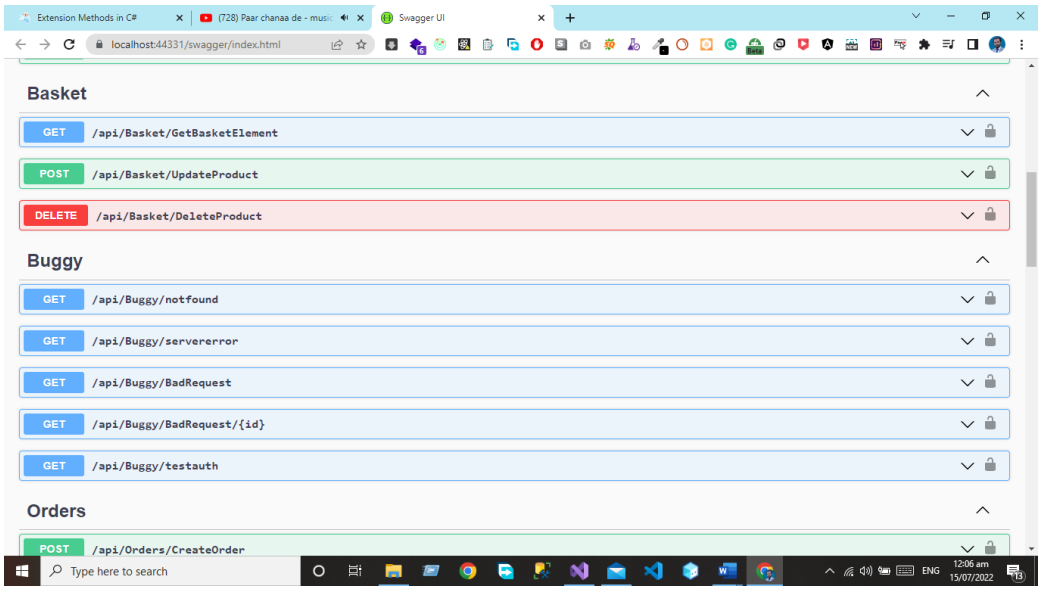
```
using AutoMapper;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Skinet.Core.Entities;
using Skinet.Core.Interfaces;
using Skinet.Core.Specifications;
using Skinet.Dtos;
using Skinet.Errors;
using Skinet.Helpers;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace Skinet.Controllers
{

    public class ProductsController : BaseApiController
    {
        private readonly IGenericRepository<Products>
_productRepository;
        private readonly IGenericRepository<ProductBrand>
_brandRepository;
        private readonly IGenericRepository<ProductType> _productType;
        public IMapper _Mapper;

        public ProductsController(IGenericRepository<Products>
productRepository,
            IGenericRepository<ProductBrand> BrandRepository,
            IGenericRepository<ProductType> productType, IMapper
mapper)
        {
            _productRepository = productRepository;
            _brandRepository = BrandRepository;
            _productType = productType;
            _Mapper = mapper;
        }
        [HttpGet(nameof(GetProducts))]
        public async Task<ActionResult<Pagination<ProductDto>>>
GetProducts(
            [FromQuery]ProductSpecPrams productSpecPrams)
        {
            var spec = new
ProductWithSpecificationTypesAndBrand(productSpecPrams);
            var speccount = new
ProductWithFilterCountSpecification(productSpecPrams);
            var total = await _productRepository.CountAsync(spec);
```

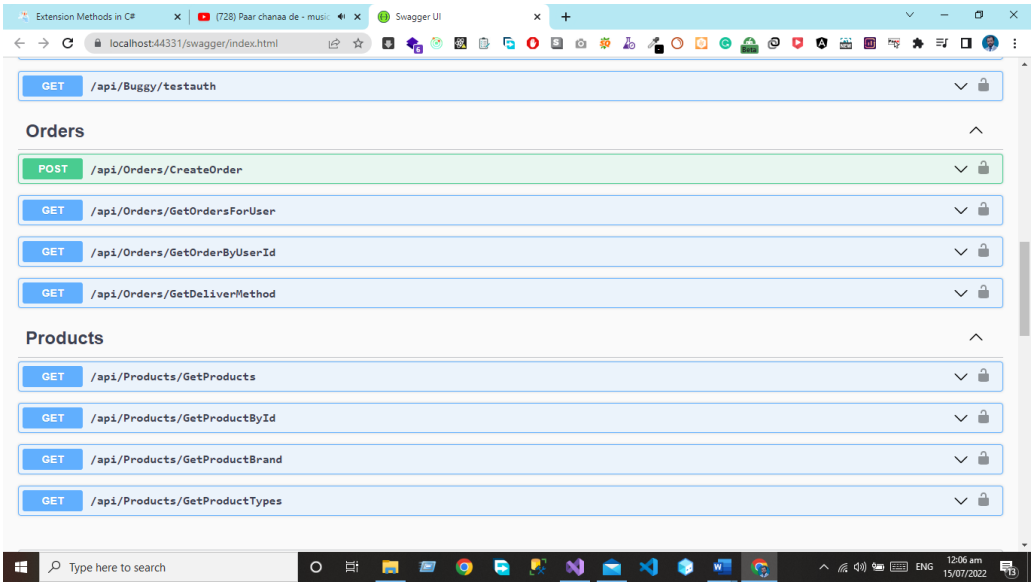
```
            var products = await _productRepository.ListAsync(spec);
            var data = _Mapper.Map<IReadOnlyList<Products>,
IReadOnlyList<ProductDto>>(products);
            return Ok(new
Pagination<ProductDto>(productSpecPrams.pageIndex,
productSpecPrams.pageSize, total, data));
        }

        [HttpGet(nameof(GetProductById))]
        [ProducesResponseType(StatusCodes.Status200OK)]

[ProducesResponseType(typeof(APIResponce),StatusCodes.Status404NotFound
)]
        public async Task<ProductDto> GetProductById([FromQuery] int
Id)
        {
            var spec = new ProductWithSpecificationTypesAndBrand(Id);
            var products = await _productRepository.GetByIdAsync(Id);
            return _Mapper.Map<ProductDto>(products);
        }

        [HttpGet(nameof(GetProductBrand))]
        public async Task<ActionResult<List<ProductBrand>>>
GetProductBrand()
        {
            var obj = await _brandRepository.GetAllAsync();
            return Ok(obj);
        }

        [HttpGet(nameof(GetProductTypes))]
        public async Task<ActionResult<List<ProductType>>>
GetProductTypes()
        {
            var obj = await _productRepository.GetAllAsync();
            return Ok(obj);
        }
    }
}
```

Output

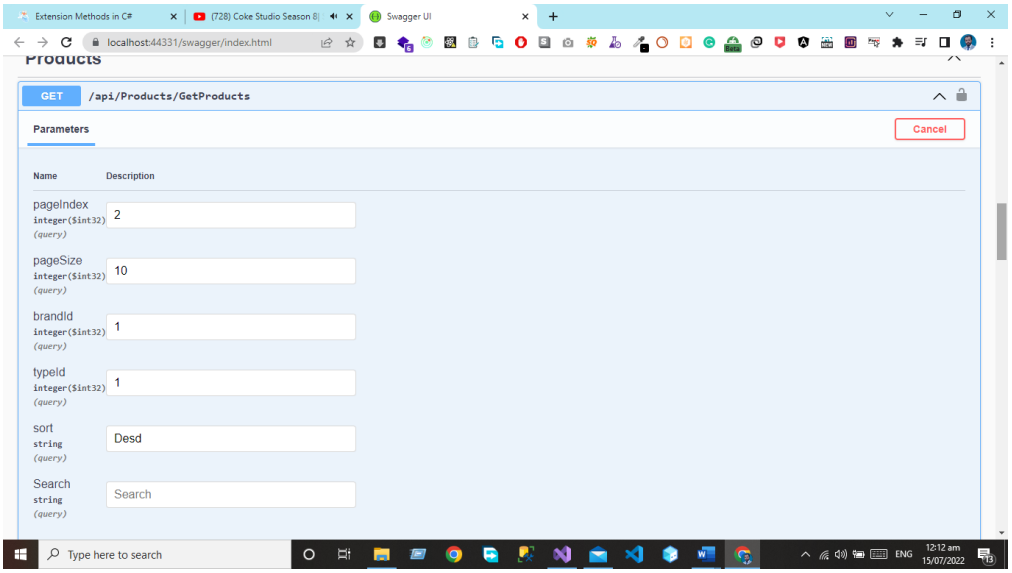
Now we will run the project and will see the output using the swagger.



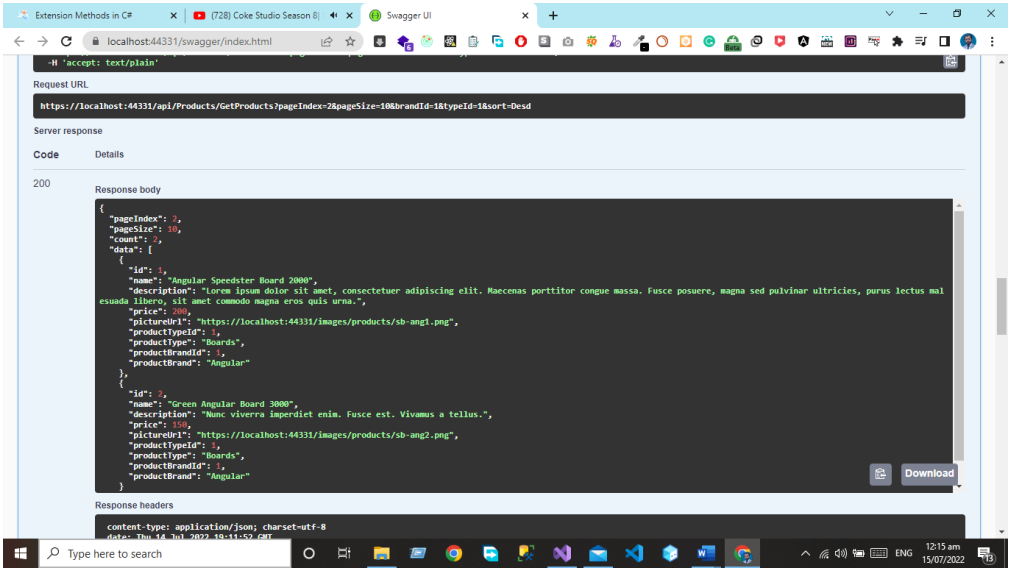




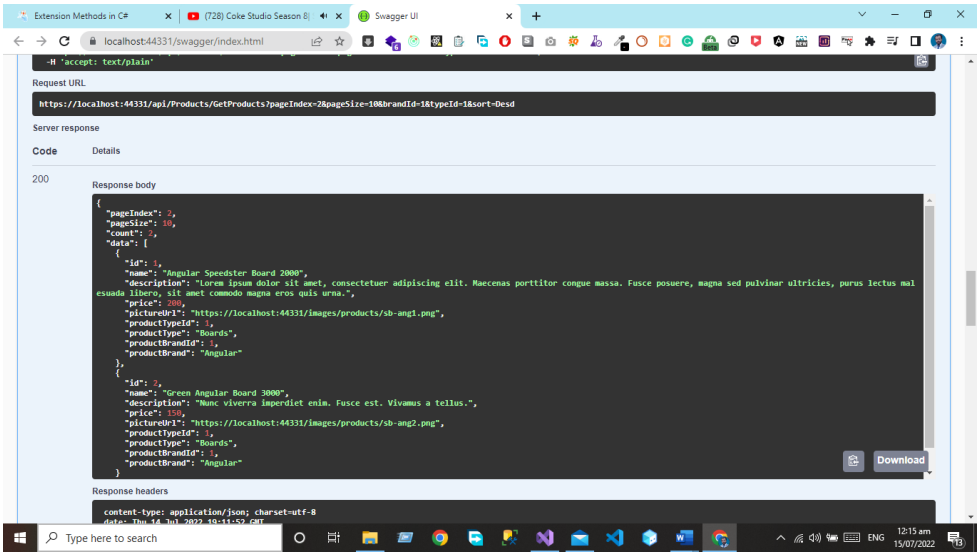
Let’s Get the Product using the Product Controller



These are the parameter that is very helpful for getting the desired data.



Now we can see when we hit the get products endpoint, we can see the data of students from the database in the form of the JSON object.



Conclusion

In this chapter, we have implemented the clean architecture using the Entity Framework and Code First approach. We have now the knowledge of how the layer communicates with each other's in clean architecture and how we can write the Generic code for the Interface repository and services. Now we can develop our project using clean architecture for API Development OR MVC Core Based projects.

Complete Project Code

You can download the complete project code from the following GitHub repository. Follow the link below and down the complete project code and practice the code by yourself and learn how we can implement the clean architecture in asp.net code Webmail.
The complete Code of the project will be available by following the given below link.

Click the link for the git hub repository

6

Building Microservice Using CQRS Design Pattern

Overview

In this chapter, we explore Microservices and CQRS, covering CQRS fundamentals, Microservices Boundaries, and Command and Query Handlers. We explore Separating Data Models, Event Sourcing, and Asynchronous Communication. Additionally, we discuss Testing and Monitoring, Deployment and Scaling, and Documentation and Communication, concluding with insights on Continuous Integration and Communication.

Introduction to Microservices and CQRS

In recent years, microservices architecture has gained popularity as a way to build scalable, flexible, and maintainable systems. Microservices decompose applications into smaller, independently deployable services, each focusing on a specific business domain or functionality. Command Query Responsibility Segregation (CQRS) is a design pattern that complements microservices architecture by separating the concerns of reading and writing data.

Understanding CQRS

CQRS stands for Command Query Responsibility Segregation. It is a design pattern that advocates for separating the concerns of commands (write operations) and queries (read operations) into separate models. This separation allows for independent scaling, optimization, and evolution of the read and write sides of an application.

Identifying Microservice Boundaries

Microservices should be designed around business capabilities or bounded contexts. Each microservice should encapsulate a specific business domain and have a clear responsibility. When applying CQRS within microservices architecture, it's essential to identify clear boundaries to separate command and query responsibilities.

Implementing Command and Query Handlers

Within each microservice, separate handlers are implemented for commands and queries. Command handlers are responsible for executing commands that modify the application state, while query handlers are responsible for retrieving data from the application state.

Separating Data Models

CQRS advocates for maintaining separate data models for read and write operations. Write models are optimized for data modification and may not match the read models exactly. Read models are denormalized and optimized for efficient querying.

Event Sourcing (Optional)

Event sourcing is often used in conjunction with CQRS. Event sourcing involves capturing all changes to an application's state as a sequence of events. These events can be used to reconstruct the current state of the application and provide audit trails. Event sourcing complements the CQRS pattern by providing a mechanism for reliably capturing and storing changes to the application state.

Asynchronous Communication

Microservices implementing CQRS often use asynchronous communication between the command and query sides of the system. Commands may be executed asynchronously to improve responsiveness and scalability, while queries can be optimized for performance by using caching and other techniques.

Testing and Monitoring

Thorough testing is essential for both the command and query sides of a microservice implementing CQRS. Unit tests, integration tests, and end-to-end tests should be conducted to ensure the correctness and reliability of the system. Additionally, monitoring tools should be

used to monitor the performance and health of the microservices, allowing for timely detection and resolution of issues.

Deployment and Scaling

Microservices implementing CQRS can be deployed using containerization technologies such as Docker and orchestrated using tools like Kubernetes. This allows for easy deployment, scaling, and management of microservices. Read and write components can be scaled independently based on the workload, providing flexibility and efficiency.

Documentation and Communication

Clear documentation of the command and query contracts is essential for effective communication between microservices. Developers should understand the design decisions and guidelines governing the implementation of CQRS within the microservices architecture to ensure consistency and maintainability.

Continuous Improvement

Microservices architecture is an evolving paradigm, and continuous improvement is key to its success. Feedback from users and stakeholders should be incorporated into the design and implementation of microservices. Refactoring and optimization should be performed regularly to maintain a scalable, flexible, and maintainable system.

Conclusion

In conclusion, building microservices using the Command Query Responsibility Segregation (CQRS) design pattern offers numerous benefits, including scalability, flexibility, and maintainability. By separating the concerns of reading and writing data, CQRS allows for independent scaling, optimization, and evolution of microservices. When implementing CQRS within a microservices architecture, it's essential to identify clear boundaries, implement separate handlers for commands and queries, maintain separate data models, consider event sourcing, use asynchronous communication, conduct thorough testing and monitoring, deploy and scale effectively, document and communicate effectively, and continuously improve the system. By following these principles and best practices, developers can build robust, scalable, and maintainable microservices architectures using the CQRS design pattern.

7

Practical Implementation of CQRS Design Pattern

Overview

In this chapter, we introduce the CQRS Design Pattern and guide you through creating solutions, models, and command/query handlers for managing products. We cover implementing the Repository Pattern and provide insights into the key principles and benefits of CQRS.

Introduction

CQRS stands for Command Query Responsibility Segregation. It is a design pattern that advocates for separating the concerns of commands (write operations) and queries (read operations) into separate models. This separation allows for independent scaling, optimization, and evolution of the read and write sides of an application.

Implementation Of CQRS Design Pattern.

Implementing the CQRS (Command Query Responsibility Segregation) pattern in ASP.NET Core Web API involves separating read and write operations. It's a structural pattern that can help in scaling and maintaining applications. Here, I'll guide you through the steps for implementing CQRS in an ASP.NET Core Web API.
For brevity, I'll provide a high-level overview with code snippets. You'll need to adapt and extend these examples to suit your specific needs and structure.

Create a Solution and Projects

Create an ASP.NET Core Web API project and two separate projects for commands and queries.

Create a Model for Product

```
namespace MicroservicesWithCQRSDesignPattern.Model
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

Create a Model for Query with data Filters

```
namespace MicroservicesWithCQRSDesignPattern.Model
{
    public class GetProductsQuery
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public int PageNumber { get; set; }
        public int PageSize { get; set; }
        public string SearchTerm { get; set; }
        public decimal? MinPrice { get; set; }
        public decimal? MaxPrice { get; set; }
    }
}
```

Command and Query Models for Create Product

```
Create models for commands and queries. For instance.
namespace MicroservicesWithCQRSDesignPattern.Quries.CommandModel
{
    public class CreateProductCommand
    {
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

Command and Query Models for Delete Product

```
namespace MicroservicesWithCQRSDesignPattern.Quries.QueryModel
{
    public class DeleteProductCommand
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

Command and Query Models for Update Product

```
namespace MicroservicesWithCQRSDesignPattern.Quries.QueryModel
{
    public class UpdateProductCommand
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

Create Interface for ICommandHandler<TCommand>

```
namespace MicroservicesWithCQRSDesignPattern.Interfaces
{
    public interface ICommandHandler<TCommand>
    {
        Task Handle(TCommand command);
    }
}
```

Command and Query Models for Get All Product

```
namespace MicroservicesWithCQRSDesignPattern.Quries.QueryModel
{
    public class GetAllProductCommand
    {
```

```
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

Create Interface for IQueryHandler<TQuery, TResult>

```
namespace MicroservicesWithCQRSDesignPattern.Interfaces
{
    public interface IQueryHandler<TQuery, TResult>
    {
        Task<TResult> Handle(TQuery query);
    }
}
```

Create an Interface for IRepository

```
namespace MicroservicesWithCQRSDesignPattern.Interfaces
{
    public interface IRepository<T>
    {
        Task<T> GetByIdAsync(int id);
        Task<IEnumerable<T>> GetAllAsync();
        Task AddAsync(T entity);
        Task UpdateAsync(T entity);
        Task DeleteAsync(T entity);
        Task SaveAsync();
    }
}
```

Implement the Repository Pattern for Products

```
using MicroservicesWithCQRSDesignPattern.AppDbContext;
using MicroservicesWithCQRSDesignPattern.Interfaces;
using MicroservicesWithCQRSDesignPattern.Model;
using Microsoft.EntityFrameworkCore;
namespace MicroservicesWithCQRSDesignPattern.Repository
{
    public class ProductRepository : IRepository<Product>
    {
        private readonly ApplicationDbContext _dbContext;
        public ProductRepository(ApplicationDbContext dbContext)
        {
            _dbContext = dbContext;
        }
        public async Task<Product> GetByIdAsync(int id)
        {
            return await _dbContext.Set<Product>().FindAsync(id);
        }
        public async Task<IEnumerable<Product>> GetAllAsync()
```

```
        {
            return await _dbContext.Set<Product>().ToListAsync();
        }
        public async Task AddAsync(Product entity)
        {
            await _dbContext.Set<Product>().AddAsync(entity);
        }
        public async Task UpdateAsync(Product entity)
        {
            _dbContext.Set<Product>().Update(entity);
        }
        public async Task DeleteAsync(Product entity)
        {
            _dbContext.Set<Product>().Remove(entity);
        }
        public async Task SaveAsync()
        {
            await _dbContext.SaveChangesAsync();
        }
    }
}
```

Create Handlers for Create, Update, Delete, GetAllProducts

Create Product Command Handler

```
using MicroservicesWithCQRSDesignPattern.Interfaces;
using MicroservicesWithCQRSDesignPattern.Model;
using MicroservicesWithCQRSDesignPattern.Quries.CommandModel;
namespace MicroservicesWithCQRSDesignPattern.Handlers
{
    public class CreateProductCommandHandler :
ICommandHandler<CreateProductCommand>
    {
        private readonly IRepository<Product> _repository;

        public CreateProductCommandHandler(IRepository<Product>
repository)
        {
            _repository = repository;
        }
        public async Task Handle(CreateProductCommand command)
        {
            var product = new Product
            {
                Name = command.Name,
                Price = command.Price
            };
            await _repository.AddAsync(product);
```

```
            await _repository.SaveAsync();
        }
    }
}
```

Delete Product Command Handler

```
using MicroservicesWithCQRSDesignPattern.Interfaces;
using MicroservicesWithCQRSDesignPattern.Quries.CommandModel;
using MicroservicesWithCQRSDesignPattern.Model;
using MicroservicesWithCQRSDesignPattern.Quries.QueryModel;
namespace MicroservicesWithCQRSDesignPattern.Handlers
{
    public class DeleteProductCommandHandler :
ICommandHandler<DeleteProductCommand>
    {
        private readonly IRepository<Product> _repository;

        public DeleteProductCommandHandler(IRepository<Product>
repository)
        {
            _repository = repository;
        }
        public async Task Handle(DeleteProductCommand command)
        {
            var productToDelete = await
_repository.GetByIdAsync(command.Id);

            if(productToDelete != null)
            {
                await _repository.DeleteAsync(productToDelete);
            }
            else
            {
                throw new Exception("Product not found"); // Handle
product not found scenario
            }
        }
    }
}
```

Get Products Query Handler

```
using MicroservicesWithCQRSDesignPattern.Interfaces;
using MicroservicesWithCQRSDesignPattern.Model;
using MicroservicesWithCQRSDesignPattern.Quries.QueryModel;
namespace MicroservicesWithCQRSDesignPattern.Handlers
{
    public class GetProductsQueryHandler :
IQueryHandler<GetProductsQuery,   IEnumerable<GetAllProductCommand>>
```

```
    {
        private readonly IRepository<Product> _repository; // Inject
repository or database context
        public GetProductsQueryHandler(IRepository<Product> repository)
        {
            _repository = repository;
        }
        public async Task<IEnumerable<GetAllProductCommand>>
Handle(GetProductsQuery query)
        {
            var products = await _repository.GetAllAsync(); //
Implement repository method
            // Map products to ProductViewModel
            return products.Select(p => new GetAllProductCommand
            {
                Id = p.Id,
                Name = p.Name,
                Price = p.Price
            });
        }
    }
}
```

Update Product Command Handler

```
using MicroservicesWithCQRSDesignPattern.Interfaces;
using MicroservicesWithCQRSDesignPattern.Quries.CommandModel;
using MicroservicesWithCQRSDesignPattern.Model;
using MicroservicesWithCQRSDesignPattern.Quries.QueryModel;
namespace MicroservicesWithCQRSDesignPattern.Handlers
{
    public class UpdateProductCommandHandler :
ICommandHandler<UpdateProductCommand>
    {
        private readonly IRepository<Product> _repository;
        public UpdateProductCommandHandler(IRepository<Product>
repository)
        {
            _repository = repository;
        }
        public async Task Handle(UpdateProductCommand command)
        {
            var productToUpdate = await
_repository.GetByIdAsync(command.Id);
            if(productToUpdate != null)
            {
                productToUpdate.Name = command.Name;
                // Save changes to the repository
                await _repository.UpdateAsync(productToUpdate);
```

```
            }
            else
            {
                throw new Exception("Product not found"); // Handle
product not found scenario
            }
        }
    }
}
```

Dependency Injection of Services and Handler in IOC Container

```
using MicroservicesWithCQRSDesignPattern.AppDbContext;
using MicroservicesWithCQRSDesignPattern.Handlers;
using MicroservicesWithCQRSDesignPattern.Interfaces;
using MicroservicesWithCQRSDesignPattern.Model;
using MicroservicesWithCQRSDesignPattern.Quries.CommandModel;
using MicroservicesWithCQRSDesignPattern.Quries.QueryModel;
using MicroservicesWithCQRSDesignPattern.Repository;
using Microsoft.EntityFrameworkCore;
var builder = WebApplication.CreateBuilder(args);
var configuration = builder.Configuration;
// Add services to the container.
builder.Services.AddDbContext<ApplicationDbContext>(options =>
{

options.UseSqlServer(configuration.GetConnectionString("DefaultConnecti
on")); // Replace with your database provider and connection string
});
builder.Services.AddScoped<IRepository<Product>, ProductRepository>();
builder.Services.AddTransient<ICommandHandler<CreateProductCommand>,
CreateProductCommandHandler>();
builder.Services.AddTransient<IQueryHandler<GetProductsQuery,
IEnumerable<GetAllProductCommand>>, GetProductsQueryHandler>();
builder.Services.AddTransient<ICommandHandler<UpdateProductCommand>,
UpdateProductCommandHandler>();
builder.Services.AddTransient<ICommandHandler<DeleteProductCommand>,
DeleteProductCommandHandler>();
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
// Configure the HTTP request pipeline.
if(app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection();
```

```
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Application Database Context Class

```
using MicroservicesWithCQRSDesignPattern.Model;
using Microsoft.EntityFrameworkCore;
namespace MicroservicesWithCQRSDesignPattern.AppDbContext
{
    public class ApplicationDbContext : DbContext
    {
        public
ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options)
        {
        }
        public DbSet<Product> Products { get; set; }
        protected override void OnModelCreating(ModelBuilder
modelBuilder)
        {
            //modelBuilder.Entity<Entity>().HasKey(e => e.Id);
        }
    }
}
```

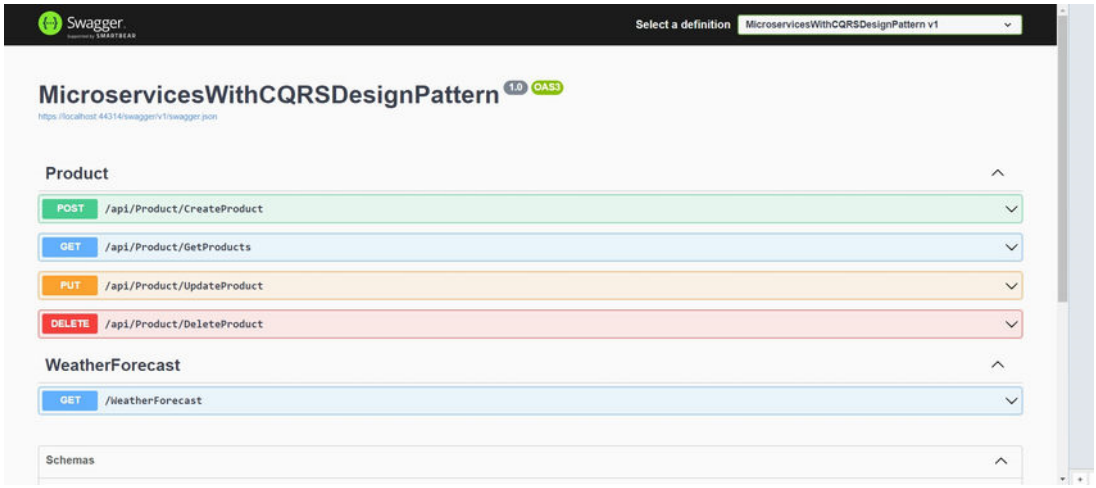
Create a Controller for Handling the HTTP Requests

```
using MicroservicesWithCQRSDesignPattern.Interfaces;
using MicroservicesWithCQRSDesignPattern.Model;
using MicroservicesWithCQRSDesignPattern.Quries.CommandModel;
using MicroservicesWithCQRSDesignPattern.Quries.QueryModel;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
namespace MicroservicesWithCQRSDesignPattern.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        private readonly ICommandHandler<CreateProductCommand>
_createProductCommandHandler;
        private readonly IQueryHandler<GetProductsQuery,
IEnumerable<GetAllProductCommand>> _getProductsQueryHandler;
        private readonly ICommandHandler<UpdateProductCommand>
_updateProductCommandHandler;
        private readonly ICommandHandler<DeleteProductCommand>
_deleteProductCommandHandler;

        public ProductController(
```

```
            ICommandHandler<CreateProductCommand>
createProductCommandHandler,
            IQueryHandler<GetProductsQuery,
IEnumerable<GetAllProductCommand>> getProductsQueryHandler,
            ICommandHandler<UpdateProductCommand>
updateProductCommandHandler)
        {
            _createProductCommandHandler = createProductCommandHandler;
            _getProductsQueryHandler = getProductsQueryHandler;
            _updateProductCommandHandler = updateProductCommandHandler;
        }
[HttpPost(nameof(CreateProduct))]
        public async Task<IActionResult>
CreateProduct(CreateProductCommand command)
        {
            await _createProductCommandHandler.Handle(command);
            return Ok();
        }
        [HttpGet(nameof(GetProducts))]
        public async Task<IActionResult> GetProducts()
        {
            var products = await _getProductsQueryHandler.Handle(new
GetProductsQuery());
            return Ok(products);
        }
        [HttpPut(nameof(UpdateProduct))]
        public async Task<IActionResult>
UpdateProduct(UpdateProductCommand command)
        {
            try
            {
                await _updateProductCommandHandler.Handle(command);
                return Ok("Product updated successfully");
            }
            catch(Exception ex)
            {
                return
StatusCode(StatusCodes.Status500InternalServerError, $"Error updating
product: {ex.Message}");
            }
        }

        [HttpDelete(nameof(DeleteProduct))]
        public async Task<IActionResult> DeleteProduct(int productId)

        {
            try
            {
```

```
                var command = new DeleteProductCommand { Id = productId
};
                await _deleteProductCommandHandler.Handle(command);
                return Ok("Product deleted successfully");
            }
            catch(Exception ex)
            {
                return
StatusCode(StatusCodes.Status500InternalServerError, $"Error deleting
product: {ex.Message}");
            }
        }
    }
}
```

Output: Output of the Microservice with CQRS Design Pattern.



GitHub Project Link

https://github.com/SardarMudassarAliKhan/MicroservicesWithCQRSDesignPattern

Conclusion

The Command Query Responsibility Segregation (CQRS) pattern is an architectural principle that separates the responsibility for handling commands (write operations that change state) from queries (read operations that retrieve state). It advocates having separate models for reading and writing data.

Components of CQRS

Command: Represents an action that mutates the system's state.

- **Query:** Represents a request for data retrieval without changing the system's state.
- **Command Handler:** Responsible for executing commands and updating the system's state.
- **Query Handler:** Responsible for handling read requests and returning data in response to queries.
- **Command Model:** Contains the logic and rules necessary to process commands and update the data store.

- **Query Model:** Optimized for querying and presenting data to users, often involving denormalized or optimized data structures tailored for specific queries.

Key Principles

- **Separation of Concerns:** Splitting the responsibilities of reading and writing data helps in maintaining simpler, more focused models for each task.
- **Performance Optimization:** Enables independent scaling of read and write operations. The read model can be optimized for query performance without affecting the write model.
- **Flexibility:** Allows for different models to be used for reading and writing, which can cater to specific requirements and optimizations for each use case.
- **Complex Domain Logic:** Particularly beneficial in domains where read and write logic significantly differ, allowing tailored models for each type of operation.

Benefits

- **Scalability:** CQRS enables scaling read and write operations independently, optimizing performance.
- **Flexibility and Optimization:** Tailoring models for specific tasks allows for better optimization of the system.
- **Complexity Management:** Separating concerns can make the system easier to understand and maintain.

Challenges

- **Increased Complexity:** Introducing separate models for reading and writing can add complexity to the system.
- **Synchronization:** Keeping the read and write models synchronized can pose challenges, potentially requiring mechanisms like eventual consistency.

CQRS is not a one-size-fits-all solution and is typically employed in systems with complex business logic or were read and write operations vastly differ in terms of frequency, complexity, or optimization requirements. Its application should be carefully considered based on the specific needs and trade-offs of a given system.

8

Building a Microservices API Gateway with Ocelot

Overview

In this chapter, we delve into Building a Microservices API Gateway with Ocelot, exploring its core concepts and practical applications. We start by defining What an API Gateway is and then focus on What Ocelot API Gateway is. Prepare to learn about API Gateway Implementation using Ocelot in ASP.NET Core Web API and gain hands-on experience with Practical Implementation. We discuss Why We Use an API Gateway, along with its Advantages and Disadvantages, providing a balanced view of its role in microservices architecture.

Building a Microservices API Gateway with Ocelot

Ocelot is a popular open-source API Gateway built for .NET and designed to work within a microservices architecture. In a microservices-based system, various services need to communicate with each other. An API Gateway like Ocelot serves as a central entry point for all incoming requests, providing various features and benefits:

Routing: Ocelot allows you to define routing rules for incoming requests, directing them to the appropriate microservice based on URL patterns, HTTP methods, or other criteria. This simplifies the client-side experience as clients only need to interact with the API Gateway.

Load Balancing: Ocelot can distribute incoming requests across multiple instances of a microservice to achieve load balancing. This ensures even distribution of traffic and improved system reliability.

Authentication and Authorization: It provides mechanisms for handling authentication and authorization at a central point, reducing the need to implement these security features in each individual microservice.

Caching: Ocelot can be configured to cache responses from microservices, reducing the load on these services and improving response times for frequently requested data.

Logging and Monitoring: You can configure Ocelot to log incoming requests and responses for debugging and monitoring purposes. This can be helpful for tracking the flow of requests through your microservices architecture.

Transformation: Ocelot can transform request and response data to adapt it to the needs of clients or microservices, including protocol translation (e.g., HTTP to HTTPS) and response content modification.

Rate Limiting: You can set up rate limiting policies to prevent abuse or overuse of your microservices, protecting them from excessive traffic.

Retries and Circuit Breaker: Ocelot can be configured to handle retries for failed requests and implement circuit breakers to prevent the overload of downstream services.

When implementing Ocelot in a microservices architecture, you typically define routing rules and configurations in a JSON or a configuration file. Ocelot sits between the client and your various microservices, managing the request flow.

Here's a basic example of an Ocelot configuration in JSON:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/service1/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "service1",
          "Port": 5000
        }
      ],
      "UpstreamPathTemplate": "/api/service1/{everything}",
      "UpstreamHttpMethod": [ "GET" ]
    },
```

```
    {
      "DownstreamPathTemplate": "/api/service2/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "service2",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/api/service2/{everything}",
      "UpstreamHttpMethod": [ "GET" ]
    }
  ]
}
```

Ocelot is routing requests based on the URL path and HTTP method to different microservices.

By using Ocelot, you can simplify the interaction between clients and your microservices, improve security, and add various features to your microservices architecture without modifying the individual services themselves. It's a valuable tool for managing the complexities of microservices communication.

What is API Gateway

An API Gateway is a server or service that acts as an entry point for a collection of microservices or APIs (Application Programming Interfaces). Its primary function is to manage, route, and control the access to these APIs, providing a centralized point for various clients to interact with different services within a system or application. API Gateways are commonly used in modern software architecture, especially in microservices and serverless environments. Here are some of the key functions and benefits of an API Gateway:

API Routing: API Gateways route incoming requests to the appropriate microservices or APIs based on the request URL, HTTP headers, or other parameters. This simplifies the client's interaction by presenting a single-entry point to the system.

Load Balancing: They can distribute incoming requests across multiple instances of a service to ensure high availability, improved performance, and scalability.

Security: API Gateways can provide security features like authentication, authorization, and encryption. They often have built-in support for authentication mechanisms like API keys, OAuth, and JWT (JSON Web Tokens).

Rate Limiting: You can use an API Gateway to implement rate limiting and throttling to prevent abuse or overuse of APIs, ensuring fair usage and resource allocation.

Caching: Caching can be managed at the API Gateway level to reduce the load on backend services and improve response times for frequently requested data.

Logging and Monitoring: API Gateways can generate logs and metrics for incoming and outgoing traffic, providing valuable insights for debugging, performance optimization, and security analysis.

Transformation: They can modify requests and responses, converting data formats, adding or removing headers, or even aggregating data from multiple services to present a unified response to clients.

Analytics: API Gateways can collect and analyse data about API usage, allowing organizations to gain insights into how their APIs are being used and identify areas for improvement.

Versioning: Managing API versions and backward compatibility is often simplified through an API Gateway.

Error Handling: They can standardize error responses, making it easier for clients to understand and handle errors consistently.

API Documentation: Many API Gateways offer tools for generating and hosting API documentation, making it easier for developers to understand and use the available APIs.

Commonly used API Gateway solutions include AWS API Gateway, Google Cloud Endpoints, Azure API Management, and open-source options like Nginx and Kong. The choice of an API Gateway depends on the specific requirements and the technology stack of the organization or project.

Certainly! Let's delve into more details about the key functions and benefits of an API Gateway:

Let’s Explain One by One

API Routing:

- API Gateways act as traffic directors, directing incoming client requests to the appropriate backend microservices or APIs based on the request's URL, HTTP method, and other request attributes.
- Routing can be based on URL path, domain, HTTP headers, or any other criteria that help determine the destination service.
- This routing capability simplifies client access and ensures that clients do not need to know the specific locations or endpoints of individual services.

Load Balancing:

- API Gateways often incorporate load balancing features, distributing incoming traffic across multiple instances or replicas of a service.
- Load balancing improves system availability, fault tolerance, and ensures that no single service instance is overwhelmed with requests.
- It can also help with scaling services horizontally to handle increased load.

Security:

- API Gateways play a crucial role in ensuring the security of the APIs and the data they expose.
- They can enforce authentication mechanisms, such as API keys, OAuth, JWT, or other identity providers.
- Authorization rules can be defined to control which clients or users have access to specific API endpoints.
- Encryption, such as HTTPS, can be enforced to protect data in transit.

Rate Limiting:

- API Gateways can set rate limits on incoming requests, preventing abuse or overuse of the APIs.
- Rate limiting helps in managing server resources and ensuring fair usage among clients.
- It can be configured based on various criteria like IP address, user, or client type.

Caching:

- Caching at the API Gateway level can store responses to commonly requested data, reducing the load on backend services and improving response times.
- Caching can be configured with rules specifying what to cache, how long to cache data, and when to invalidate or refresh cached data.

Logging and Monitoring:

- API Gateways generate logs and metrics for incoming and outgoing traffic.
- These logs and metrics provide valuable insights into API usage, helping with debugging, performance optimization, and security analysis.
- Monitoring tools can be integrated to provide real-time visibility into the health and performance of APIs.

Transformation:

- API Gateways can modify requests and responses to meet specific requirements.
- Data transformation can involve converting data formats, adding or removing headers, or aggregating data from multiple services to present a unified response to clients.
- This can help decouple the client from the intricacies of the backend services.

Analytics:

- API Gateways collect and analyse data about API usage, client behaviour, and error rates.
- Analytics enable organizations to gain insights into how their APIs are being used and identify areas for improvement and optimization.

Versioning:

- Managing API versions and ensuring backward compatibility is often simplified through the API Gateway.
- It can route requests to the appropriate version of the API based on version identifiers in the request.

Error Handling:

- API Gateways can standardize error responses to provide consistent error codes and messages to clients.
- This simplifies error handling for developers and enhances the clarity of error reporting to clients.

API Documentation:

- Many API Gateways offer tools for generating and hosting API documentation.
- This documentation provides developers with detailed information on how to use the available APIs, making it easier to integrate with the system.

The specific features and capabilities of an API Gateway can vary depending on the chosen solution, and organizations often select an API Gateway that aligns with their specific requirements and technology stack.

What is Ocelot API Gateway

Ocelot is an open-source API Gateway that is designed to work with the .NET ecosystem, making it particularly well-suited for building and managing API gateways in .NET-based microservices architectures. Ocelot is developed and maintained by the community and is available under the MIT license. It provides a range of features to help developers route, secure, and manage API requests.

Here are some key features and aspects of Ocelot:

- **Routing:** Ocelot allows you to define routing rules for incoming requests, forwarding them to the appropriate microservices or backend services based on criteria such as URL paths, HTTP headers, and more. This simplifies the process of routing client requests to the correct destinations.
- **Load Balancing:** It supports various load balancing algorithms to distribute traffic across multiple instances of backend services, ensuring high availability and improved performance.
- **Authentication and Authorization:** Ocelot can be configured to handle authentication and authorization for your APIs. It supports various authentication mechanisms, such as API keys, OAuth, and JWT. You can define authorization policies to control access to specific routes or services.
- **Request/Response Transformation:** Ocelot can transform requests and responses as they pass through the gateway. This can involve modifying headers, aggregating data from multiple services, or converting response formats to match client requirements.
- **Rate Limiting and Throttling:** You can use Ocelot to set rate limits and implement request throttling to prevent abuse or overuse of your APIs. This helps manage resource usage and ensure fair access for clients.
- **Caching:** Ocelot supports response caching to reduce the load on backend services and improve response times for frequently requested data.
- **Logging and Monitoring:** You can integrate Ocelot with logging and monitoring tools to capture and analyse data about incoming and outgoing API traffic.
- **Security:** Ocelot allows you to enforce security measures like HTTPS encryption to protect data in transit, as well as validate client requests.
- **Dynamic Configuration:** Ocelot provides mechanisms for dynamic configuration updates, allowing you to change routing rules and other settings without the need for application redeployment.
- **API Documentation:** Ocelot can generate API documentation to help developers understand the available routes and endpoints within your API gateway.

Ocelot is particularly popular in the .NET ecosystem because it can be easily integrated with ASP.NET Core applications. It simplifies the process of building API gateways that route and manage traffic in microservices architectures. However, it's important to note that Ocelot is primarily focused on .NET technologies and may not be the best choice for organizations using different programming languages or platforms for their microservices.

API Gateway Implementation using Ocelot in Asp.net Core.

Implementing an API Gateway using Ocelot in ASP.NET Core is a popular choice for building microservices architectures. Ocelot is an open-source .NET API Gateway that provides routing, load balancing, authentication, and other essential features for managing and securing your microservices. In this guide, I'll walk you through the steps to create a simple API Gateway using Ocelot in ASP.NET Core.

Prerequisites:

Make sure you have .NET Core SDK and Visual Studio, or another code editor of your choice installed.
You should be familiar with ASP.NET Core and microservices concepts.

Step 1: Create a new ASP.NET Core Web API project.

Create a new ASP.NET Core Web API project that will serve as your API Gateway. You can do this using the command-line or Visual Studio.

- dotnet new webapi -n ApiGateway

Step 2: Install Ocelot

In your API Gateway project, add the Ocelot NuGet package.

- dotnet add package Ocelot.

Step 3: Create an Ocelot Configuration

Create a ocelot.json configuration file to define how requests should be routed to your microservices. Here's a simple example:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/service1/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/service1/{everything}"
    },
    {
      "DownstreamPathTemplate": "/api/service2/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5002
        }
      ],
      "UpstreamPathTemplate": "/service2/{everything}"
    }
  ],
  "GlobalConfiguration": {
    "BaseUrl": "http://localhost:5000"
  }
}
```

This configuration instructs Ocelot to route requests based on the UpstreamPathTemplate to the appropriate microservice specified by DownstreamPathTemplate, DownstreamScheme, and DownstreamHostAndPorts.

Step 4: Configure Ocelot in Startup.cs

In your Startup.cs file, configure Ocelot with the ocelot.json configuration:

```
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.DependencyInjection;
public class Startup
{
    public void ConfigureServices(IServiceCollection services)
    {
        services.AddOcelot();
    }

    public void Configure(IApplicationBuilder app)
    {
        app.UseOcelot().Wait();
    }
}
```

Step 5: Run the API Gateway

Run your API Gateway project. It should start on the specified port (e.g., http://localhost:5000), and it will route incoming requests to the appropriate microservices based on the ocelot.json configuration.

Step 6: Start Your Microservices

Ensure your microservices (e.g., service1 and service2) are running on the specified ports (e.g., http://localhost:5001 and http://localhost:5002). The API Gateway will forward requests to these services. we have created a basic API Gateway using Ocelot in ASP.NET Core. You can now route, secure, and manage requests to your microservices using this gateway. Remember to configure additional features such as load balancing, authentication, and caching as needed for your specific use case.

Practical Implementation of Ocelot Gateway for Our Microservices

Creating a proper ASP.NET Core Web API using Ocelot as an API Gateway for your three microservices (Auth, Student, and Department) with School Products endpoints is a multi-step process. Here's a step-by-step guide on how to implement it:

Step 1: Create a new ASP.NET Core API Gateway Project

Create a new ASP.NET Core Web API project that will act as your API Gateway. You can do this using the following commands:

- dotnet new webapi -n ApiGateway
- cd ApiGateway

Step 2: Install Ocelot NuGet Package

Install the Ocelot NuGet package into your API Gateway project using the NuGet Package Manager or by adding the package reference to your .csproj file:

- dotnet add package Ocelot.

Step 3: Create Ocelot Configuration

In the API Gateway project, create an Ocelot configuration file, for example, ocelot.json, to define routing rules. You can put this file at the root of your project. Here's a sample ocelot.json configuration for your microservices:

```
{
  "ReRoutes": [
    {
      "DownstreamPathTemplate": "/auth/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "auth-service",
          "Port": 5000
        }
      ],
      "UpstreamPathTemplate": "/auth/{everything}"
    },
    {
      "DownstreamPathTemplate": "/student/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "student-service",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/student/{everything}"
    },
    {
      "DownstreamPathTemplate": "/department/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "department-service",
          "Port": 5002
        }
      ],
      "UpstreamPathTemplate": "/department/{everything}"
    },
    {
      "DownstreamPathTemplate": "/products/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "products-service",
          "Port": 5003
```

```
        }
      ],
      "UpstreamPathTemplate": "/products/{everything}"
    }
  ]
}
```

Ensure that the DownstreamHostAndPorts values match your microservices' host and port settings.

Step 4: Configure Ocelot Middleware

In the Startup.cs file of your API Gateway project, configure Ocelot and add it as middleware. You will need to load the Ocelot configuration file and set up the Ocelot pipeline.

```
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
public class Startup
{
    public IConfiguration Configuration { get; }
    public Startup(IConfiguration configuration)
    {
        Configuration = configuration;
    }
    public void ConfigureServices(IServiceCollection services)
    {
        // Configuration, services, and authentication setup if needed
    }
    public void Configure(IApplicationBuilder app, IWebHostEnvironment
env)
    {
        if (env.IsDevelopment())
        {
            app.UseDeveloperExceptionPage();
        }
        app.UseRouting();
        app.UseAuthentication();
        app.UseAuthorization();
        // Add Ocelot middleware with the configuration file
        app.UseOcelot(Configuration);
    }
}
```

Step 5: Secure, Test, and Deploy

Implement security mechanisms such as authentication and authorization as needed. Ensure that your microservices are reachable from the API Gateway. Test the API Gateway to ensure that requests are correctly routed to your microservices.

Once you are satisfied with the setup, deploy the API Gateway and your microservices to your chosen hosting environment.

This is a simplified guide to setting up Ocelot as an API Gateway for your microservices. You may need to implement authentication, authorization, logging, and monitoring based on your specific requirements.

Why we use API-Gateway in Microservices architecture.

API Gateway plays a crucial role in a microservices architecture, serving as a central entry point for external clients and internal services. It offers several advantages, and here are the complete details on why it's an essential component in a microservices ecosystem:

Centralized Entry Point: In a microservices architecture, you have numerous services running independently. An API Gateway acts as a single-entry point for all incoming requests, making it easier to manage and secure the system. Clients interact with the API Gateway, which then routes requests to the appropriate microservice.

Load Balancing: An API Gateway can distribute incoming requests across multiple instances of a microservice, ensuring that the system remains responsive and can handle high traffic loads. This is crucial for maintaining reliability and scalability.

Authentication and Authorization: API Gateways are well-suited for enforcing security and access control. They can handle authentication, authorization, and security protocols, ensuring that only authorized clients can access certain services. This is especially important in microservices, where multiple services with different security requirements are involved.

Rate Limiting and Throttling: API Gateways allow you to set rate limits and throttling policies to prevent abuse or overuse of services. This is important for maintaining fair resource allocation and service quality.

CORS Handling: Cross-Origin Resource Sharing (CORS) issues are common when dealing with multiple clients and services. API Gateways can manage CORS configurations, making it easier to handle requests from various domains.

Request and Response Transformation: API Gateways can transform requests and responses between clients and microservices. This is useful for versioning, data format conversion, or other transformations to ensure compatibility and reduce the burden on individual microservices.

Logging and Monitoring: Centralized logging and monitoring are crucial for maintaining the health and performance of a microservices architecture. API Gateways can log requests, responses, and metrics, making it easier to monitor and troubleshoot issues.

Cache Management: API Gateways can implement caching strategies to improve response times and reduce the load on microservices. Caching can be configured based on the specific needs of different services.

API Composition: In many cases, a single client request may require data from multiple microservices. API Gateways can handle the orchestration and composition of multiple service calls, providing a unified response to the client. This reduces the complexity for clients and optimizes network traffic.

Simplifying Client Code: API Gateways abstract the details of the underlying microservices, which can help simplify client code. Clients don't need to be aware of the individual services, their endpoints, or how they are orchestrated.

Versioning: API Gateways can assist in managing the versioning of APIs, allowing for backward compatibility while introducing new features or changes to services.

Error Handling: API Gateways can provide consistent error responses, making it easier for clients to handle errors gracefully. They can also route traffic away from unhealthy services.

Service Discovery: Many API Gateways integrate with service discovery mechanisms, making it easier to adapt to changes in the microservices landscape. They can route requests to the appropriate service instances even as they scale up or down.

API Gateways play a crucial role in managing and simplifying the complexity of a microservices architecture. They provide centralized control for routing, security, monitoring, and other cross-cutting concerns, allowing individual microservices to focus on their specific business logic. This simplifies development, maintenance, and scaling of microservices-based applications.

Benefits of Ocelot API Gateway

Ocelot is a popular API Gateway for microservices architectures in ASP.NET Core. It offers several benefits for managing and securing your microservices ecosystem:

Routing and Request Aggregation: Ocelot allows you to define routing rules in a configuration file. You can route requests to specific microservices based on various criteria like URL path, HTTP method, headers, and query parameters. It also enables you to aggregate multiple microservices' responses into a single response, reducing the number of requests to clients.

Load Balancing: Ocelot supports load balancing, which helps distribute incoming requests evenly across multiple instances of a microservice. This improves system scalability, availability, and performance.

Service Discovery: Ocelot can work with service discovery mechanisms like Consul, Eureka, or DNS-based discovery, making it easy to manage the dynamic nature of microservices and their locations.

Authentication and Authorization: Ocelot provides features for securing your APIs. You can enforce authentication and authorization policies at the gateway level. This means you can handle authentication and authorization concerns once in the API Gateway, rather than implementing them in each microservice individually.

CORS Support: Cross-Origin Resource Sharing (CORS) can be handled centrally in Ocelot. You can configure CORS policies for your APIs in one place, simplifying the process of enabling or restricting access from different origins.

Request and Response Transformation: Ocelot allows you to modify requests and responses as they pass through the gateway. This feature is valuable for adapting data formats, aggregating data, or making other adjustments to meet the specific needs of clients.

Logging and Monitoring: Ocelot can be configured to log request and response details, which aids in troubleshooting and monitoring the health of your system. You can integrate it with tools like ELK Stack, Prometheus, or other monitoring solutions.

Rate Limiting and Throttling: Ocelot supports rate limiting and request throttling to protect your microservices from abuse and ensure fair resource allocation.

WebSocket's: Ocelot has support for WebSocket communication, enabling real-time interaction between clients and microservices through the gateway.

Easy Maintenance and Configuration: With a centralized API Gateway, you can manage and update configurations, security policies, and routing rules without touching individual microservices. This simplifies maintenance and reduces the risk of errors.

Scaling and High Availability: Ocelot can be easily scaled and configured for high availability. This ensures that the gateway itself doesn't become a single point of failure.

Faster Development: Ocelot can speed up development since you can develop and deploy microservices independently. Changes to routing and security policies are centralized, making it easier to adapt to evolving requirements.

Vendor Agnostic: Ocelot is not tied to any specific cloud provider or platform, making it a flexible choice for both on-premises and cloud-based microservices architectures.

Ocelot simplifies the complexity of managing a microservices ecosystem by offering a unified point of entry for clients, centralizing key functionalities, and providing tools for security, monitoring, and flexibility in routing requests to the appropriate microservices. However, it's essential to configure and use Ocelot thoughtfully to ensure optimal performance and security in your specific use case.

Let’s Discuss one by one in details.

Routing and Request Aggregation:

- Ocelot allows you to define routing rules in a configuration file (usually ocelot.json). This file specifies how incoming requests should be routed to the corresponding microservices.
- You can use URL paths, HTTP methods, headers, query parameters, or other request attributes to determine the routing logic.
- Ocelot also supports request aggregation, where you can merge responses from multiple microservices into a single response. This reduces the number of requests made by clients and minimizes the complexity of client-side logic.

Load Balancing:

- Ocelot provides load balancing capabilities, enabling you to distribute incoming requests across multiple instances of a microservice.
- Load balancing helps in improving the performance, scalability, and availability of your microservices by ensuring that requests are evenly distributed among healthy instances.
- Ocelot supports different load balancing algorithms, such as round-robin, weighted round-robin, and least connections.

Service Discovery:

- Ocelot can integrate with various service discovery mechanisms, such as Consul, Eureka, or DNS-based service discovery.
- Service discovery allows Ocelot to dynamically locate the appropriate instances of microservices, even in a highly dynamic and containerized environment.
- This helps in maintaining the accuracy of routing configurations as microservices scale up or down.

Authentication and Authorization:

- Ocelot enables centralized management of authentication and authorization concerns for your microservices.
- You can enforce authentication and authorization policies at the gateway level, eliminating the need to implement these security measures separately in each microservice.
- Common authentication methods like API keys, JWT, or OAuth can be managed at the gateway.

CORS Support:

- Cross-Origin Resource Sharing (CORS) is essential for enabling web applications running in one domain to request resources from a different domain (origin).

- Ocelot simplifies CORS configuration by allowing you to define and manage CORS policies centrally.
- You can specify which origins are allowed to access your APIs and which HTTP methods and headers are permitted.

Request and Response Transformation:

- Ocelot provides the capability to modify requests and responses as they pass through the gateway.
- This feature is valuable for adapting data formats, aggregating data from different sources, or making other adjustments to align with the specific needs of clients.
- It's particularly useful when you need to transform data from microservices to match a common API schema or for versioning.

Logging and Monitoring:

- Ocelot can log request and response details, which is crucial for monitoring and troubleshooting.
- By integrating Ocelot with tools like ELK Stack, Prometheus, or other monitoring solutions, you can gain insights into the health and performance of your API ecosystem.
- Logging can help in identifying issues, measuring latency, and tracking the usage of APIs.

Rate Limiting and Throttling:

- Ocelot can enforce rate limiting and request throttling, which is important for protecting your microservices from abuse, ensuring fair resource allocation, and preventing overloading of services.
- You can define policies that limit the number of requests a client or an IP address can make within a specific time period.

WebSocket’s:

- Ocelot has support for WebSocket communication, enabling real-time, bidirectional communication between clients and microservices.
- WebSocket’s are beneficial for building features like live chats, notifications, or online gaming within a microservices architecture.

Easy Maintenance and Configuration:

- Ocelot centralizes configuration and management. This simplifies maintenance, as changes to routing and security policies can be handled in one place (the API Gateway).
- It reduces the risk of errors that might occur when trying to synchronize changes across multiple microservices.

Scaling and High Availability:

- Ocelot itself can be scaled horizontally to handle increased traffic and ensure high availability.
- By deploying Ocelot instances behind a load balancer, you can ensure that the gateway doesn't become a single point of failure.

Vendor Agnostic:

- Ocelot is not tied to a specific cloud provider or platform, making it a flexible choice for both on-premises and cloud-based microservices architectures.
- It can be used in a variety of environments, including on-premises servers, containers, or cloud services like AWS, Azure, or Google Cloud.

Ocelot API Gateway simplifies the complexities of managing a microservices ecosystem by providing a unified point of entry for clients and centralizing key functionalities such as routing, security, monitoring, and request/response transformation. These features contribute to the robustness, security, and scalability of your microservices architecture. However, it's essential to configure and use Ocelot thoughtfully to ensure optimal performance and security in your specific use case.

Dis-Advantages of Ocelot API-Gateway

While Ocelot API Gateway offers many benefits, it's important to be aware of its disadvantages and potential challenges when implementing it in your microservices architecture. Some of the disadvantages of Ocelot include:

Learning Curve: Ocelot, like any technology, has a learning curve. You'll need to understand its configuration, routing, and various features to effectively set up and manage the gateway. This may require additional time and effort from your development and operations teams.

Single Point of Failure: If not configured for high availability, the API Gateway itself can become a single point of failure. If it goes down, it can disrupt all incoming and outgoing traffic to your microservices. It's crucial to plan for redundancy and failover mechanisms.

Performance Overhead: An API Gateway like Ocelot introduces additional processing and network communication for each request. While this overhead is typically small, it can become significant in high-traffic scenarios. Careful optimization and proper hardware sizing are necessary to mitigate this.

Increased Latency: The additional processing and routing performed by the API Gateway can introduce latency into requests. Depending on your use case, this added latency may or may not be acceptable. Microservices architectures often prioritize low latency.

Complex Configuration: Configuring Ocelot for complex routing scenarios, especially when handling many microservices with diverse routing requirements, can be challenging. Managing a complex ocelot. Json configuration file may lead to errors and maintenance challenges.

Limited Language Support: Ocelot is primarily designed for .NET Core, limiting its usage in environments that rely on different programming languages or frameworks. If your microservices are built using a variety of technologies, this may not be the best fit.

Lack of Built-in Rate Limiting: While Ocelot does support rate limiting, it doesn't provide advanced rate limiting features out-of-the-box. More complex rate limiting scenarios might require additional third-party tools or custom development.

Increased Network Traffic: API Gateways, including Ocelot, can increase network traffic within your infrastructure. For example, when requests need to be aggregated or transformed, additional internal service-to-service communication may occur, impacting network utilization.

Potential for Overhead: When routing requests to microservices, Ocelot can sometimes introduce unnecessary overhead, especially when your microservices are exposed over fast, low-latency connections. You should carefully evaluate whether an API Gateway is necessary for your specific use case.

Maintenance and Upkeep: Managing and maintaining an API Gateway adds an additional component to your system. You need to keep it up to date, apply security patches, and ensure it aligns with the evolving needs of your microservices.

Customization Challenges: Ocelot is a feature-rich API Gateway, but extensive customization or implementing unique features may require significant effort. If your use case demands highly specialized functionality, you may need to extend Ocelot or consider alternative solutions.

It's essential to weigh the advantages and disadvantages of using Ocelot or any API Gateway in your microservices architecture carefully. Your choice should align with your specific requirements, the complexity of your system, and the resources available for implementation and maintenance. In some cases, alternative solutions or a custom-built gateway might be more suitable.

Learning Curve:

- Ocelot, like any technology, has a learning curve. Developers and operations teams need to invest time in understanding its configuration, routing, and features.
- The learning curve might be steeper if your team is not already familiar with Ocelot or API Gateway concepts.

Single Point of Failure:

- By default, Ocelot operates as a single instance. If it goes down, it can disrupt all incoming and outgoing traffic to your microservices, making it a single point of failure.
- High availability configurations (e.g., deploying multiple instances behind a load balancer) are necessary to mitigate this risk.

Performance Overhead:

- Ocelot adds a level of processing for each incoming request, including routing and potential transformations. This processing overhead is generally small but can accumulate in high-traffic scenarios.
- To mitigate performance overhead, careful optimization of your Ocelot configuration and adequate hardware resources are needed.

Increased Latency:

- Due to the additional processing and routing, Ocelot can introduce latency into requests. While the latency is typically minimal, it can be a concern in microservices architectures where low latency is a priority.
- Consider performance benchmarks to understand the latency implications in your specific use case.

Complex Configuration:

- Configuring Ocelot for complex routing scenarios, especially when dealing with a large number of microservices, can be challenging. Managing a complex ocelot.json configuration file can be error-prone.
- Documenting configurations and following best practices in naming conventions and organization can help mitigate this issue.

Limited Language Support:

- Ocelot is primarily designed for .NET Core environments. If your microservices are built using a variety of programming languages or frameworks, Ocelot might not be the best fit.
- In a polyglot microservices environment, you may need to consider a more language-agnostic API Gateway or handle routing and aggregation differently.

Lack of Built-in Rate Limiting:

- While Ocelot does support basic rate limiting, it may not provide advanced rate limiting features out-of-the-box.
- More complex rate limiting scenarios, such as per-user rate limiting or dynamic rate limits based on service health, may require additional third-party tools or custom development.

Increased Network Traffic:

- An API Gateway like Ocelot can increase network traffic within your infrastructure. For example, when requests need to be aggregated or transformed, additional internal service-to-service communication may occur.
- This increased network utilization can impact the overall performance and costs of your system.

Potential for Overhead:

- When routing requests through an API Gateway, such as Ocelot, there's a potential for introducing overhead, especially if your microservices communicate over fast, low-latency connections.
- Carefully assess whether the benefits of an API Gateway outweigh the potential overhead for your specific use case.

Maintenance and Upkeep:

- Implementing an API Gateway adds an additional component to your system, which requires ongoing maintenance and upkeep.
- Regular updates, security patches, and alignment with the evolving needs of your microservices and organization are necessary.

Customization Challenges:

- While Ocelot is a feature-rich API Gateway, extensive customization or implementing unique features may require significant effort.
- If your use case demands highly specialized functionality, you may need to extend Ocelot or consider alternative solutions, potentially involving custom development.

Ocelot API Gateway provides numerous advantages for managing and securing microservices, it's essential to understand its potential disadvantages and challenges. Careful planning, thorough performance testing, and consideration of the specific needs and constraints of your microservices architecture will help you make an informed decision about whether Ocelot is the right choice for your project. If the disadvantages are significant for your use case, you might explore alternative API Gateway solutions or custom-built gateways tailored to your specific requirements.

Advantages of Ocelot:

Routing and Request Aggregation: Ocelot simplifies routing and allows for request aggregation, reducing the complexity of client-side logic and the number of requests.

Load Balancing: Ocelot supports load balancing, improving system scalability and availability.

Service Discovery: It can integrate with service discovery mechanisms, adapting to the dynamic nature of microservices.

Authentication and Authorization: Ocelot provides centralized management of authentication and authorization, simplifying security measures.

CORS Support: CORS policies can be centrally configured, allowing or restricting access from different origins.

Request and Response Transformation: Data can be transformed to meet the specific needs of clients, simplifying data format adaptation and aggregation.

Logging and Monitoring: Ocelot can log request and response details, facilitating monitoring and troubleshooting.

Rate Limiting and Throttling: Protection against abuse and resource allocation is available through rate limiting and request throttling.

Web Sockets: Real-time communication through WebSocket support opens opportunities for live chats, notifications, and more.

Easy Maintenance and Configuration: Centralized management simplifies configuration updates and reduces the risk of errors.

Scaling and High Availability: Ocelot can be scaled and configured for high availability to prevent it from becoming a single point of failure.

Vendor Agnostic: It can be used in various environments, making it flexible for different deployment scenarios.

Disadvantages and Challenges:

Learning Curve: Ocelot has a learning curve, requiring time for developers to become familiar with its configuration and features.

Single Point of Failure: Without high availability configurations, Ocelot can be a single point of failure, impacting the reliability of your system.

Performance Overhead: The gateway introduces processing overhead, which can affect performance, especially in high-traffic scenarios.

Increased Latency: The additional processing and routing can lead to added latency, which might not be suitable for latency-sensitive applications.

Complex Configuration: Complex routing configurations can be challenging to manage, leading to potential errors and maintenance difficulties.

Limited Language Support: Ocelot primarily supports .NET Core, which may not be ideal in polyglot microservices environments.

Lack of Built-in Rate Limiting: Advanced rate limiting may require additional third-party tools or custom development.

Increased Network Traffic: The gateway can increase internal network traffic, affecting performance and costs.

Potential for Overhead: There's a potential for introducing overhead, which may need careful assessment for your use case.

Maintenance and Upkeep: Ongoing maintenance and updates are necessary to keep the gateway in sync with evolving requirements.

Customization Challenges: Highly specialized functionality might require extensive customization or custom development.

In deciding, consider the specific needs of your microservices architecture and weigh the advantages against the disadvantages. If the disadvantages are significant in your case, explore alternative API Gateway solutions or custom-built gateways that align better with your requirements. Proper planning and thorough testing will help you make an informed choice that suits your project's goals and constraints.

Conclusion

In conclusion, Ocelot API Gateway is a powerful tool for managing and securing microservices in an ASP.NET Core environment. It offers a range of benefits, such as routing, load balancing, authentication, and more, which simplify the complexities of microservices architecture. However, it's crucial to weigh these advantages against the disadvantages and potential challenges when considering whether Ocelot is the right choice for your project. Here's a detailed summary:

Api Gateway Introduction & Development Using Yarp

In this Part we will develop the Api Gateway for Communication between Different Microservices

API Gateway Development is a crucial aspect of modern software architecture, providing a centralized entry point for accessing and managing multiple APIs. The API Gateway acts as a proxy between clients and backend services, offering a unified interface and facilitating seamless communication between various systems. During development, API Gateway handles essential tasks such as routing requests, load balancing, authentication, authorization, and caching. It abstracts the complexities of underlying microservices or APIs, simplifying the client-side integration process and promoting loose coupling between different components. API Gateway Development also enables efficient traffic management, allowing developers to monitor and control API usage, set rate limits, and implement throttling mechanisms. Additionally, it offers powerful analytics and logging capabilities, providing insights into API usage patterns and aiding in performance optimization. With its comprehensive feature set, API Gateway Development streamlines the development and management of APIs, enhances security, improves performance, and ensures a consistent and reliable experience for API consumers.

9

Building a Microservices API Gateway With YARP

Overview

In this chapter, we explore the Microservices API Gateway, focusing on its core concepts and applications. We explore the Key Features and Benefits of YARP API Gateway and guide you through API Gateway Development for Our Microservices Architecture. Prepare to understand Why We Use YARP API Gateway and uncover its Advantages and Disadvantages. This comprehensive journey will equip you with the knowledge to effectively utilize YARP in your microservices architecture.

Introduction:

YARP typically stands for Yet Another Reverse Proxy in the context of microservices and web application architecture. It is a reverse proxy server designed to work seamlessly with ASP.NET Core. YARP is an open-source project developed by Microsoft that provides powerful capabilities for routing and load balancing requests in a microservices architecture.
Here's a complete introduction to YARP in the context of microservices:

What is YARP?

YARP is a reverse proxy server that simplifies the process of routing and load balancing HTTP requests in a microservices-based architecture. It allows developers to create complex routing rules, manage traffic, and apply custom middleware to incoming and outgoing requests.

Key Features:

YARP offers several key features:

Request Routing: It can route incoming HTTP requests to different microservices based on various criteria like path, host, query parameters, etc.

Load Balancing: YARP supports load balancing methods like round-robin, least-connections, and more, ensuring even distribution of requests across microservices.

Middleware: Developers can apply custom middleware to requests and responses, enabling features like authentication, authorization, logging, and request modification.

Dynamic Configuration: YARP allows you to dynamically update routing rules and configuration without restarting the proxy, making it highly adaptable to changes in your microservices ecosystem.

Use Cases:

YARP is well-suited for various use cases, including:

Microservices Architecture: It is particularly valuable in a microservices environment where you have multiple services that need to be exposed through a single-entry point.

API Gateway: YARP can be used as an API gateway to manage, route, and secure API requests from clients.

Load Balancing: YARP's load balancing features ensure that requests are evenly distributed across backend services, improving system reliability and performance.

Configuration:

YARP can be configured using a JSON file, which defines the routing rules, load balancing options, and middleware pipeline. This configuration can be dynamically updated without requiring a server restart.

Installation:

To use YARP, you need to add it to your ASP.NET Core application as a NuGet package and configure it within your application.

Example: Here's a simple example of YARP configuration in JSON format:

```
{
  "ReverseProxy": {
    "Routes": [
      {
```

```
        "RouteId": "myRoute",
        "ClusterId": "myCluster",
        "Match": {
          "Path": "/api"
        }
      }
    ],
    "Clusters": {
      "myCluster": {
        "Destinations": {
          "app1": { "Address": "http://app1-service:5000" },
          "app2": { "Address": "http://app2-service:5000" }
        }
      }
    }
  }
}
```

In this example, incoming requests to "/api" are routed to either "app1-service" or "app2-service" based on load balancing rules.

Benefits:

YARP simplifies the management of routing and load balancing in microservices, making it easier to scale, manage, and secure your application.
Remember that the YARP project may have evolved or received updates since my last knowledge update in September 2021, so it's a good idea to check the official Microsoft documentation or YARP repository for the latest information and updates.

What is YARP Api Gateway

YARP, which stands for "Yet Another Reverse Proxy," is an open-source project developed by Microsoft. It is a reverse proxy and load balancer designed for use in ASP.NET Core applications. YARP allows developers to build custom proxy and load-balancing solutions for their applications, making it particularly useful for creating API gateways and routing requests to backend services in a microservices architecture.

Key features and benefits of YARP include:

Reverse Proxy: YARP acts as a reverse proxy, accepting incoming HTTP requests and forwarding them to the appropriate backend services. This is essential for routing requests to various microservices within an application.

Custom Routing: YARP enables you to define custom routing rules, which can include routing based on the request path, host, or other criteria. This flexibility allows for fine-grained control over how requests are handled.

Load Balancing: YARP includes load-balancing capabilities, allowing you to distribute incoming requests across multiple backend services to ensure scalability and reliability.

Middleware for ASP.NET Core: YARP can be seamlessly integrated into ASP.NET Core applications as middleware, making it a natural choice for .NET developers.

Configuration: YARP can be configured via JSON or other configuration sources, making it easy to set up and manage routing and proxying rules.

Customization: YARP is designed to be highly customizable. Developers can implement custom behaviours for request and response processing, making it suitable for a wide range of use cases.

Performance: YARP is optimized for high-performance, ensuring that it can handle a significant volume of incoming requests without significant latency.

YARP is particularly well-suited for applications that require complex routing, load balancing, and API gateway functionality, which are common in microservices architectures. It allows developers to create custom proxy and routing solutions tailored to the specific needs of their applications.

Please note that the details and features of YARP may have evolved or changed since my last knowledge update in September 2021. If you plan to use YARP, I recommend checking the official documentation and resources provided by Microsoft or the YARP project for the most up-to-date information.

Let’s Explain Each point in Details.

Reverse Proxy:

A reverse proxy is a network device or server-side software that acts as an intermediary for client requests seeking resources from servers. It forwards client requests to the appropriate backend servers and returns the server's response to the client. YARP serves as a reverse proxy for HTTP requests in ASP.NET Core applications.

Custom Routing:

YARP allows you to create custom routing rules for incoming HTTP requests. This means you can specify how YARP should determine which backend service should handle a request based on various factors such as the request path, host, or other request attributes.
Custom routing is vital in microservices architectures, where different services may handle specific parts of an application.

Load Balancing:

Load balancing is a technique used to distribute incoming requests evenly across multiple backend servers or instances. This is crucial for achieving scalability, high availability, and optimal resource utilization.
YARP includes load-balancing capabilities, enabling you to evenly distribute requests among backend services to prevent overloading any single service.

Middleware for ASP.NET Core:

In ASP.NET Core, middleware components are used to process HTTP requests and responses as they flow through the application's request pipeline. YARP is implemented as a middleware component, which means it can be seamlessly integrated into your ASP.NET Core application. Placing YARP in the middleware pipeline allows you to intercept and modify requests and responses as they travel through your application.

Configuration:

YARP can be configured using JSON or other configuration sources to define how it should behave as a reverse proxy. This configuration includes setting up clusters (backend services) and routes (request routing rules).

Here's a simplified example of a YARP configuration in JSON:

```
{
    "ReverseProxy": {
        "Clusters": {
            "mycluster": {
                "Destinations": {
                    "destination1": {
                        "Address": "https://service1.example.com"
                    },
                    "destination2": {
                        "Address": "https://service2.example.com"
                    }
                }
            },
            "Routes": [
                {
                    "RouteId": "route1",
                    "ClusterId": "mycluster",
                    "Match": {
                        "Path": "/service1"
                    }
                },
                {
                    "RouteId": "route2",
                    "ClusterId": "mycluster",
                    "Match": {
                        "Path": "/service2"
                    }
                }
            ]
        }
    }
}
```

Customization:

YARP is designed to be highly extensible and customizable. You can implement custom behaviours by developing your own YARP middleware components.
This level of customization allows you to adapt YARP to the specific needs and requirements of your application.

Performance:

YARP is optimized for high performance. It's engineered to handle a large volume of incoming HTTP requests with minimal overhead or latency.
High performance is essential for ensuring that the reverse proxy does not slow down your application, even under heavy load.
YARP is a powerful tool for managing the routing and load balancing of incoming HTTP requests, making it an excellent choice for building API gateways, and managing complex microservices architectures. It provides flexibility, control, and performance, and it can be integrated seamlessly into your ASP.NET Core application. Remember to refer to the official YARP documentation and project resources for the latest information and guidance on using YARP in your specific application scenario.

API Gateway Development Using Asp.net Core Web API with YARP

Creating a detailed example of building an API Gateway using YARP in ASP.NET Core involves several steps.

Create a New ASP.NET Core Web API Project:

Start by creating a new ASP.NET Core Web API project using your preferred development environment. You can use Visual Studio, Visual Studio Code, or the .NET CLI.

- dotnet new webapi -n ApiGatewayExample
- cd ApiGatewayExample

Install YARP:

Install the YARP package in your project using the .NET CLI.

- dotnet add package Microsoft.ReverseProxy

Configure YARP in Startup.cs:

Open the Startup.cs file and configure YARP as middleware. You need to add routing rules to define how YARP should handle incoming requests. This is an example of Startup.cs:

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.ReverseProxy.Service;
// ...
public void ConfigureServices(IServiceCollection services)
{
    // Configure YARP with a custom configuration provider
    services.AddReverseProxy()
            .LoadFromConfig(Configuration.GetSection("ReverseProxy"));
}

public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    app.UseRouting();
    // Add the YARP middleware
    app.UseEndpoints(endpoints => { endpoints.MapReverseProxy(); });
}
Define YARP Configuration:
In your appsettings.json file or another configuration source, define
YARP's routing and proxying rules. Here's a sample configuration:
{
    "ReverseProxy": {
        "Clusters": {
            "mycluster": {
                "Destinations": {
                    "service1": {
                        "Address": "https://api.service1.com"
                    },
```

```
                    "service2": {
                        "Address": "https://api.service2.com"
                    }
                }
            }
        },
        "Routes": [
            {
                "RouteId": "route1",
                "ClusterId": "mycluster",
                "Match": {
                    "Path": "/service1/{**catch-all}"
                }
            },
            {
                "RouteId": "route2",
                "ClusterId": "mycluster",
                "Match": {
                    "Path": "/service2/{**catch-all}"
                }
            }
        ]
    }
}
```

In this example, two services (service1 and service2) are defined as destinations, and routing rules match requests based on the path. You can add more routes and destinations as needed.

Controller for the API Gateway:

Create a controller in your ASP.NET Core project to handle requests that come to the API Gateway. This controller can have actions for any common API gateway functionalities, like authentication, logging, or request transformation.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.ReverseProxy.Abstractions.ClusterDiscovery.Contract;
using Microsoft.ReverseProxy.Service.Proxy;

[Route("api")]
[ApiController]
public class GatewayController : ControllerBase
{
    private readonly IReverseProxyService _reverseProxyService;
    public GatewayController(IReverseProxyService reverseProxyService)
    {
        _reverseProxyService = reverseProxyService;
    }
    [HttpGet("{service}/{**path}")]
    public async Task<IActionResult> RouteRequest(string service,
string path)
    {
```

```
        // Here, you can implement your API gateway logic.
        // You might want to do authentication, logging, or request
transformation.
        // For demonstration purposes, we'll use YARP to reverse proxy
the request
        var routeConfig = new RouteConfig { RouteId = service,
ClusterId = "mycluster" };
        var destination = new DestinationConfig { Address =
$"https://{service}.example.com" };

        var cluster = new ClusterConfig { Destinations = new
Dictionary<string, DestinationConfig> { { service, destination } } };
        var clusterManager = new ClusterManager(new[] { cluster });
        // Create a request context
        var httpContext = HttpContext;
        var routeConfigSelector = new ConfigSelector(new[] {
routeConfig }, clusterManager, destinationManager: null);
        var route = routeConfigSelector.SelectConfig(httpContext);
        if (route != null)
        {
            await _reverseProxyService.ProxyAsync(httpContext, route);
            return new EmptyResult(); // The response is handled by
YARP.
        }

        return NotFound("Route not found");
    }
}
```

Testing the API Gateway:

Start your ASP.NET Core Web API project. Now, when you send requests to the API Gateway, YARP will route them to the appropriate backend services based on the configured rules.
For example, a request to http://your-api-gateway/api/service1/resource will be routed to the service1 backend, and a request to http://your-api-gateway/api/service2/resource will be routed to the service2 backend.
This is a simplified example of building an API Gateway using YARP in ASP.NET Core. In a real-world scenario, you may want to implement additional features like authentication, caching, logging, and more in your API Gateway logic. Make sure to refer to YARP's documentation for more advanced configuration options and customization possibilities to suit your specific use case.

API Gateway Development For our Microservices using YARP.

To implement an API Gateway using YARP (Yet Another Reverse Proxy), you can follow these steps to create a gateway that routes requests to your three microservices: Auth, Student, and Department, and manages the School Products endpoints. YARP is a .NET project that provides reverse proxy capabilities for routing and load balancing requests.
Here's a step-by-step guide on how to set up the YARP API Gateway:

Create a New ASP.NET Core Project:

Start by creating a new ASP.NET Core project that will serve as your API Gateway. You can do this using Visual Studio or the dotnet new command.

- dotnet new web -n ApiGateway

Install YARP:

You need to add the YARP package to your project. You can do this using the NuGet package manager:

- dotnet add package Microsoft.ReverseProxy

Configure YARP in Startup.cs:

In your Startup.cs file, configure YARP in the ConfigureServices and Configure methods:

```
using Microsoft.ReverseProxy.Service
public void ConfigureServices(IServiceCollection services)
{
    services.AddReverseProxy()
        .LoadFromConfig(Configuration.GetSection("ReverseProxy"));
}
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    app.UseRouting();
    app.UseEndpoints(endpoints =>
    {
        endpoints.MapReverseProxy();
    });
}
```

Configure Routing Rules:

Create a reverse-proxy.json configuration file to define routing rules. In this file, you specify the target endpoints for each route. Place this file in the root of your project.

```
{
    "ReverseProxy": {
        "Routes": [
            {
                "RouteId": "Auth",
                "ClusterId": "authCluster",
                "Match": {
                    "Path": "/auth/{**catch-all}"
                }
            },
            {
                "RouteId": "Student",
                "ClusterId": "studentCluster",
                "Match": {
                    "Path": "/student/{**catch-all}"
                }
```

```
            },
            {
                "RouteId": "Department",
                "ClusterId": "departmentCluster",
                "Match": {
                    "Path": "/department/{**catch-all}"
                }
            },
            {
                "RouteId": "Products",
                "ClusterId": "productsCluster",
                "Match": {
                    "Path": "/products/{**catch-all}"
                }
            }
        ],
        "Clusters": {
            "authCluster": [
                {
                    "Destinations": {
                        "authService": {}
                    }
                }
            ],
            "studentCluster": [
                {
                    "Destinations": {
                        "studentService": {}
                    }
                }
            ],
            "departmentCluster": [
                {
                    "Destinations": {
                        "departmentService": {}
                    }
                }
            ],
            "productsCluster": [
                {
                    "Destinations": {
                        "productsService": {}
                    }
                }
            ]
        }
    }
}
```

Be sure to replace authService, studentService, departmentService, and productsService with the actual URLs of your microservices.

Configure appsettings.json:

In your appsettings.json file, add the configurations for your microservices' endpoints. For example:

```
{
    "AuthEndpoint": "http://auth-service-url",
    "StudentEndpoint": "http://student-service-url",
    "DepartmentEndpoint": "http://department-service-url",
    "ProductsEndpoint": "http://products-service-url"
}
```

Call Microservices from Gateway:

In your gateway's controllers or middleware, you can make HTTP requests to the appropriate microservices using the configured endpoints. Use the IHttpClientFactory to create and manage HTTP client instances.

Run Your API Gateway:

Run your API Gateway project, and it will route requests to the appropriate microservices based on the defined rules.
This setup allows you to create a flexible and scalable API Gateway using YARP that can route requests to your Auth, Student, Department, and Products microservices. Make sure you adjust the configurations and endpoints according to your actual microservice setup.

Why we use YARP API Gateway for microservices Architecture.

YARP (Yet Another Reverse Proxy) is a reverse proxy and load balancer developed by Microsoft. It is designed to be used as an API Gateway in microservices architectures and has several advantages that make it a good choice for this purpose:

Lightweight and High Performance: YARP is built on top of Kestrel, which is the web server used in ASP.NET Core. This means it's designed to be lightweight and highly performant, making it a good fit for microservices environments where low latency and high throughput are critical.

Native Support for .NET: Since YARP is part of the .NET ecosystem, it seamlessly integrates with other .NET technologies. This can simplify development, deployment, and monitoring, especially if your microservices are also built with .NET.

Configuration-Driven Routing: YARP allows you to configure routing rules through a configuration file (usually JSON). This makes it easy to set up and manage routes to your microservices without having to write a lot of code. It's especially useful in scenarios where you need to manage a large number of microservices.

Dynamic Routing: YARP supports dynamic routing, which means you can change routing rules at runtime without having to restart the gateway. This flexibility is valuable in dynamic microservices environments.

Load Balancing: YARP can be configured for load balancing across multiple instances of a microservice. This ensures that traffic is distributed evenly and can handle failover scenarios.

Security Features: YARP supports authentication and authorization, which is crucial in securing access to your microservices. You can configure it to perform authentication checks before forwarding requests to the microservices.

Centralized Logging and Monitoring: YARP can be configured to log requests and responses, which is essential for monitoring and troubleshooting. You can integrate it with centralized logging and monitoring solutions.

Scalability: YARP can be scaled horizontally, just like your microservices. This means you can add more instances of the gateway to handle increased traffic as your system grows.

Customization: YARP allows you to extend and customize its behavior by writing custom middleware. This flexibility is helpful when you have specific requirements that are not covered by default features.

Open Source: YARP is open source, which means you have full access to the source code, can contribute to the project, and it has an active community and support.

While YARP has many advantages, it's important to note that the choice of an API Gateway, including YARP, should be made after carefully considering the specific requirements and constraints of your microservices architecture. Other API Gateway solutions, like Nginx, HAP Roxy, or cloud-native options, may also be suitable depending on your needs and constraints.

Let’s Explain each point one by one

Lightweight and High Performance:

YARP is designed to be highly performant and is built on top of Kestrel, which is a lightweight and efficient web server used in ASP.NET Core. This means it's well-suited for scenarios where low latency and high throughput are crucial, such as microservices communication.

Native Support for .NET:

YARP is part of the .NET ecosystem, which makes it a natural fit for organizations already using .NET for their microservices. It integrates well with other .NET technologies, which can simplify development, deployment, and monitoring.

Configuration-Driven Routing:

YARP allows you to configure routing rules through a configuration file. This configuration-driven approach makes it easy to set up and manage routes to your microservices. You can define routing rules, transformations, and other behaviors without writing a lot of custom code.

Dynamic Routing:

YARP supports dynamic routing, which means you can change routing rules at runtime without restarting the gateway. This is particularly valuable in dynamic microservices environments where services may come and go, or where you need to adapt to changing traffic patterns.

Load Balancing:

YARP can be configured for load balancing across multiple instances of a microservice. It can distribute incoming requests evenly, helping to ensure that no single instance becomes overloaded. This is essential for scalability and high availability.

Security Features:

YARP supports authentication and authorization. You can configure it to perform authentication checks before forwarding requests to the microservices. This adds an important layer of security to your microservices architecture by ensuring that only authorized requests are allowed through.

Centralized Logging and Monitoring:

YARP can be configured to log requests and responses, which is vital for monitoring, troubleshooting, and auditing. You can integrate it with centralized logging and monitoring solutions like Application Insights or ELK Stack to gain insights into how requests flow through your gateway.

Scalability:

YARP itself can be scaled horizontally, similar to your microservices. As your system grows and the number of incoming requests increases, you can add more instances of the YARP gateway to handle the higher traffic. This horizontal scalability ensures that the gateway doesn't become a bottleneck.

Customization:

YARP allows you to extend and customize its behavior by writing custom middleware. This flexibility is valuable when you have specific requirements that are not covered by default features. You can add custom logic for request and response processing, allowing you to adapt the gateway to your unique needs.

Open Source:

YARP is an open-source project, which means you have access to the source code. This can be helpful if you need to make modifications or want to understand how it works at a deep level. Additionally, it has an active community, which can provide support and contribute to the development of the project. When considering YARP as an API Gateway for your microservices, it's essential to evaluate how each of these advantages aligns with your specific architectural and operational requirements. Additionally, keep in mind that YARP is primarily designed for use in .NET-based environments, so its suitability depends on your technology stack.

Advantages of YARP API-Gateway

YARP (Yet Another Reverse Proxy) offers several advantages when used as an API Gateway or reverse proxy in microservices architectures:

Lightweight and High Performance:

YARP is built on top of Kestrel, the same web server used in ASP.NET Core. This architecture makes it lightweight and highly performant, suitable for handling a large number of requests with low latency.

Native Integration with .NET:

YARP seamlessly integrates with the .NET ecosystem. If your microservices are built with .NET, using YARP can streamline development, deployment, and monitoring processes. It's also compatible with ASP.NET Core and other .NET technologies.

Configuration-Driven Routing:

YARP allows you to define routing rules, transformations, and other behaviours through a configuration file. This configuration-driven approach simplifies the setup and management of routes to your microservices, eliminating the need for extensive custom code.

Dynamic Routing:

YARP supports dynamic routing, enabling you to change routing rules at runtime without needing to restart the gateway. This flexibility is valuable in dynamic microservices environments where services may be added or removed dynamically.

Load Balancing:

YARP can be configured to perform load balancing across multiple instances of a microservice, ensuring even distribution of incoming requests. This load balancing is essential for achieving scalability and fault tolerance.

Security Features:

YARP provides built-in support for authentication and authorization. You can configure it to enforce security policies, ensuring that only authorized requests are forwarded to your microservices. This is crucial for securing your API endpoints.

Centralized Logging and Monitoring:

YARP can be configured to log requests and responses, facilitating monitoring, troubleshooting, and auditing. You can integrate it with popular logging and monitoring solutions, such as Application Insights or ELK Stack, to gain insights into your API traffic.

Scalability:

YARP itself can be horizontally scaled, just like your microservices. As your system grows and experiences increased traffic, you can add more YARP instances to handle the higher load, preventing it from becoming a bottleneck.

Customization:

YARP offers the flexibility to extend and customize its behaviour by writing custom middleware. This allows you to implement specific business logic, request/response transformations, or custom behaviours tailored to your unique requirements.

Open Source and Community Support:

YARP is an open-source project, making the source code available for inspection and modification. Additionally, it has an active community of users and contributors, which means you can find support and benefit from ongoing improvements and enhancements.
YARP's combination of performance, integration with the .NET ecosystem, configuration-driven routing, dynamic routing capabilities, security features, scalability, and customization options make it a compelling choice for implementing an API Gateway in microservices architectures, especially in .NET-centric environments.

Dis-Advantages of YARP API-Gateway

While YARP (Yet Another Reverse Proxy) offers several advantages as an API Gateway, it's important to be aware of potential disadvantages and limitations:

.NET Ecosystem Dependency:

YARP is tightly integrated with the .NET ecosystem. While this is an advantage for .NET-based applications, it can be a disadvantage if your microservices architecture includes non-.NET technologies. YARP may not be the best choice if you're dealing with a polyglot environment.

Complex Configuration:

Configuring YARP can be complex, especially when dealing with a large number of microservices or complex routing scenarios. The configuration files may become lengthy and difficult to manage.

Limited Language Support:

YARP is primarily designed for use with .NET Core/ASP.NET Core applications. If you have microservices built with other programming languages, you may not benefit as much from the tight integration it offers.

Learning Curve:

YARP might have a learning curve, especially for those who are not familiar with the .NET ecosystem. This could slow down the setup and configuration process.

Development Effort:

Customizing and extending YARP to meet specific requirements may require development effort. Writing custom middleware, for example, might be necessary in certain scenarios.

Maintenance and Updates:

As an open-source project, YARP may require more maintenance and updates. You'll need to ensure that you keep up with the latest releases and security patches to maintain a secure and efficient gateway.

Community and Ecosystem:

While YARP has an active community, it may not be as widely adopted or have as large an ecosystem as some other API Gateway solutions. This can affect the availability of third-party plugins and support.

Advanced Features:

YARP is designed to be lightweight and focused on core reverse proxy functionality. If you require advanced features, such as built-in API rate limiting, caching, or complex transformations, you might need to implement or integrate additional components separately.

Scalability Limits:

While YARP can be horizontally scaled to handle increased traffic, it may have scalability limits compared to more specialized API Gateway solutions in high-traffic scenarios. Load balancing and routing capabilities may not be as advanced as some dedicated solutions.

Vendor Lock-In:

Using YARP in a predominantly .NET environment might create some vendor lock-in concerns. If you decide to switch away from .NET or move to a different cloud platform, it might be challenging to adapt your gateway setup.
When considering YARP for your microservices architecture, it's essential to evaluate these disadvantages alongside its advantages to determine if it aligns with your specific needs and constraints. Depending on your architectural requirements, you may find that a different API Gateway solution is a better fit for your environment.

Conclusion:

YARP (Yet Another Reverse Proxy) offers a range of advantages as an API Gateway in a microservices architecture, particularly in .NET-centric environments. Its lightweight design, integration with the .NET ecosystem, configuration-driven routing, and support for dynamic routing, load balancing, security features, and customization make it a compelling choice. Additionally, being open source and having an active community can be beneficial.
However, it's important to be aware of the potential disadvantages of YARP, such as its strong dependency on the .NET ecosystem, complexity in configuration, limitations in language

support, and the need for development effort to customize and extend its functionality. Maintenance, ecosystem size, and scalability limits should also be considered.
The suitability of YARP as an API Gateway for your microservices architecture depends on your specific technology stack, requirements, and the level of expertise within your team. It's advisable to carefully assess your architectural needs and evaluate whether YARP aligns with them, or if other API Gateway solutions better meet your specific use case. Ultimately, the choice of an API Gateway should be made in consideration of your unique architectural and operational constraints.

10

Microservices Deployments on Microsoft Azure App Service

Overview

In this chapter, we explore Azure App Service and its role in deploying microservices on Microsoft Azure Cloud. We discuss its importance in microservices architecture, explore other Azure options, and review the advantages and disadvantages of Azure Cloud Services.

What is Azure App Service

Azure App Service is a Platform as a Service (PaaS) offering provided by Microsoft Azure, which is a cloud computing platform. It's designed to simplify the deployment and management of web applications, APIs, and mobile app backends. Azure App Service allows developers to focus on their code and application logic while Azure takes care of the infrastructure, scalability, and other operational aspects.

Here are some key features and components of Azure App Service:

Web Apps: Azure App Service primarily targets web applications. It supports a variety of programming languages and frameworks, including .NET, Java, Node.js, Python, PHP, and more. You can deploy web applications, whether they are simple websites, content management systems, or complex web APIs.

Scalability: App Service allows you to easily scale your applications vertically (by changing the instance size) or horizontally (by adding more instances) based on your needs. This helps your application handle traffic spikes and grow as your user base expands.

Deployment Options: You can deploy your applications to Azure App Service using different methods, such as Git, FTP, local Git, Azure DevOps, or containerization with Docker. Continuous Integration and Continuous Deployment (CI/CD) are also supported.

Staging Environments: Azure App Service enables you to create staging slots where you can deploy and test new versions of your application before promoting them to production. This helps ensure a smooth deployment process.

Custom Domains and SSL: You can map custom domains to your web apps and enable SSL (Secure Sockets Layer) to secure your applications using your own SSL certificates or Azure-managed certificates.

Integration with Azure Services: App Service easily integrates with various Azure services like Azure SQL Database, Azure Functions, Azure Cosmos DB, and more, allowing you to build comprehensive cloud-based applications.

Auto-scaling: You can set up automatic scaling rules to dynamically adjust the number of instances based on metrics such as CPU utilization, memory usage, or custom performance counters.

Built-in Monitoring and Diagnostics: Azure App Service provides tools for monitoring your applications and diagnosing issues. You can use Application Insights for performance monitoring, logging, and troubleshooting.

Security: It offers security features like Web Application Firewall (WAF) to protect against web application attacks, authentication and authorization options, and the ability to restrict access to your application using IP restrictions.

Operating System Choice: You can choose between Windows-based and Linux-based web app environments, depending on your application's requirements.

Support for Containers: Azure App Service for Containers allows you to deploy Docker containers as web applications, giving you flexibility in your application architecture.

Managed Service: Azure takes care of the underlying infrastructure, including patching, scaling, and high availability, allowing you to focus on your application development.

Azure App Service is an excellent choice for organizations and developers looking to host web applications in the cloud without the complexity of managing infrastructure. It's a cost-effective and scalable solution for a wide range of web-based projects.

Web Apps:

Azure App Service primarily focuses on hosting web applications. It supports a wide range of programming languages and frameworks, including .NET, Java, Node.js, Python, PHP, and more. Whether you're building a simple website, a content management system, or a complex web API, you can deploy and manage your web applications on Azure App Service.

Scalability:

Azure App Service offers easy scalability options. You can scale your applications vertically by adjusting the instance size, which means changing the computational resources allocated to your app. You can also scale horizontally by adding more instances of your app to handle increased traffic. This scalability is essential for ensuring your application's performance and availability.

Deployment Options:

You have multiple deployment options with Azure App Service. You can deploy your applications using Git, FTP, local Git, Azure DevOps, or containerization with Docker. This flexibility allows you to choose the deployment method that best fits your workflow. Additionally, you can set up Continuous Integration and Continuous Deployment (CI/CD) pipelines for automated deployments.

Staging Environments:

Azure App Service supports staging slots, which are separate environments for testing and deploying new versions of your application. This allows you to verify that changes work as expected before promoting them to the production environment. Staging environments are crucial for maintaining the reliability of your applications.

Custom Domains and SSL:

You can map custom domains (e.g., www.yourdomain.com) to your web apps hosted on Azure App Service. Additionally, you can enable SSL (Secure Sockets Layer) to secure data transmission between your users and your application. Azure allows you to use your own SSL certificates or Azure-managed certificates for this purpose.

Integration with Azure Services:

Azure App Service seamlessly integrates with various Azure services. This integration allows you to build comprehensive cloud-based applications. For example, you can connect your web app to Azure SQL Database for data storage, Azure Functions for serverless compute, and Azure Cosmos DB for globally distributed databases, among others.

Auto-scaling:

Auto-scaling enables your application to adapt to changing workloads automatically. Azure App Service allows you to set up rules based on metrics like CPU utilization, memory usage, or custom performance counters. When these metrics breach defined thresholds, Azure will automatically adjust the number of instances to meet demand.

Built-in Monitoring and Diagnostics:

Azure App Service provides tools for monitoring and diagnosing your applications. Application Insights is a built-in tool that helps you monitor application performance, capture logs, and troubleshoot issues. It offers insights into how your application is performing and where problems may arise.

Security:

Azure App Service offers various security features. You can use Azure Web Application Firewall (WAF) to protect your web applications against common web application attacks. It also supports authentication and authorization options, and you can restrict access to your application by setting up IP restrictions.

Operating System Choice:

You can choose the operating system for your web app. Azure App Service supports both Windows and Linux environments, allowing you to select the platform that aligns with your application's requirements and technology stack.

Support for Containers:

Azure App Service for Containers allows you to deploy applications within Docker containers. This is particularly beneficial if your application architecture is containerized. It offers flexibility in deploying and managing containerized web applications in the Azure ecosystem.

Managed Service:

One of the key benefits of Azure App Service is that it's a fully managed service. Microsoft Azure takes care of the underlying infrastructure, including server provisioning, patching, scaling, and high availability. This allows developers and teams to concentrate on building and maintaining their applications without worrying about server management tasks.
Azure App Service simplifies web application deployment and management, provides scalability and deployment flexibility, offers built-in monitoring and security features, and seamlessly integrates with other Azure services to create robust cloud-based solutions.

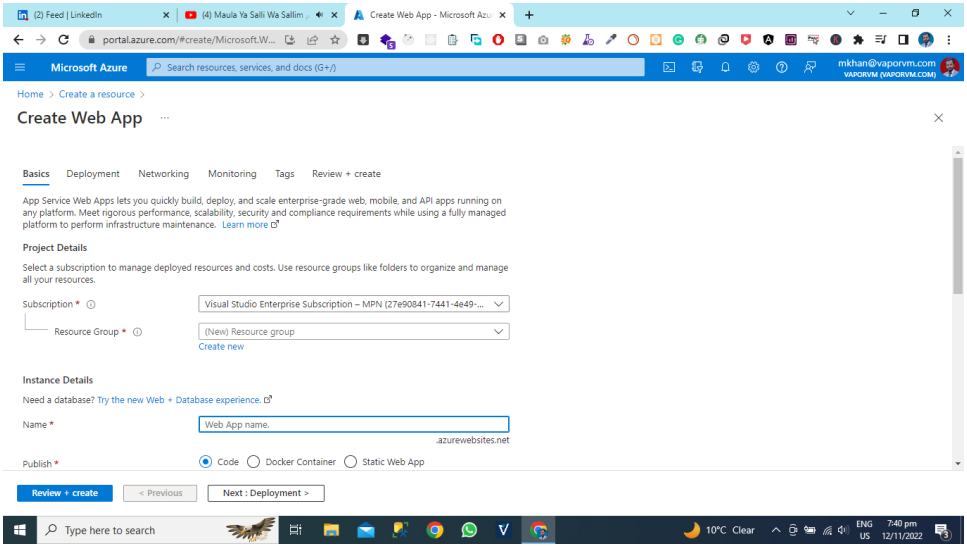
Deploy Microservices on Azure Cloud

Open Your Azure portal

First you need to open your Microsoft azure account after that go to home and then select the subscription where you want to create azure web app.

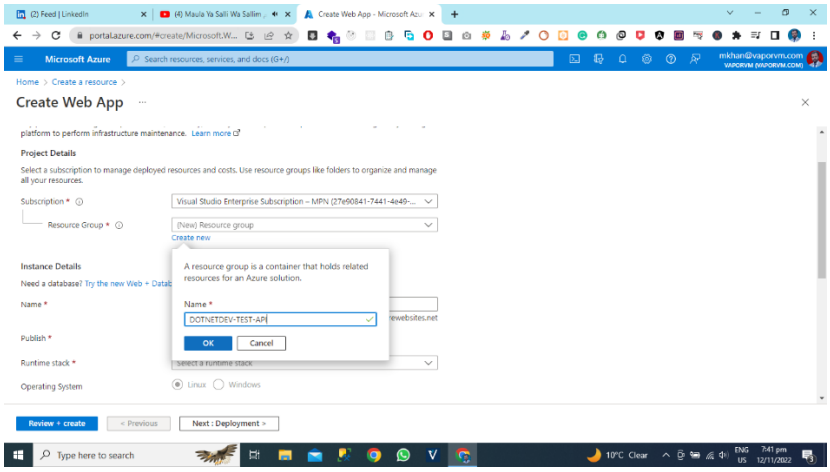
Create Azure Web App

Select the azure app from resources then follow the steps that are necessary for the azure application Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.



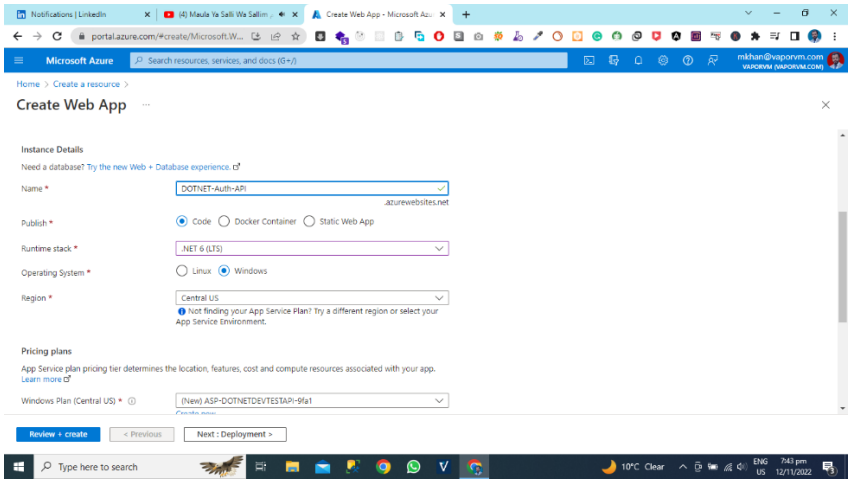
Create the Azure Resource Group.

Now Select the Azure Subscription and create the Azure resource group.



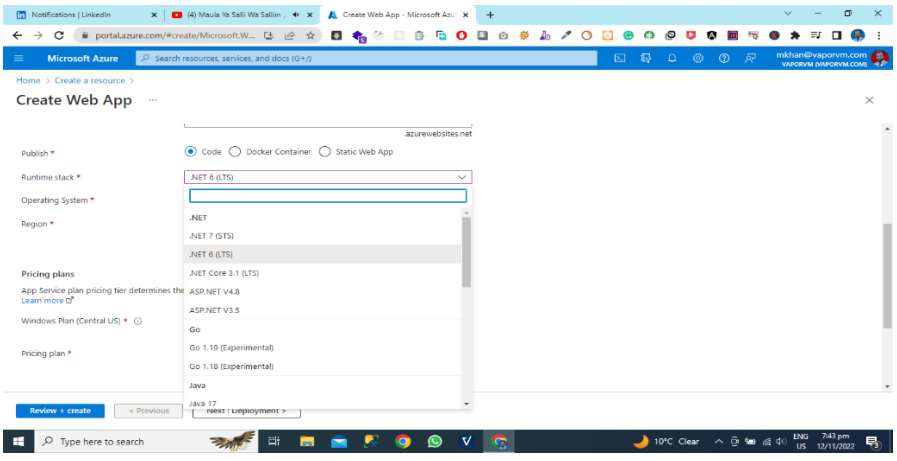
Select the Name of Your Application

Select the Azure App Name and I am selecting my web app name is DOTNET-Auth-API



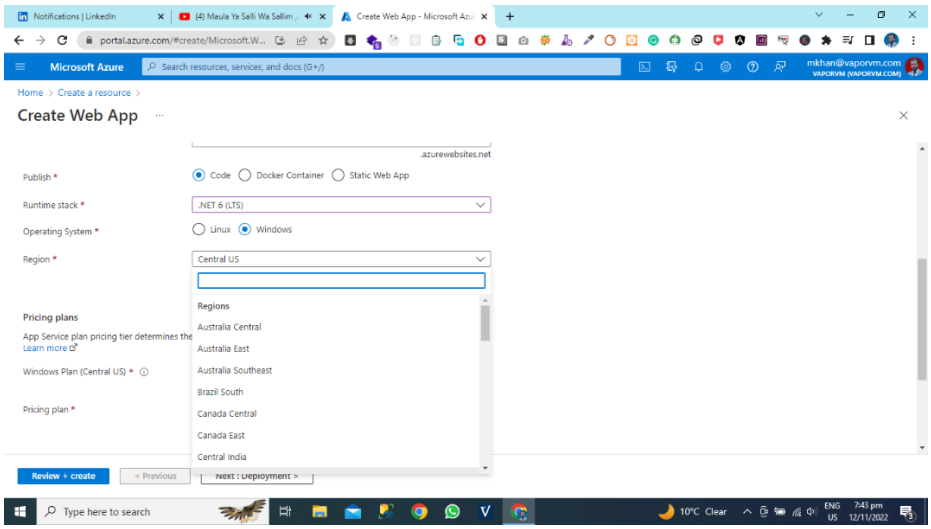
Select the Runtime Stack

After selecting the name and Recourse group now select the code version for your application and my application is using Microsoft Azure Asp.Net 6 so I am selecting .NET 6 Latest stable Version.



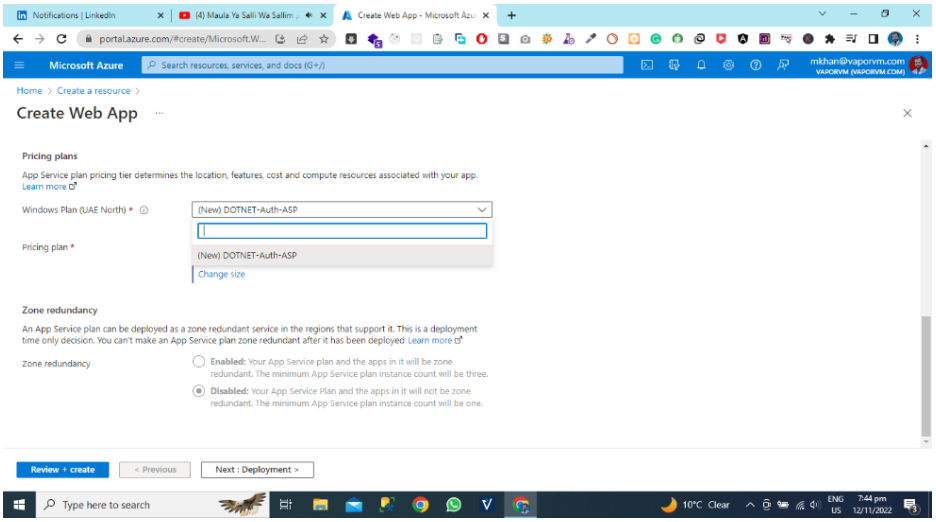
Select the Azure Region

After Selecting the Azure Code Runtime and then select the azure region where you want to deploy your application azure regions are very important for application cost and application deployment cost. So, I am selecting Central US.



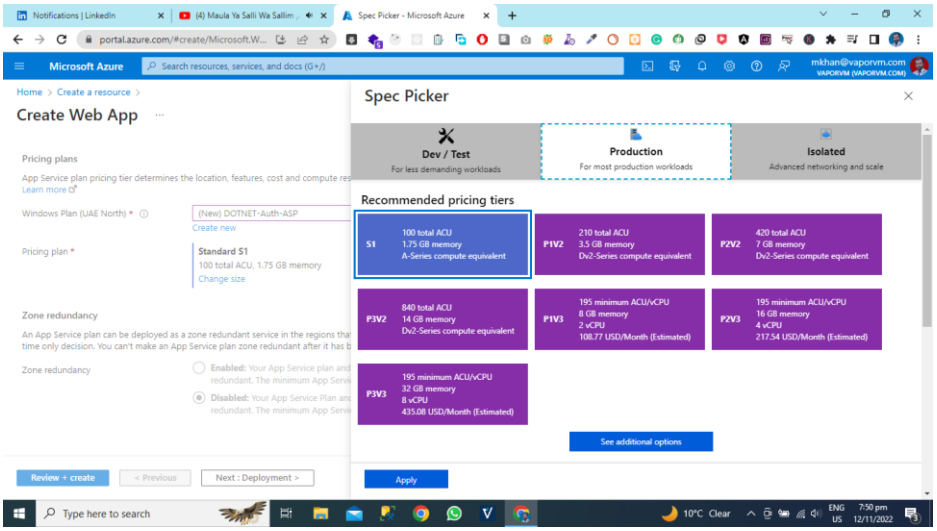
Create App Service Plan

An App Service plan defines a set of compute resources for a web app to run. These compute resources are analogous to the server farm in conventional web hosting. One or more apps can be configured to run on the same computing resources (or in the same App Service plan).

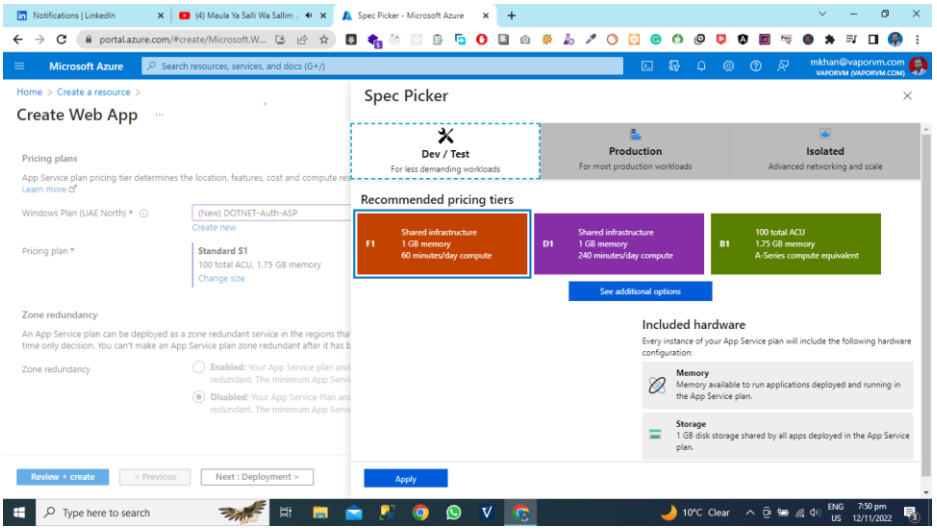


Select the pricing plan.

Now Select the pricing Tier Dev/Test, Production, Isolated

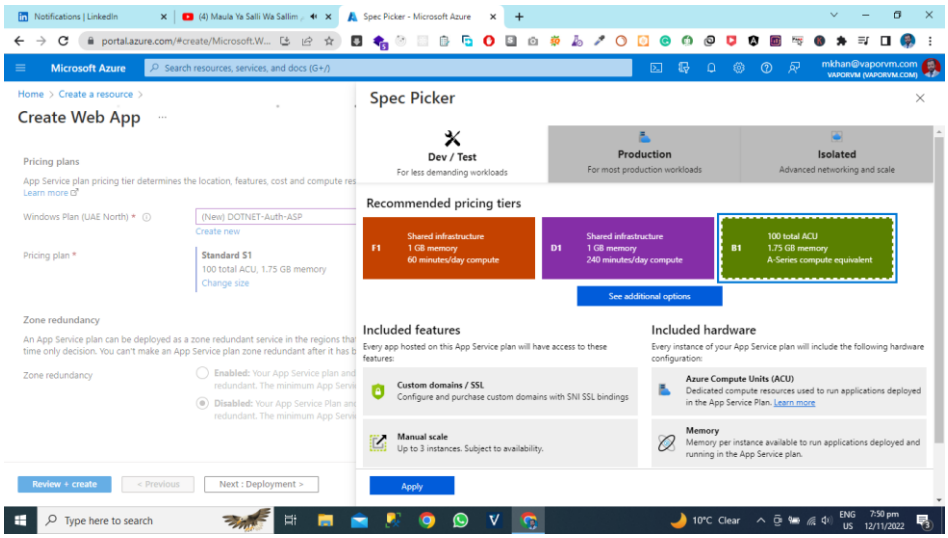


Now our purpose of application is development and testing, so we are selecting Dev/Test



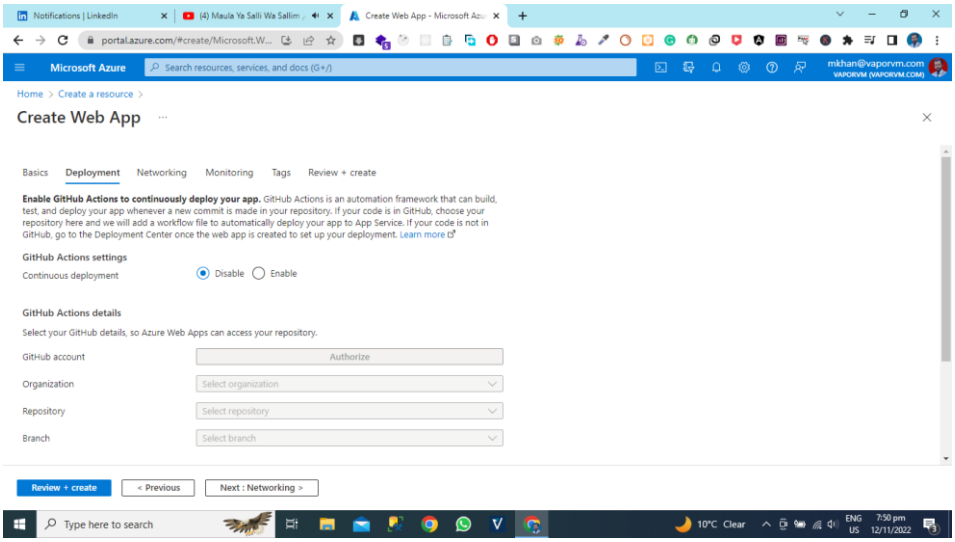
Select the Recommended Pricing Tier

Here you can see the recommended pricing tier and we are selecting the BI Green



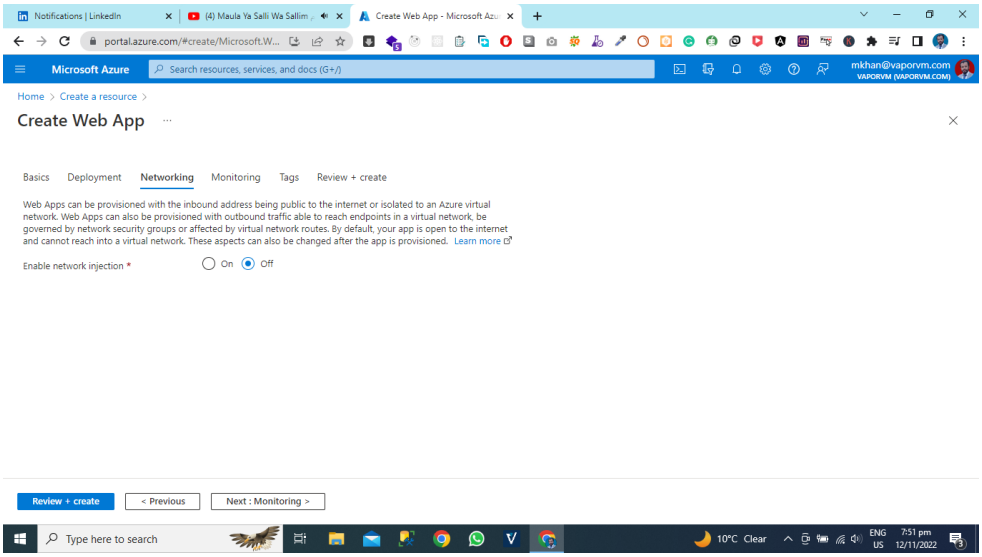
Select the Deployment Option.

Enable GitHub Actions to continuously deploy your app. GitHub Actions is an automation framework that can build, test, and deploy your app whenever a new commit is made in your repository. If your code is in GitHub, choose your repository here and we will add a workflow file to automatically deploy your app-to-App Service. If your code is not in GitHub, go to the Deployment Center once the web app is created to set up your deployment.



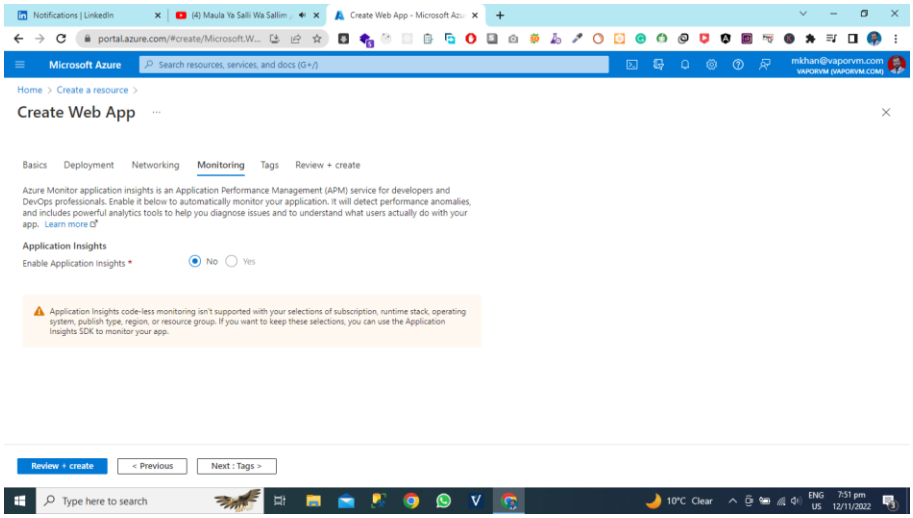
Select The Net Working Options

Web Apps can be provisioned with the inbound address being public to the internet or isolated to an Azure virtual network. Web Apps can also be provisioned with outbound traffic able to reach endpoints in a virtual network, be governed by network security groups or affected by virtual network routes. By default, your app is open to the internet and cannot reach into a virtual network. These aspects can also be changed after the app is provisioned.



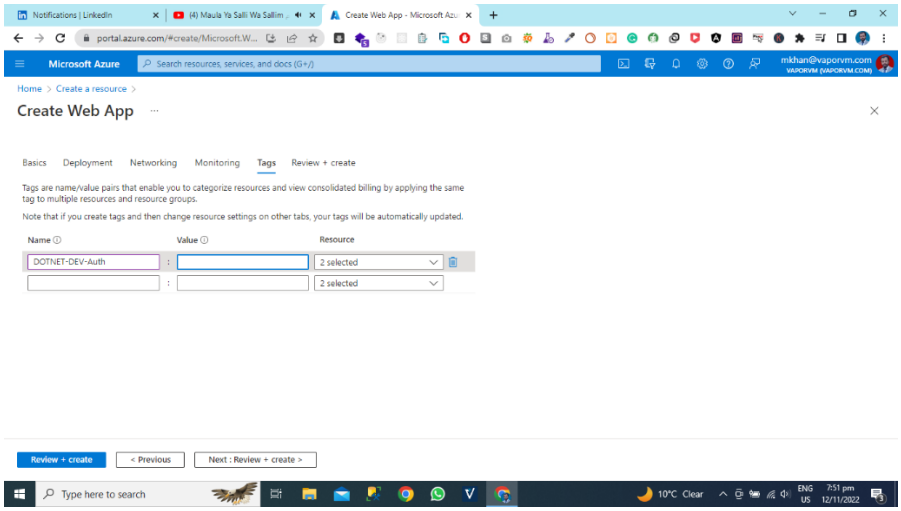
Select the Azure Monitoring Options

Azure Monitor application insights is an Application Performance Management (APM) service for developers and DevOps professionals. Enable it below to automatically monitor your application. It will detect performance anomalies, and includes powerful analytics tools to help you diagnose issues and to understand what users actually do with your app.



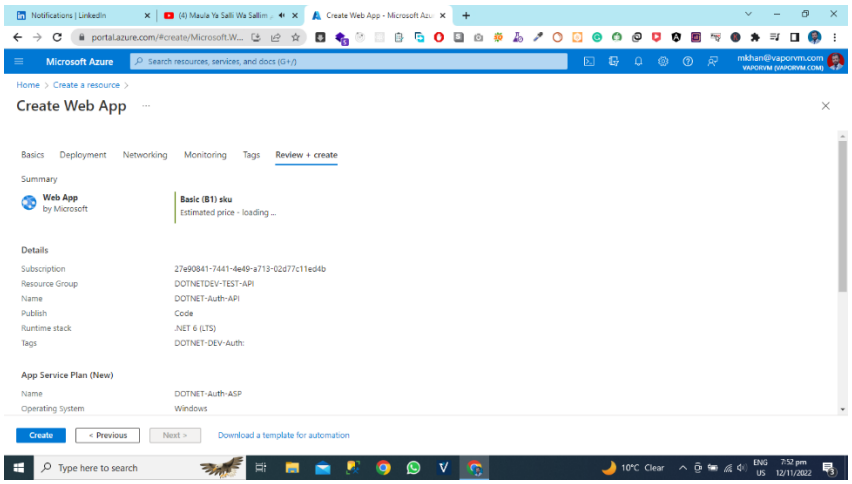
Select the Tags Options

Tags are name/value pairs that enable you to categorize resources and view consolidated billing by applying the same tag to multiple resources and resource groups.
Note that if you create tags and then change resource settings on other tabs, your tags will be automatically updated.

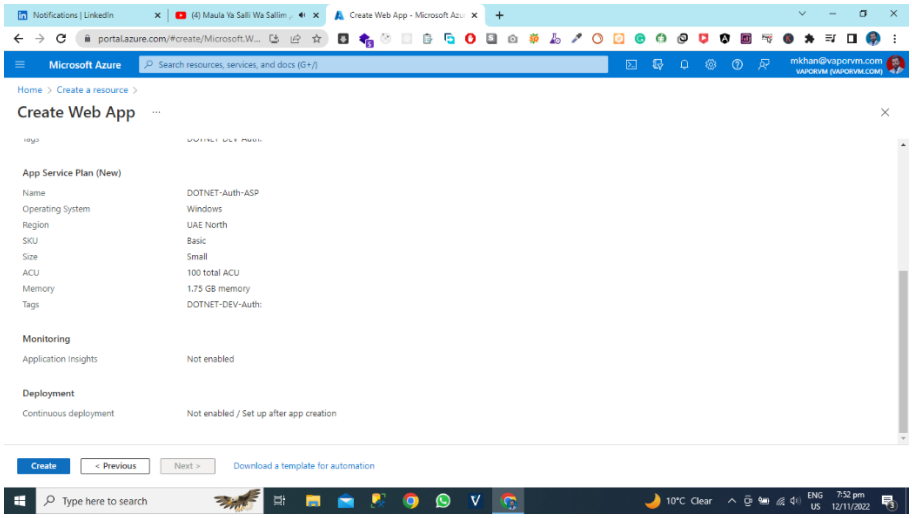


Review and Create

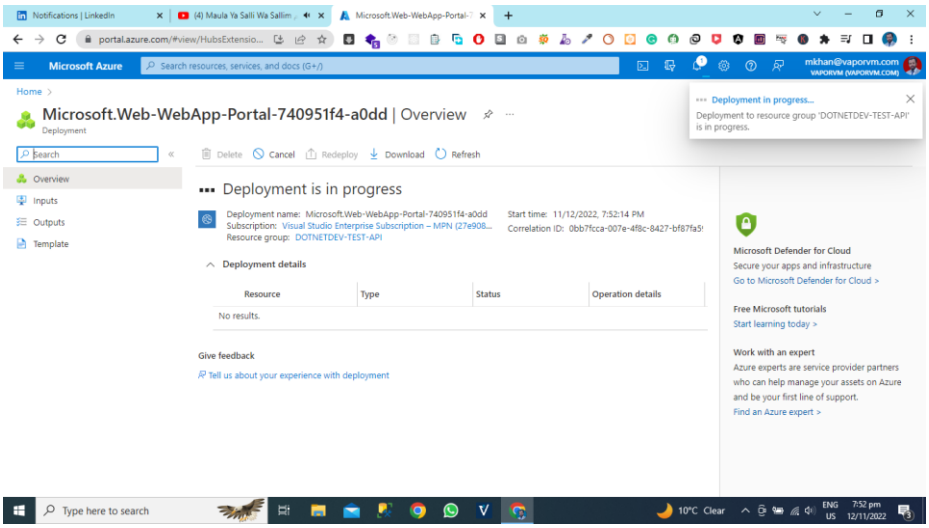
Now Review all your Steps and Create the Azure App.



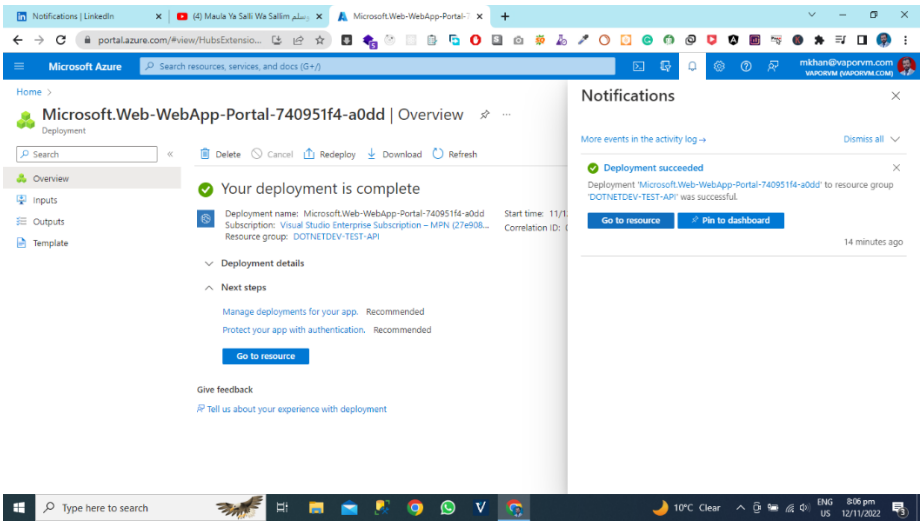
Now hit the create button and then Azure app creation process has will start.



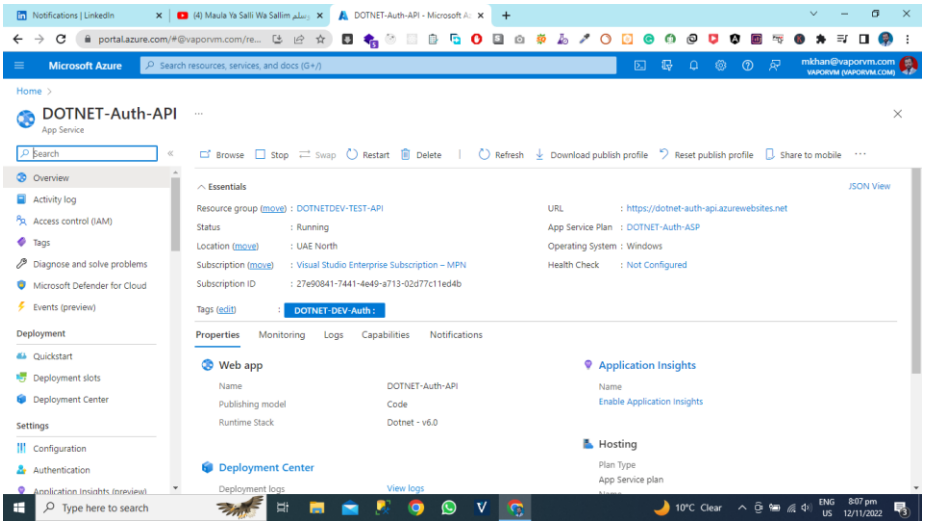
You can see in the given below picture is application deployment process is in progress.



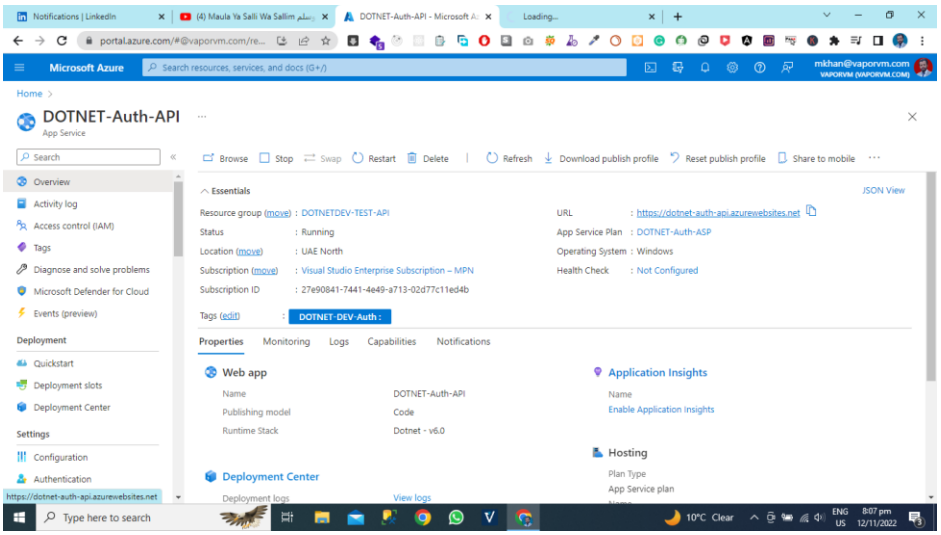
Now you can see that our deployment process succeeded, and we can go to the Created resource.



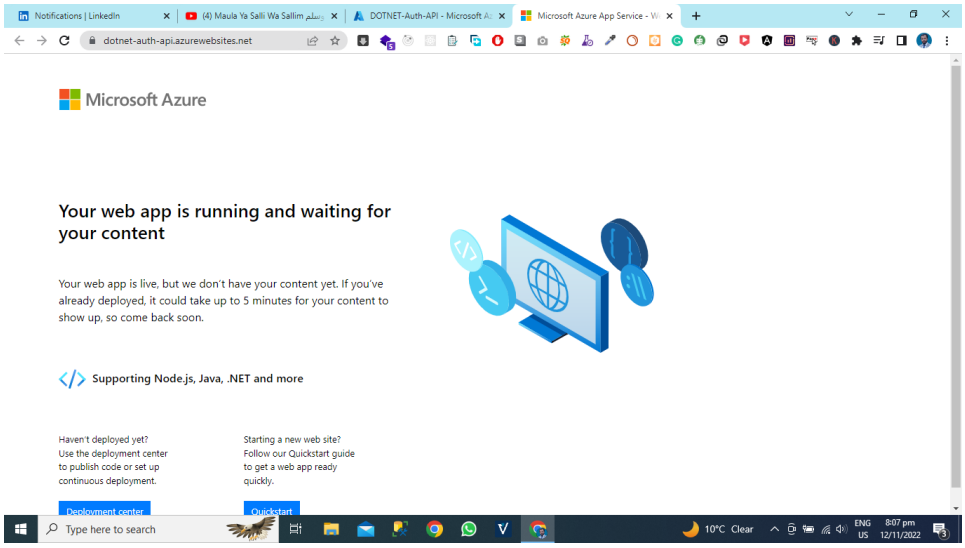
Our Azure App Creation Process Has Been Done.



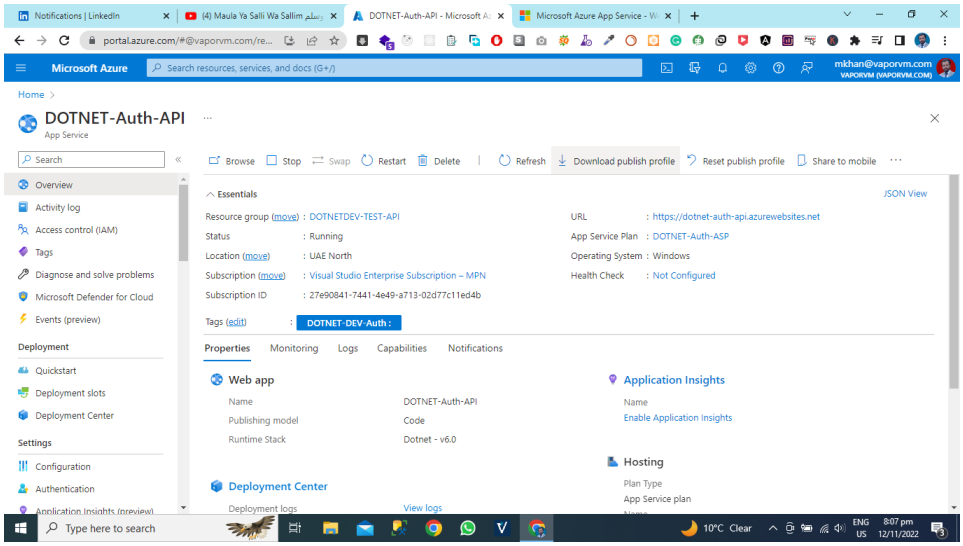
Now you need to hit the Azure App URL.



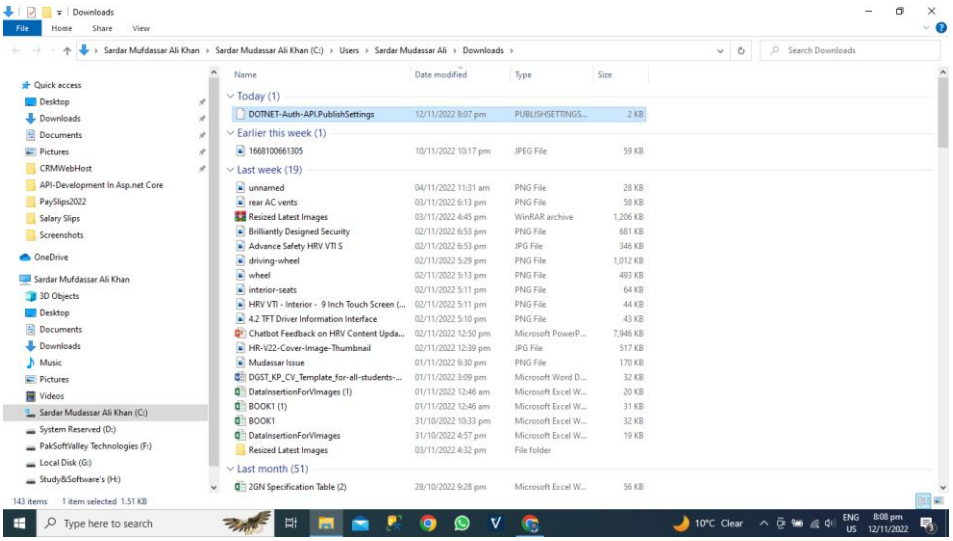
Application Status: Now You Can See that Our Application is Live on the Azure Cloud.



Get the publish profile: Now you need to download the publish profile for deployment of Asp.net core application using Visual Studio.

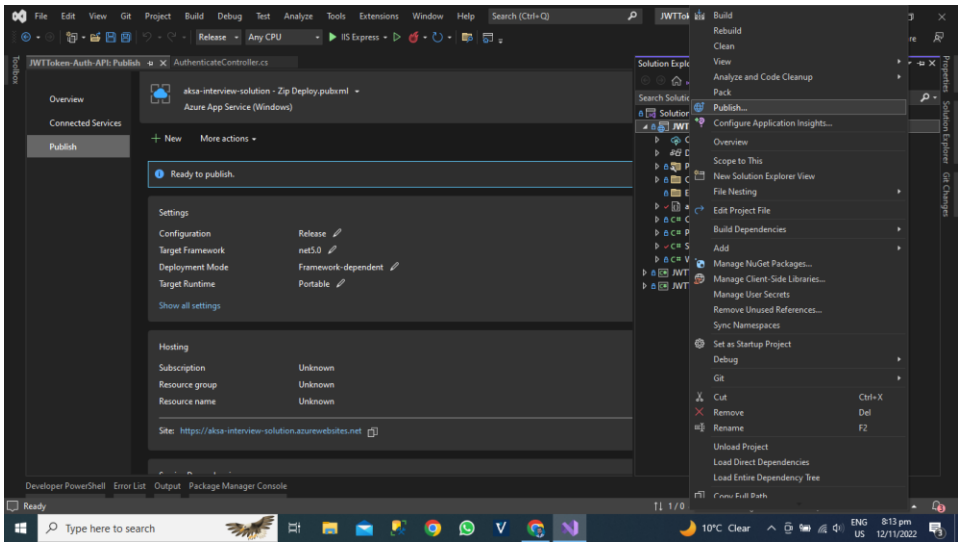


We have Downloaded the Our Application Publish Profile

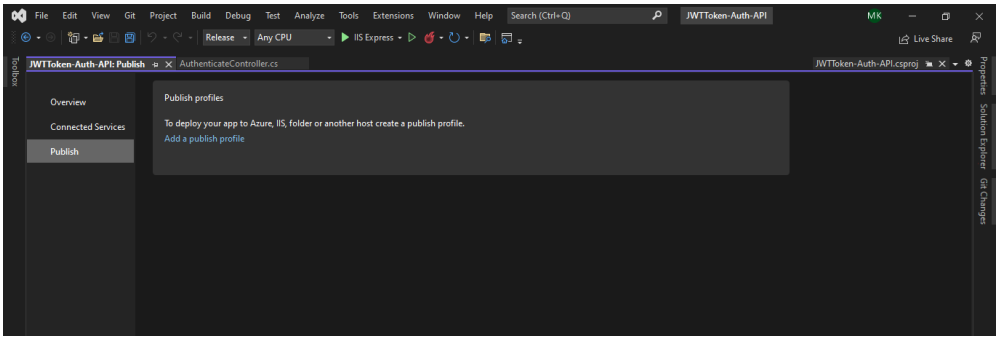


Publish the App Using Visual Studio

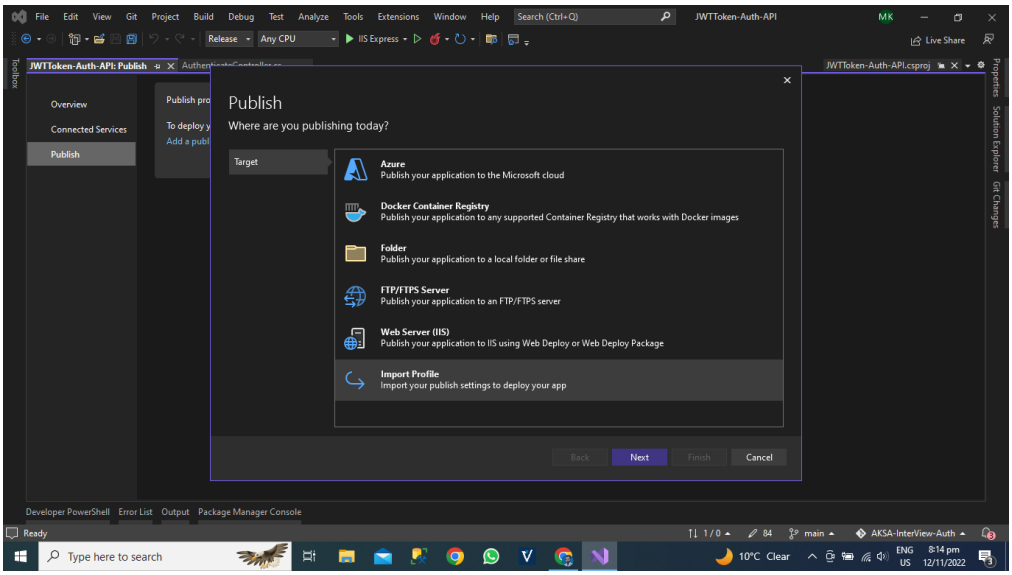
Now we will open the visual studio and then open the Auth API Project after the Right click on project select the publish Option.



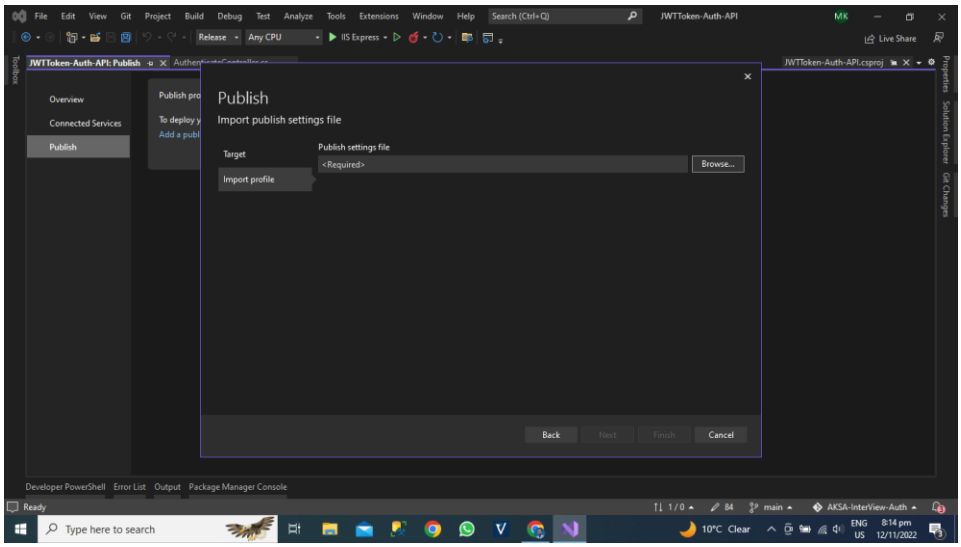
A new window will be visible in front of you, and you can see we have the option for Add Publish Profile. Click the button add publish profile.



Now you can see that we have the option for Import Profile, and we will import the profile that we have downloaded from the Azure App Service portal. Click the Import Profile Option.

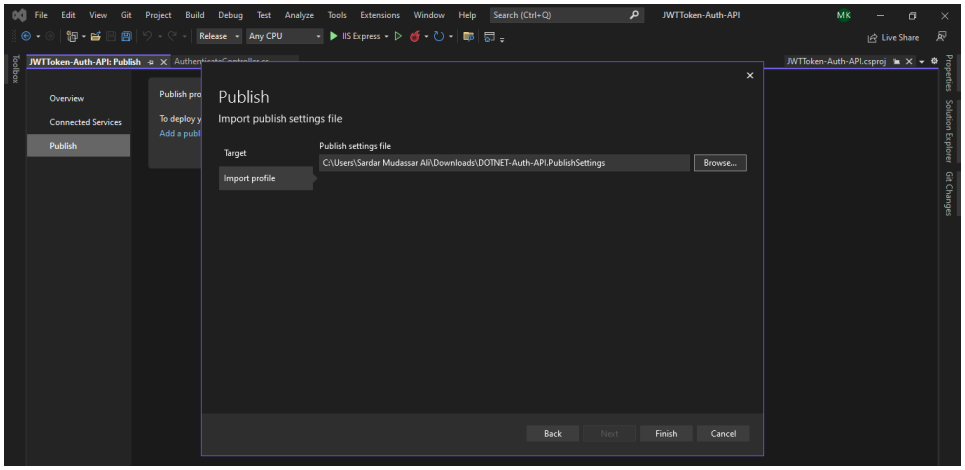


Browse the file for Azure App Publish Profile where you have saved the file.

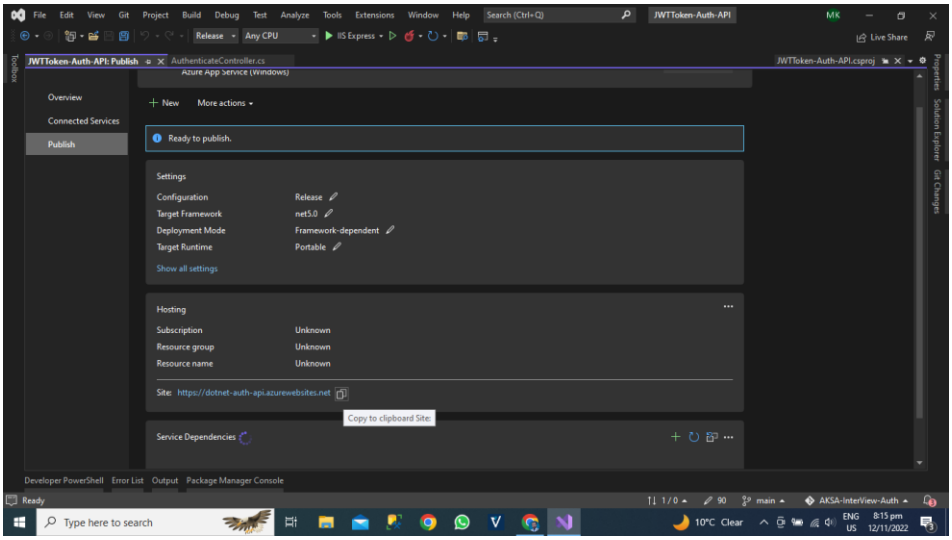


Select the Publish Profile

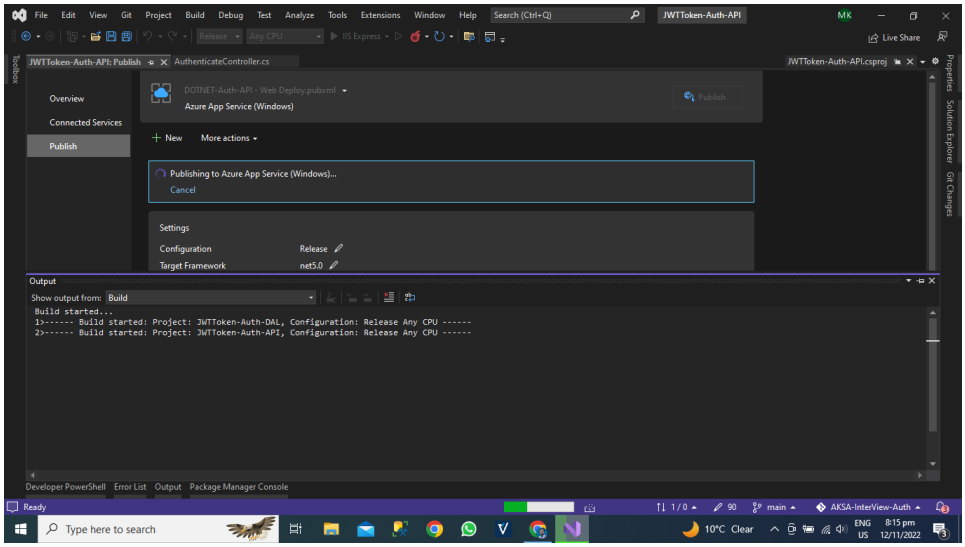
I have selected the file from the downloads folder now click the finish button after some processing visual studio will get all information the azure portal for our Azure App.



Now you can see that visual studio get all the information about our Azure App you can see the URL of our application.

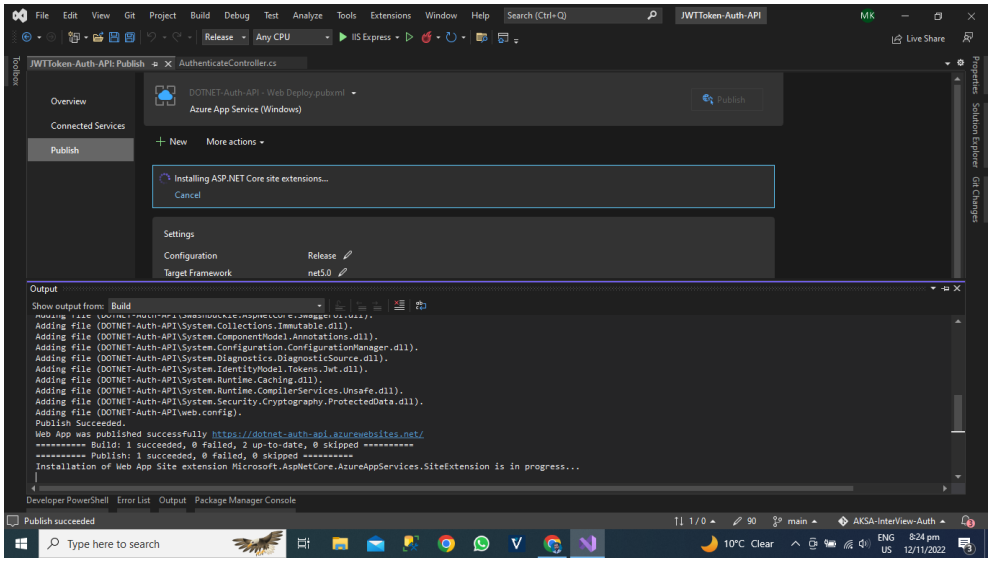


Now Hit the Publish Button after that publishing of our application on azure cloud has been started.



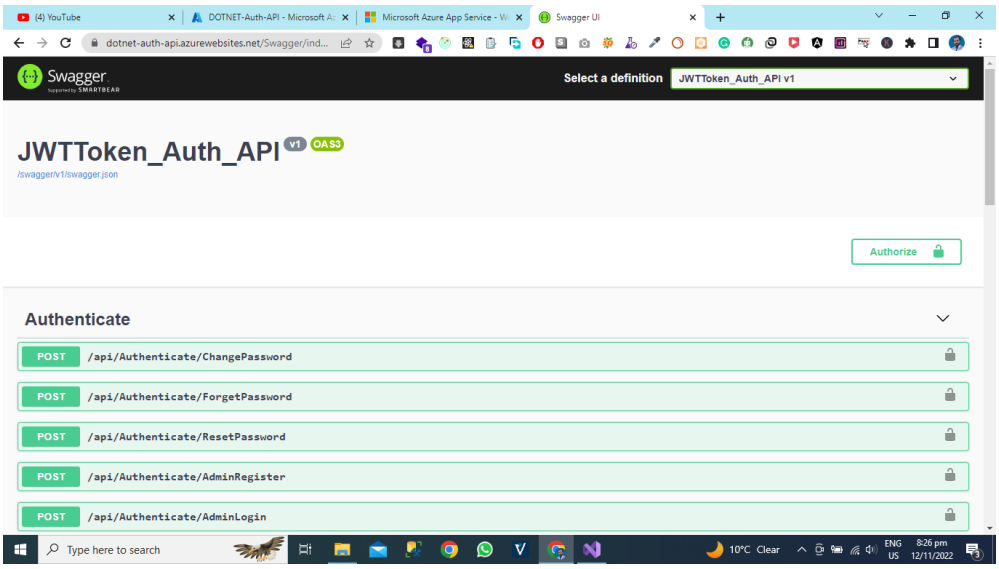
Deployment Status

Our Publication on Azure Cloud has been done. You can see the publication status successful.



Application Deployed on Azure Cloud

Now Hit the URL Of Azure App after URL Just Write URL OF YOUR APPLICATION/Swagger You can see that our all APIS Are Now Live on Azure Cloud Using Azure App service.



Why we use Azure App Service for microservices deployment.

Azure App Service is a Platform as a Service (PaaS) offering that is typically used for hosting web applications, APIs, and mobile app backends. While it's not the most common choice for deploying microservices, there are situations where Azure App Service can be a suitable option for microservices deployment, depending on the specific requirements and constraints of your project. Here are some reasons why you might use Azure App Service for microservices deployment:

Simplicity and Rapid Deployment: Azure App Service provides an easy and quick way to deploy microservices, especially if your microservices are designed to be stateless and can run as web applications. You can deploy each microservice as a separate web app, which simplifies the deployment process and reduces operational complexity.

Isolation and Scaling: By deploying each microservice as a separate web app, you can achieve isolation between microservices. This means that issues with one microservice are less likely to impact others. Azure App Service also supports auto-scaling, allowing you to scale individual microservices independently based on their specific resource needs.

Integration with Azure Services: Azure App Service seamlessly integrates with other Azure services. This can be advantageous when your microservices need to communicate with databases, message queues, caching services, or other Azure resources. You can easily connect your microservices to these services for data storage and processing.

Continuous Integration and Deployment (CI/CD): Azure App Service supports various deployment methods, including Git, Azure DevOps, Docker containers, and more. This facilitates setting up CI/CD pipelines for your microservices, automating the deployment process and ensuring that each microservice is continuously updated and tested.

Service Discovery and Load Balancing: Azure App Service includes features like Azure Traffic Manager, Application Gateway, and Azure Front Door, which can be used for service discovery and load balancing across multiple microservices. These features help distribute traffic to the appropriate microservices and manage failover.

Managed Service: Azure App Service is a fully managed platform, meaning Microsoft takes care of the underlying infrastructure, patching, and maintenance tasks. This reduces the operational burden on your development and DevOps teams, allowing them to focus more on building and improving microservices. However, it's important to note that while Azure App Service can be used for deploying microservices, it may not be the best choice for all microservices scenarios. Consider the following factors:

Stateless Microservices: Azure App Service is best suited for stateless microservices. If your microservices require persistent state or have specific containerization requirements, you might want to explore Azure Kubernetes Service (AKS) or Azure Service Fabric as alternative deployment options.

Complex Microservices Architecture: If your microservices architecture is highly complex and involves intricate communication patterns, you might find it more suitable to use container orchestration platforms like Kubernetes (AKS) for better control and management.

Resource Intensive Microservices: If your microservices have high resource demands, Azure App Service might not be cost-effective. Azure Kubernetes Service (AKS) can be more efficient for managing resource-intensive microservices.

Heterogeneous Technology Stacks: Azure App Service primarily targets web applications, so if your microservices are developed using various programming languages and technologies, you might need to adapt and configure each microservice accordingly.

While Azure App Service can be a convenient and straightforward option for deploying certain types of microservices, it's essential to assess your project's specific requirements and constraints to determine if it's the most suitable choice or if other Azure services, such as Azure Kubernetes Service or Azure Service Fabric, might better meet your needs.

Importance of Azure App Service in the scenario of Microservice Architecture

Azure App Service plays a significant role in microservices architecture by providing several key advantages and addressing various challenges commonly associated with microservices. Here are some of the important aspects of Azure App Service's role in the context of microservice architecture:

Simplicity and Rapid Deployment:

Azure App Service simplifies the deployment of microservices as individual web applications. Each microservice can be hosted as a separate web app, making it easier to manage and deploy.Microservices can be developed using a variety of programming languages and frameworks, and Azure App Service supports many of them, allowing you to choose the right technology for each microservice.

Isolation and Scalability:

Azure App Service enables you to achieve isolation between microservices by hosting each one separately. Issues in one microservice are less likely to affect others. You can independently scale individual microservices based on their specific resource and performance requirements. This ensures efficient resource utilization.

Integration with Azure Services:

Microservices often rely on various Azure services for data storage, messaging, authentication, and more. Azure App Service seamlessly integrates with these Azure services, simplifying the setup of connections to databases, message queues, and other resources. You can leverage Azure's extensive ecosystem of services for tasks like logging, monitoring, and data storage.

Continuous Integration and Deployment (CI/CD):

Azure App Service supports multiple deployment methods, including Git, Azure DevOps, Docker containers, and more. This makes it easy to set up CI/CD pipelines for your microservices, enabling automated testing and deployment of changes.
CI/CD pipelines ensure that each microservice can be updated and released independently, facilitating a more agile and efficient development process.

Service Discovery and Load Balancing:

Azure App Service can be used in conjunction with Azure services like Traffic Manager, Application Gateway, and Azure Front Door for service discovery and load balancing across multiple microservices.
These features help route traffic to the appropriate microservices, manage failover, and provide high availability for your microservices.

Managed Service:

Azure App Service is a fully managed platform. Microsoft Azure takes care of the underlying infrastructure, including server provisioning, patching, and maintenance tasks. This reduces the operational burden on your teams, allowing them to focus on developing and improving microservices.

Elastic Scalability:

Azure App Service allows for automatic scaling based on predefined rules and metrics. When your microservices experience fluctuations in traffic, they can dynamically adjust the number of instances to meet the demand.
This elastic scalability is crucial for microservices that experience varying workloads and traffic patterns.

Cost Efficiency:

Azure App Service's consumption-based pricing model can be cost-effective, especially for smaller to medium-sized microservices. You pay only for the resources you consume, which can lead to significant cost savings compared to maintaining dedicated infrastructure for each microservice.

While Azure App Service offers many advantages for microservices, it's important to recognize that it may not be suitable for all microservices scenarios. Microservices with specific requirements, such as those that are resource-intensive or require container orchestration, may be better served by other Azure services like Azure Kubernetes Service (AKS) or Azure Service Fabric. Therefore, the choice of Azure services should be tailored to the specific needs and constraints of your microservices architecture.

Importance of Microsoft Azure Cloud Apps in Microservices Architecture

Microsoft Azure offers a range of cloud services and tools that are invaluable in the context of microservices architecture. Here are some of the keyways in which Microsoft Azure Cloud Apps are important in a microservices architecture:

Scalability and Elasticity:

Azure Cloud Apps, such as Azure App Service and Azure Kubernetes Service (AKS), provide robust scaling options. Microservices often have varying workloads, and Azure enables automatic scaling based on metrics like CPU utilization, ensuring that each microservice can handle fluctuating demands.

Managed Services:

Azure's PaaS offerings, including Azure App Service, are fully managed. Azure takes care of the underlying infrastructure, which means less operational overhead for your development and operations teams. This is especially beneficial for organizations focusing on delivering microservices without the complexities of managing infrastructure.

Container Orchestration:

Azure Kubernetes Service (AKS) is an essential component for managing containerized microservices. It simplifies the orchestration, scaling, and management of containers, which are the building blocks of many microservices.

Integration with Azure Services:

Azure offers a wide range of complementary services like Azure SQL Database, Azure Cosmos DB, Azure Functions, and Azure Service Bus. These services seamlessly integrate with microservices, providing solutions for data storage, messaging, serverless computing, and more.

DevOps and CI/CD:

Azure DevOps and Azure Pipelines facilitate DevOps practices, enabling continuous integration and continuous deployment (CI/CD) for microservices. This ensures that code changes are automatically built, tested, and deployed, reducing the risk of errors and speeding up development cycles.

Monitoring and Analytics:

Azure Application Insights is a powerful tool for monitoring and diagnosing microservices. It provides insights into application performance and usage, helping to detect and resolve issues quickly.

Security and Compliance:

Azure offers robust security features, including identity and access management, encryption, and compliance certifications. These are crucial for securing microservices and ensuring that they adhere to industry-specific regulations and standards.

Hybrid and Multi-Cloud Capabilities:

Azure enables hybrid cloud and multi-cloud deployments. You can build and manage microservices that run both in the Azure cloud and on-premises, which is essential for organizations with hybrid cloud strategies.

Serverless Computing:

Azure Functions, a serverless computing service, is useful for building microservices that operate on event triggers. It allows you to execute code in response to various events, making it a valuable component of microservices architecture.

High Availability and Disaster Recovery:

Azure provides high availability and disaster recovery options for microservices. Services like Azure Site Recovery help ensure that your microservices remain operational even in the event of failures or disasters.

Cost Efficiency:

Azure's pay-as-you-go model can be cost-effective for microservices. You only pay for the resources you use, allowing you to optimize costs based on the actual demands of your microservices.

Community and Ecosystem:

Azure has a robust community and ecosystem with extensive documentation, tutorials, and third-party tools. This can accelerate the development and management of microservices in Azure. Microsoft Azure Cloud Apps play a vital role in a microservices architecture by providing scalable, managed, and integrated services that simplify the development, deployment, and management of microservices. Azure's diverse offerings cater to various microservices needs, ensuring that organizations can build and operate efficient, reliable, and secure microservices-based applications.

Microsoft Azure Other Options for Microservices deployment

Microsoft Azure provides several options for deploying microservices, allowing you to choose the best approach based on your specific requirements and constraints. Here are some other Azure services and options for microservices deployment:

Azure Kubernetes Service (AKS):

Description: AKS is a managed Kubernetes container orchestration service in Azure. It's designed to simplify the deployment, scaling, and management of containerized microservices.

Benefits: AKS is ideal for complex microservices architectures that rely on containers. It provides robust scaling, load balancing, and container management capabilities.

Azure Service Fabric:

Description: Azure Service Fabric is a platform for building, deploying, and managing microservices. It provides container orchestration, stateful microservices, and application lifecycle management.

Benefits: Service Fabric is well-suited for applications with stateful microservices or applications that require fine-grained control over service-to-service communication.

Azure Functions:

Description: Azure Functions is a serverless compute service that allows you to build and run event-driven microservices. It enables you to execute code in response to various triggers, such as HTTP requests, timers, and data changes.

Benefits: Azure Functions are efficient for building microservices that are triggered by events, such as data changes, making them cost-effective and scalable.

Azure Container Instances (ACI):

Description: Azure Container Instances is a serverless container service that allows you to deploy individual containers quickly and easily without managing clusters.

Benefits: ACI is suitable for running isolated microservices or batch processing tasks that don't require a full container orchestration platform.

Azure Logic Apps:

Description: Azure Logic Apps is a serverless workflow automation service that enables you to create microservices that integrate with various Azure and third-party services.

Benefits: Logic Apps are valuable for building microservices that automate business processes, integrate with external systems, and orchestrate workflows.

Azure Functions Premium Plan:

Description: The Azure Functions Premium Plan offers enhanced features, such as virtual network integration, unlimited execution duration, and advanced scaling capabilities for high demand microservices.

Benefits: This plan is suitable for microservices with specific networking or performance requirements.

Azure Service Bus:

Description: Azure Service Bus is a messaging service that supports asynchronous communication between microservices. It provides queuing and publish-subscribe mechanisms.

Benefits: Service Bus is critical for building microservices that need reliable messaging and event-driven communication.

Azure API Management:

Description: Azure API Management is a service that helps you create, publish, and manage APIs, including those for microservices. It offers features like API security, traffic management, and analytics.

Benefits: API Management simplifies the exposure and management of microservices APIs to external consumers, ensuring security, scalability, and analytics.

Azure Event Grid:

Description: Azure Event Grid is a fully managed event routing service that connects events from different sources to various subscribers, including microservices.

Benefits: Event Grid is valuable for building event-driven microservices that react to events and updates across your Azure services and applications.

Azure App Service for Containers:

Description: Azure App Service for Containers enables you to host containerized microservices as web applications. It combines the ease of Azure App Service with the flexibility of containerization.

Benefits: This option is ideal for microservices that need the simplicity of Azure App Service while leveraging container technology for portability and flexibility.

Azure Data Lake Analytics:

Description: Azure Data Lake Analytics is a serverless analytics service for big data processing. It's suitable for microservices that involve data analytics and processing on large datasets.

Benefits: Data Lake Analytics allows you to build data-centric microservices for tasks like data transformation, analysis, and reporting. Each of these Azure options provides a specific set of features and capabilities, making it crucial to match the right deployment option to the needs and characteristics of your microservices. Azure's versatility allows you to tailor your approach to suit the unique requirements of your applications and workloads.

Advantages of Microsoft Azure Cloud Services in the scenario of Microservices

Ease of Deployment:

Advantage: Azure App Service simplifies the deployment process, making it easy to publish web applications, APIs, and microservices.

Benefit: Developers can focus on writing code and deploying applications without worrying about the underlying infrastructure.

Scalability:

Advantage: App Service offers horizontal and vertical scalability, allowing you to scale your applications easily based on traffic and demand.

Benefit: Your applications can handle traffic spikes and grow as your user base expands.

Managed Infrastructure:

Advantage: Azure manages the underlying infrastructure, including server provisioning, patching, and maintenance.

Benefit: Reduces operational overhead and frees up resources to focus on development and application improvement.

Integration with Azure Services:

Advantage: App Service easily integrates with various Azure services like Azure SQL Database, Azure Functions, and Azure Cosmos DB.

Benefit: You can build comprehensive cloud-based applications by leveraging other Azure services for various functionalities.

Built-in Monitoring and Diagnostics:

Advantage: Azure App Service provides tools like Application Insights for monitoring, logging, and troubleshooting.

Benefit: Helps identify performance bottlenecks and diagnose issues in your applications.

Security Features:

Advantage: Azure App Service offers security features like Web Application Firewall (WAF), authentication, and IP restrictions.

Benefit: Enhances the security of your applications, protecting them against common web application attacks.

Custom Domains and SSL:

Advantage: You can map custom domains and enable SSL, allowing you to provide secure and branded access to your applications.

Benefit: Provides a professional and secure user experience.

Continuous Integration and Deployment (CI/CD):

Advantage: Supports various deployment methods, enabling the setup of CI/CD pipelines for automated deployments.

Benefit: Ensures efficient and reliable software delivery, reducing manual deployment tasks.

Disadvantages of Microsoft Azure App Service in the scenario of Microservices:

Limited Language Support:

Disadvantage: While Azure App Service supports a wide range of programming languages, it may not cover all possible technologies.

Challenge: If your application uses a less common language or framework, you might face limitations.

Resource Constraints:

Disadvantage: Azure App Service may not be suitable for resource-intensive applications.

Challenge: Complex and resource-heavy applications may require more control and dedicated resources, which can be better served by other Azure services like Azure Kubernetes Service (AKS).

Heterogeneous Technology Stacks:

Disadvantage: Azure App Service is primarily designed for web applications, which may not be the best choice for microservices built using various technologies.

Challenge: If your microservices architecture involves different programming languages or frameworks, adapting them to Azure App Service can be challenging.

Complex Microservices Architectures:

Disadvantage: Azure App Service simplifies deployment but may not be the best choice for complex microservices architectures.

Challenge: Highly complex microservices with intricate communication patterns might benefit from container orchestration platforms like Kubernetes (Azure Kubernetes Service).

Cost Considerations:

Disadvantage: While Azure App Service's consumption-based pricing can be cost-effective, it may not be the cheapest option for all scenarios.

Challenge: Understanding the cost implications and selecting the most cost-efficient Azure service for your specific needs is essential.

Azure App Service is a versatile platform for hosting web applications, APIs, and microservices, offering many benefits in terms of ease of use, scalability, and integration with Azure services. However, it's important to carefully assess your project's requirements and constraints to determine if Azure App Service is the right choice or if other Azure services would better serve your needs.

Conclusion

Azure App Service is a valuable platform provided by Microsoft Azure for hosting web applications, APIs, and, to a certain extent, microservices. It offers numerous advantages, including ease of deployment, scalability, managed infrastructure, integration with Azure services, built-in monitoring, and security features. These benefits make it a suitable choice for many application scenarios, especially those where simplicity, rapid deployment, and integration with Azure services are important.
However, Azure App Service also has its limitations, particularly in cases where applications are resource-intensive, involve complex microservices architectures, or require a broad range of programming languages and frameworks. In such situations, other Azure services like Azure Kubernetes Service (AKS) or Azure Service Fabric may be more appropriate.
Ultimately, the choice of Azure App Service or another Azure service depends on the specific requirements, constraints, and goals of your project. Careful consideration of these factors is essential to make an informed decision about which Azure service to use for your web applications, APIs, and microservices.

Ease of Deployment:

Advantage: Azure App Service simplifies the deployment process. You can publish web applications, APIs, and microservices quickly and easily.

Benefit: Developers can concentrate on writing code and deploying applications without being burdened by infrastructure management.

Scalability:

Advantage: Azure App Service offers both horizontal and vertical scalability, allowing applications to be easily scaled based on traffic and demand.

Benefit: This ensures that applications can handle traffic spikes and grow in response to an expanding user base.

Managed Infrastructure:

Advantage: Azure manages the underlying infrastructure, taking care of server provisioning, patching, and maintenance.

Benefit: This reduces operational overhead and allows development teams to focus on coding and improving applications.

Integration with Azure Services:

Advantage: Azure App Service seamlessly integrates with various Azure services, such as Azure SQL Database, Azure Functions, and Azure Cosmos DB.

Benefit: This integration enables the creation of comprehensive cloud-based applications by leveraging additional Azure services for various functions.

Built-in Monitoring and Diagnostics:

Advantage: Azure App Service provides tools like Application Insights for monitoring, logging, and troubleshooting.

Benefit: These tools assist in identifying performance bottlenecks and diagnosing issues, contributing to better application reliability.

Security Features:

Advantage: Azure App Service offers security features, including Web Application Firewall (WAF), authentication, and IP restrictions.

Benefit: These features enhance the security of applications, protecting against common web application attacks.

Custom Domains and SSL:

Advantage: Azure App Service allows you to map custom domains and enable SSL, providing a professional and secure user experience.

Benefit: Users can access applications through branded and secure domains, enhancing trust and credibility.

Continuous Integration and Deployment (CI/CD):

Advantage: Azure App Service supports multiple deployment methods, facilitating the setup of CI/CD pipelines for automated testing and deployment.

Benefit: This ensures a reliable and efficient software delivery process, reducing manual deployment tasks and errors.

11

Deployment of microservices on Microsoft Azure Cloud using CICD Pipeline.

Overview

In this chapter, we explore the Introduction of Azure CICD Pipeline, outlining its key components and benefits. We cover the Prerequisites for Azure CICD Pipeline and guide you through Creating Azure App Service. Prepare to set up and configure an Azure DevOps Project and learn how to Connect Azure DevOps with GitHub. We provide detailed steps for Creating a Release Pipeline and discuss Monitoring and Verifying Deployment to ensure smooth operations. This journey will equip you with the skills to effectively implement Azure CICD Pipelines in your projects.

Introduction

In this chapter, we will explore the process of deploying an ASP.NET MVC application to Azure Cloud using Azure DevOps and GitHub. This approach ensures a seamless continuous integration and continuous deployment (CI/CD) pipeline, facilitating an efficient and robust development workflow. By the end of this chapter, you will have a comprehensive understanding of setting up an end-to-end deployment pipeline from GitHub to Azure using Azure DevOps Classic Editor.

Step 1. Prerequisites

Before we dive into the steps, let's ensure you have the necessary prerequisites.

- **Azure Subscription:** An active Azure subscription is required to deploy your application.
- **Azure DevOps Account:** You need an Azure DevOps account to create and manage the CI/CD pipeline.
- **GitHub Repository:** Your ASP.NET MVC application should be hosted on GitHub.
- **Basic Knowledge of ASP.NET MVC:** Familiarity with ASP.NET MVC will help you follow along with the steps.
- **Azure App Service:** Set up an Azure App Service where your application will be deployed.

Step 2. Creating an Azure App Service

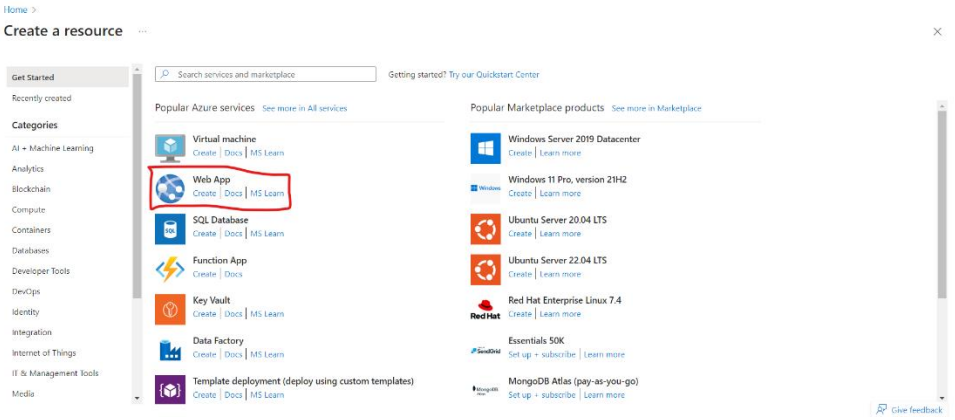
Azure App Service is a fully managed platform for building, deploying, and scaling web apps. Here's how to create one:

- **Log in to Azure Portal:** Navigate to Azure Portal.
- **Create a New Resource:** Click on "Create a resource" in the left-hand menu and select "Web App" under the "Compute" section.

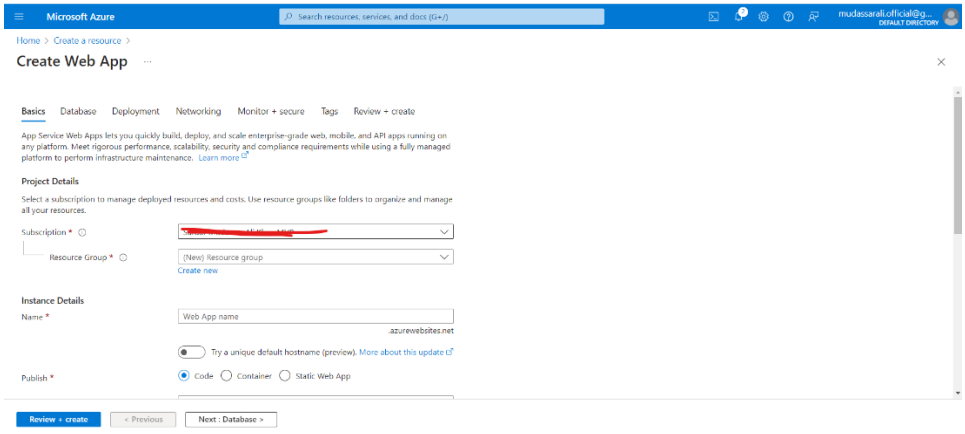
Configure Web App

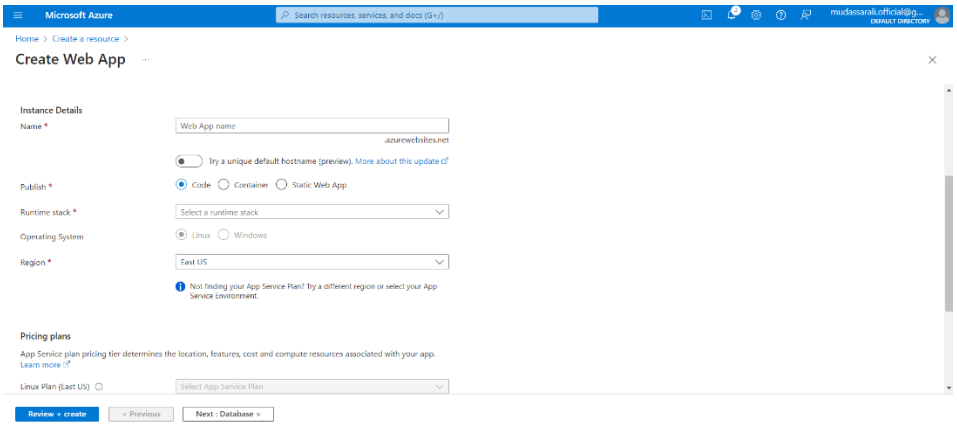
- **Subscription:** Select your Azure subscription.
- **Resource Group:** Create a new resource group or use an existing one.
- **Name:** Provide a unique name for your web app, which will be part of its URL (e.g., yourappname.azurewebsites.net).
- **Publish:** Choose "Code" since you are deploying an ASP.NET MVC application.
- **Runtime stack:** Select ".NET Core" and the appropriate version for your application.
- **Operating System:** Choose "Windows".
- **Region:** Select a region close to your user base to reduce latency.
- **App Service Plan:** Create a new plan or select an existing one. The app service plan determines pricing and available features.
- **Review and Create:** Click "Review + Create" to review your settings and then "Create" to create the web app.

Create the Azure Web app and fill in all the above details.



Fill in the details and then click next.





Step 3. Setting Up an Azure DevOps Project

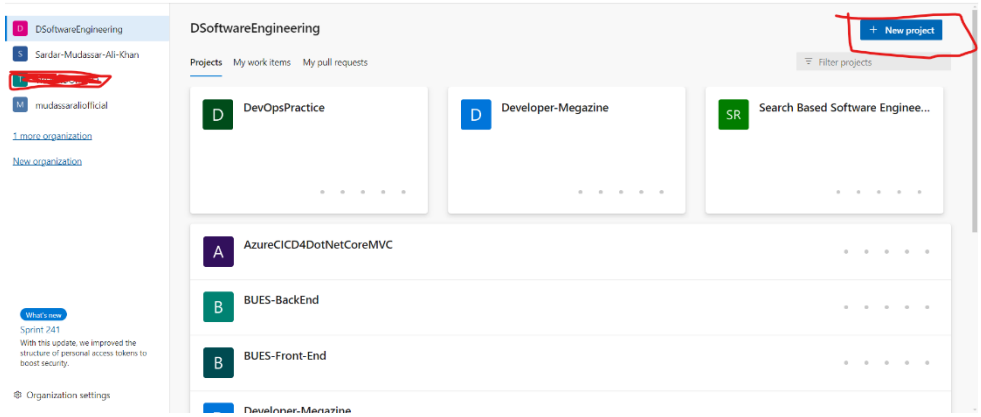
Azure DevOps is a comprehensive suite of development tools that facilitate CI/CD. Follow these steps to set up your project:

Log in to Azure DevOps: Navigate to Azure DevOps.

Create a new project

- Click on the "New Project" button.
- **Project Name:** Enter a name for your project.
- **Description:** Optionally, add a description to describe the project's purpose.
- **Visibility:** Choose either public or private based on your preference.

Create Project: Click "Create" to finalize the project setup.



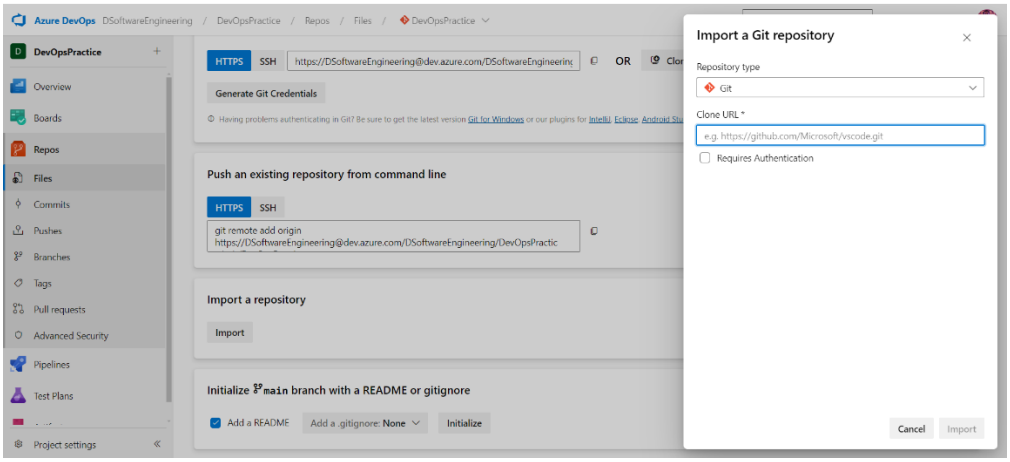
Step 4. Connecting Azure DevOps to GitHub

To link your Azure DevOps project with your GitHub repository.

Go to Repos: In your Azure DevOps project, navigate to "Repos" from the left-hand menu.
Import Repository

- Click on "Import a repository" at the top right.
- **Clone URL:** Provide the URL of your GitHub repository.
- **Authentication:** Enter your GitHub credentials if necessary to access the repository.

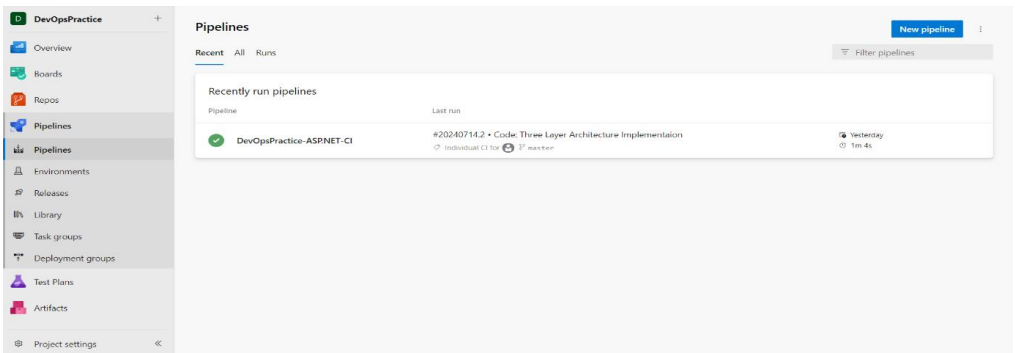
Import: Click "Import" to create a copy of your GitHub repository in Azure DevOps.



Step 5. Creating a Build Pipeline

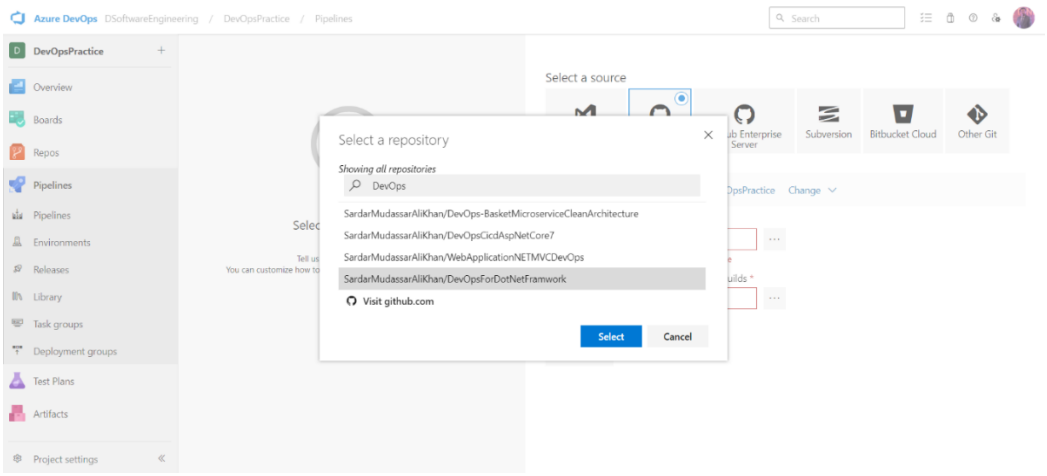
A build pipeline automates the process of building and testing your application. Here's how to create one.

Go to Pipelines: In Azure DevOps, navigate to "Pipelines" and then "Builds".



New Pipeline: Click on "New pipeline" and select "Use the classic editor".

- **Select Repository:** Choose your GitHub repository.
- **Source:** Select "GitHub".
- **Repository:** Choose your repository from the list or provide the URL.

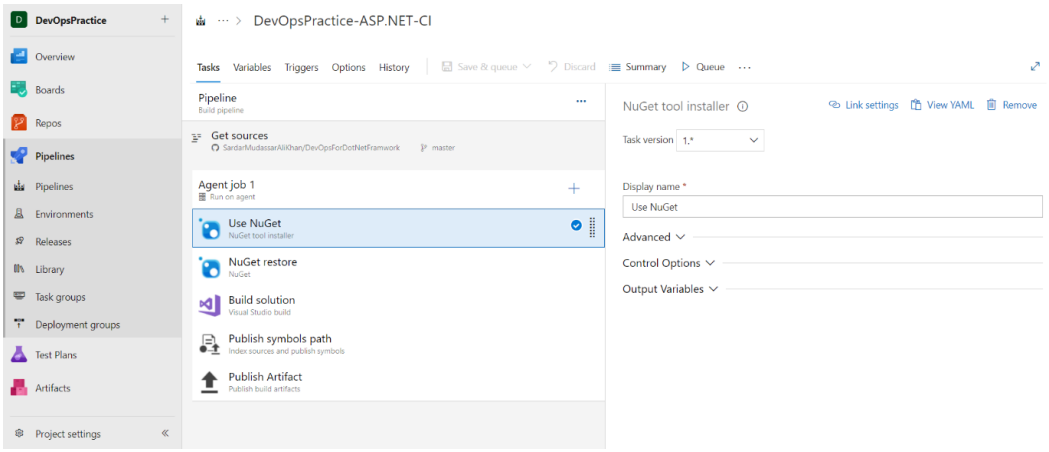


Select a Template: Choose the ASP.NET Core template, which includes tasks for restoring, building, and publishing your application.

Configure Build Pipeline

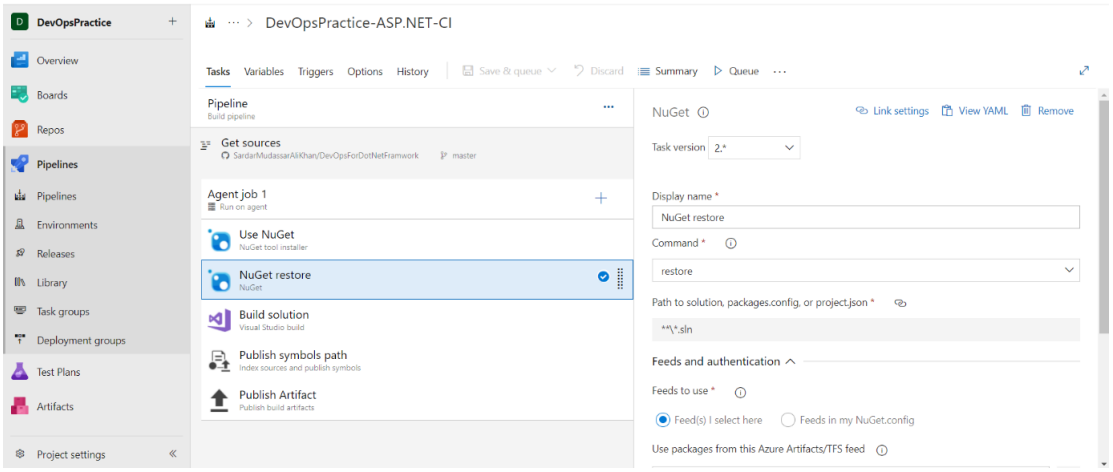
Agent Pool: Choose "Azure Pipelines" for Microsoft's hosted agents.

- **Restore:** Use the "NuGet restore" task to restore NuGet packages.
- **Task Name:** NuGet restore
- **Path to the solution:** Provide the path to your solution file (e.g., **/*.sln).



Build: Add a build task to compile your solution.

- **Task Name:** .NET Core
- **Command:** Build
- **Projects:** Provide the path to your project file (e.g., **/*.csproj).

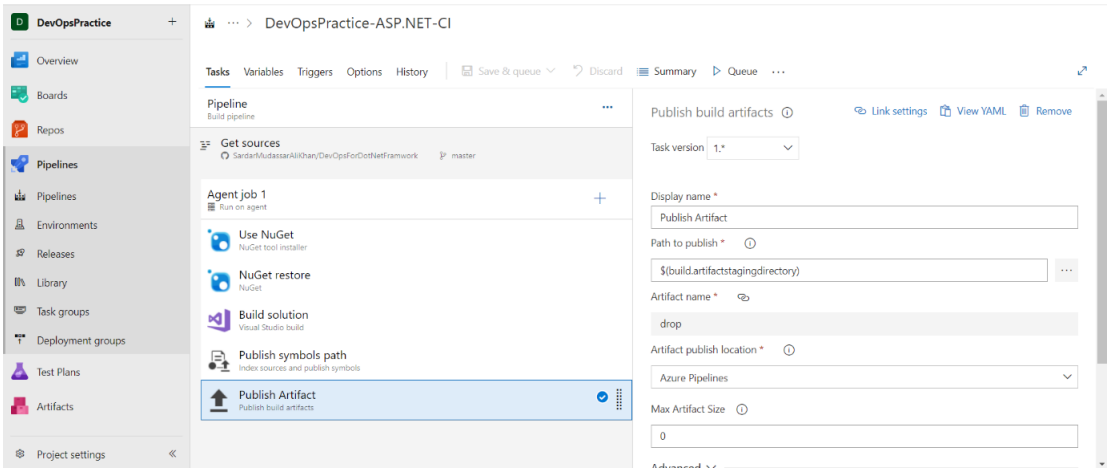


Test: Optionally, add a task to run tests.

- **Task Name:** .NET Core
- **Command:** Test
- **Projects:** Provide the path to your test project file (e.g., **/*Tests/*.csproj).

Publish: Use the "Publish Build Artifacts" task to publish the build outputs.

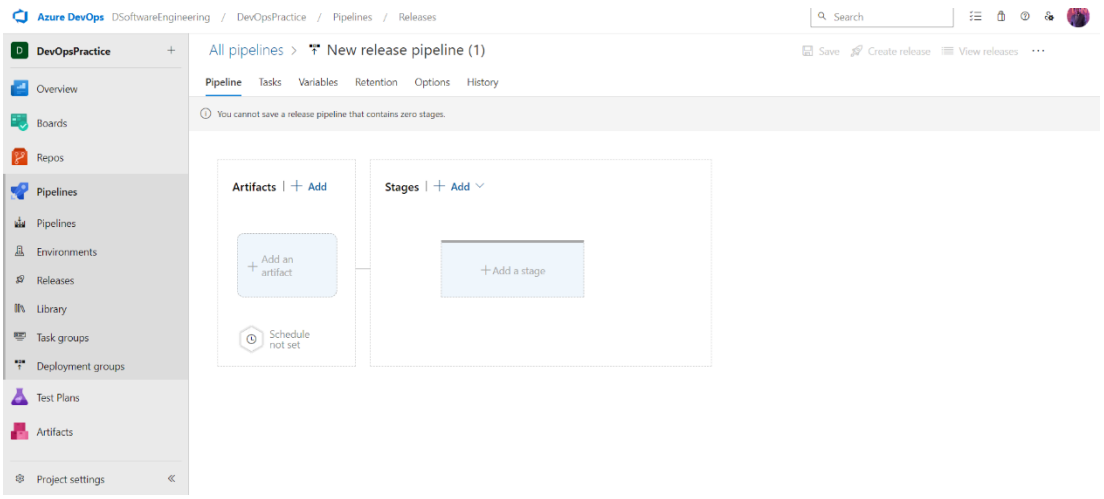
- **Task Name:** Publish and build artifacts
- **Path to Publish:** Specify the path to the directory containing your build outputs (e.g., $(Build.ArtifactStagingDirectory)).
- **Artifact Name:** Name the artifact (e.g., drop).
- **Save and Queue:** Save the pipeline and queue a new build to test the setup.



Step 6. Creating a Release Pipeline

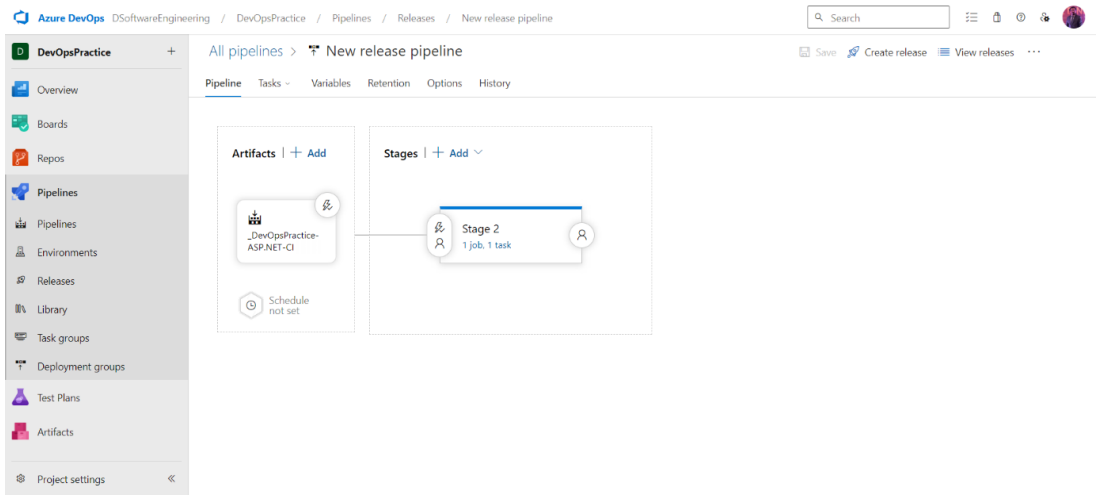
A release pipeline automates the process of deploying your application to various environments. Here's how to set it up.

- **Go to Releases:** In Azure DevOps, navigate to "Pipelines" and then "Releases".
- **New Pipeline:** Click on "New pipeline" and select "Empty job".



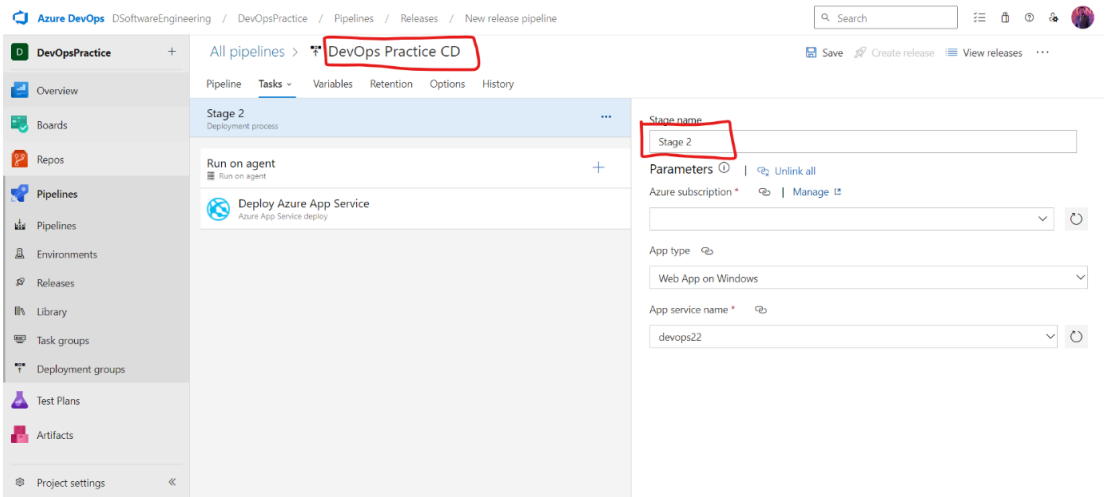
Add Artifacts: Link the artifacts from the build pipeline.

- **Source Type:** Build
- **Project:** Select your project.
- **Source:** Choose the build pipeline created earlier.
- **Default Version:** Select "Latest".



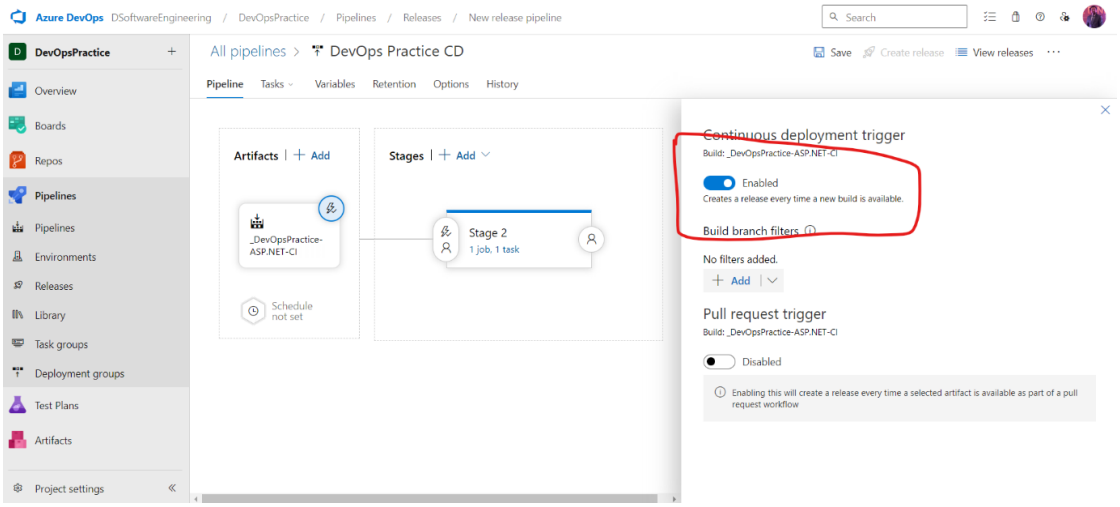
Add a Stage: Add a new stage and name it "Deployment".

- **Stage Name:** Provide a name for the stage (e.g., Dev).



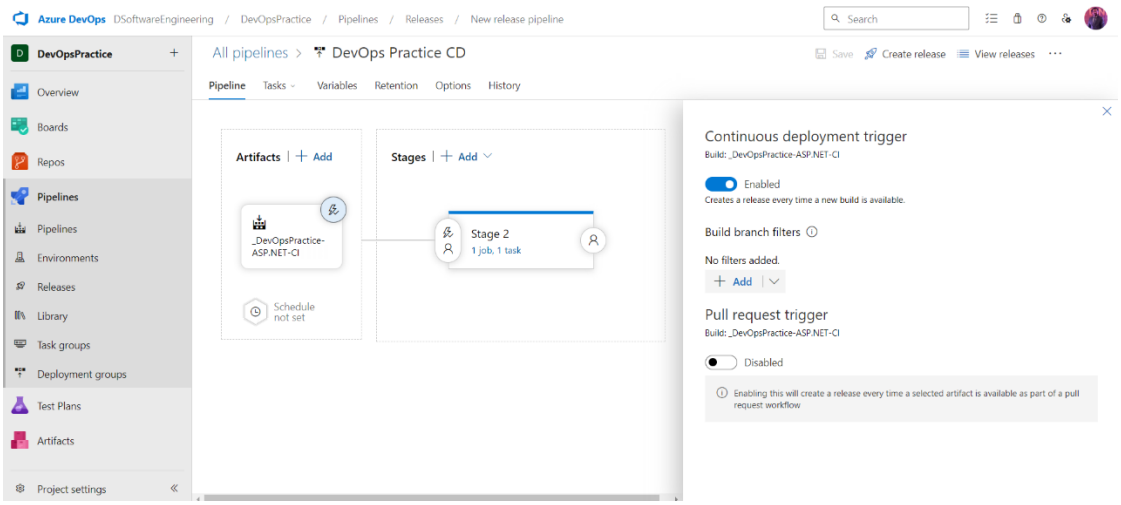
Add Deployment Task

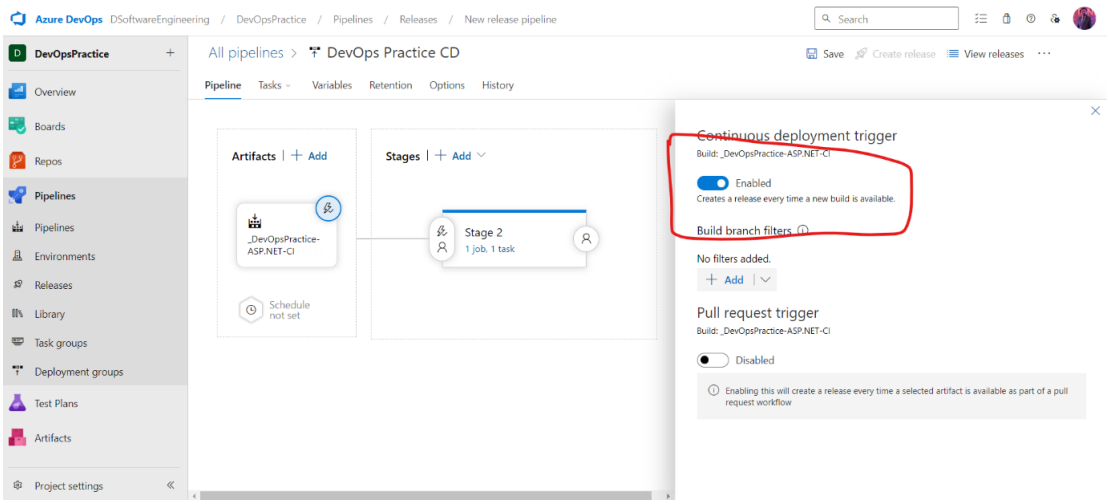
- Click on "1 job, 0 task" under your stage.
- Click the "+" icon to add a task and search for "Azure App Service deploy".
- Azure Subscription: Authorize Azure DevOps to connect to your Azure subscription.
- App Service Name: Select the web app you created earlier.
- Package or Folder: Point to the build artifacts (e.g., $(System.DefaultWorkingDirectory)/_your_build_artifact_name/drop).



Configure Continuous Deployment: Enable continuous deployment triggers to automatically deploy on successful builds.

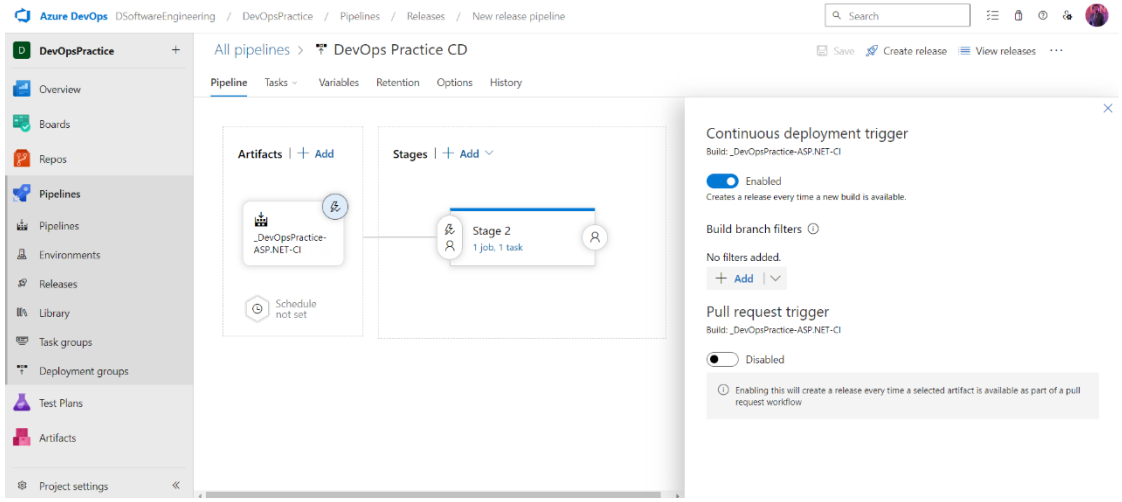
- Click on "Continuous deployment trigger" and enable it.

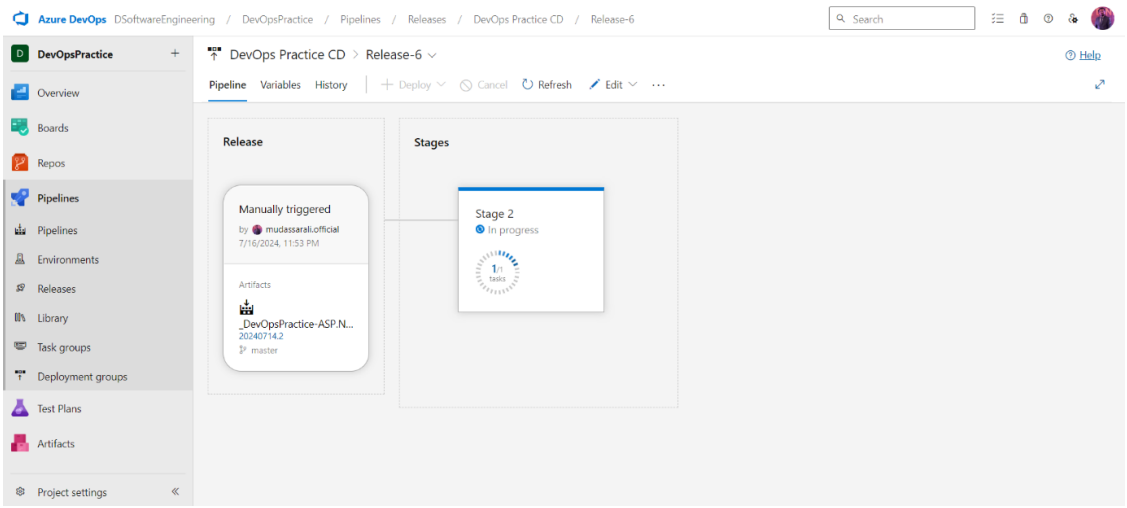




Save and Create Release: Save the pipeline and create a release to test the deployment.

- Click "Create release" and select the appropriate stages.
- Click "Create" to initiate the release process.

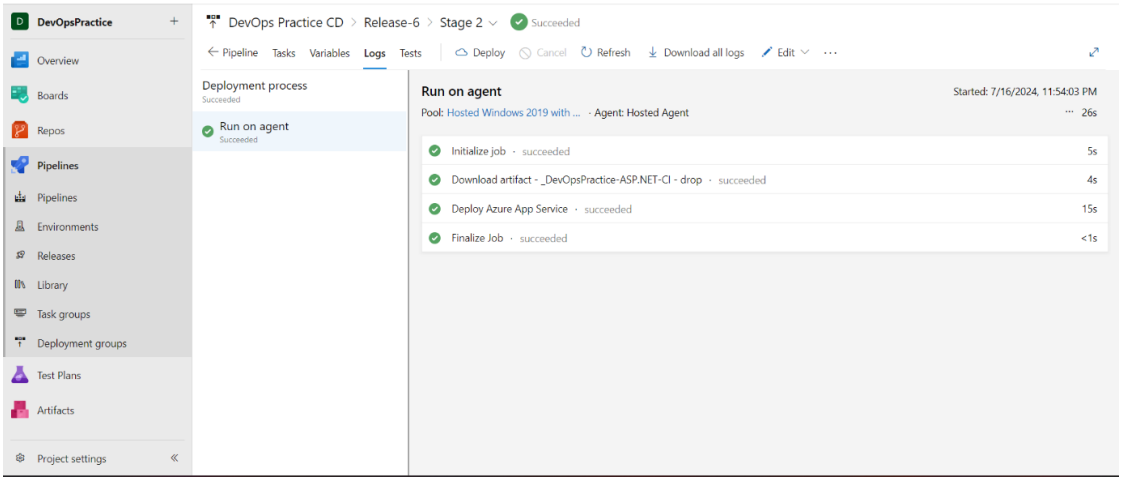




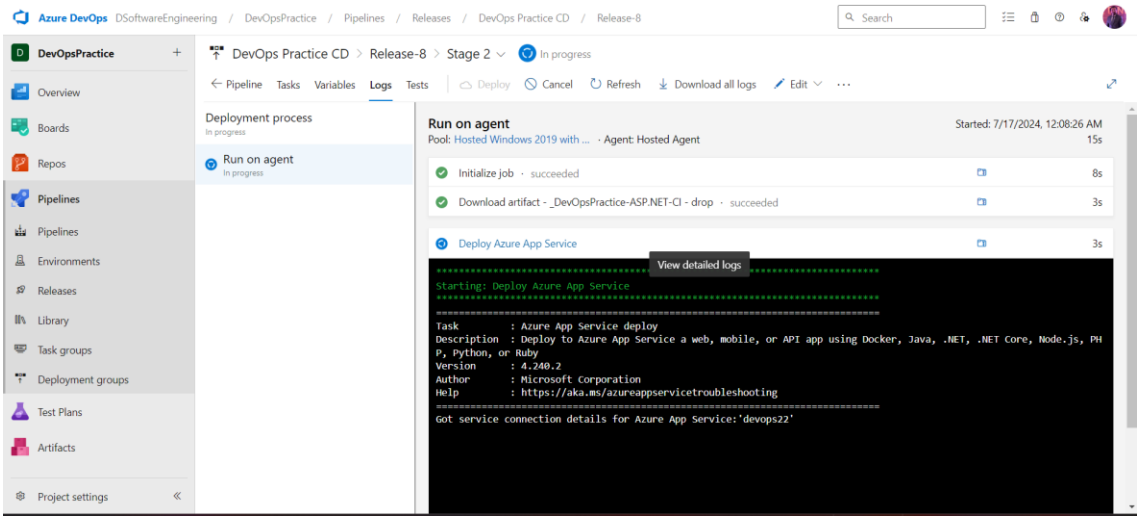
Step 7. Monitoring and Verifying Deployment

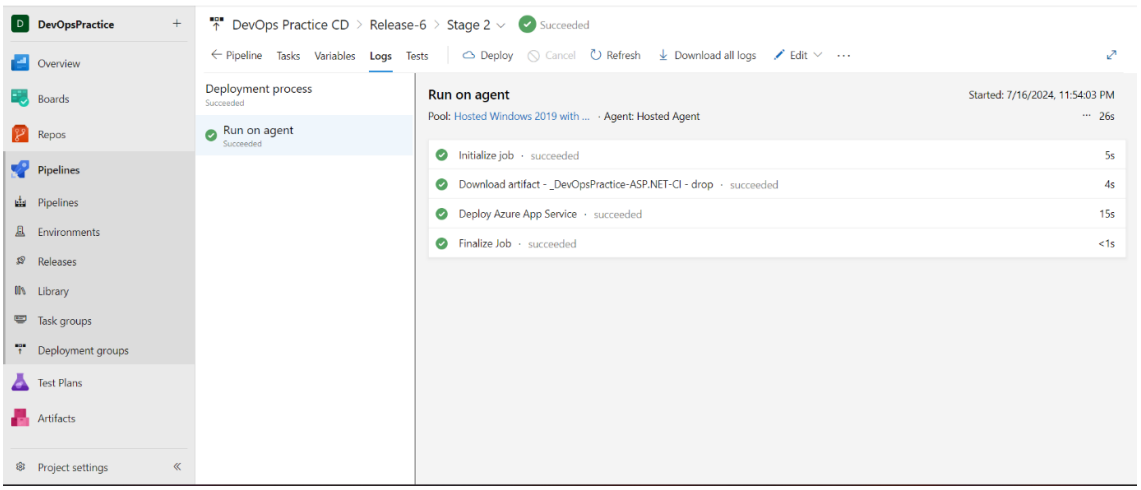
Once the release pipeline is set up, it’s important to monitor and verify the deployment:

- Monitor Release: Go to "Releases" to track the deployment process.
- Check logs and deployment status to ensure the process is completed successfully.
- Verify Deployment: After deployment, visit the Azure Portal and navigate to your web app.
- Access the URL of your web app (e.g., https://yourappname.azurewebsites.net) to ensure it is running correctly.



- Monitor the release pipeline





Conclusion

In this chapter, you have learned how to set up an end-to-end CI/CD pipeline for an ASP.NET MVC application using Azure DevOps and GitHub. By following these steps, you can automate the process of building, testing, and deploying your application to Azure App Service, ensuring a streamlined and efficient development workflow. This setup not only enhances productivity but also ensures that your application is always up to date with the latest changes, providing a seamless experience for both developers and users.

References

- https://www.c-sharpcorner.com/article/clean-architecture-in-asp-net-core-web-api/
- https://www.c-sharpcorner.com/article/onion-architecture-in-asp-net-core-6-web-api/
- https://www.c-sharpcorner.com/article/application-deployment-on-azure-cloud-using-azure-web-app-service/
- https://www.c-sharpcorner.com/article/how-to-create-azure-app-using-azure-cloud-portal/
- https://www.c-sharpcorner.com/article/crud-operation-in-asp-net-core-5-web-api/

C#Corner

OUR MISSION

Free Education is Our Basic Need! Our mission is to empower millions of developers worldwide by providing the latest unbiased news, advice, and tools for learning, sharing, and career growth. We're passionate about nurturing the next young generation and help them not only to become great programmers, but also exceptional human beings.

ABOUT US

CSharp Inc, headquartered in Philadelphia, PA, is an online global community of software developers. C# Corner served 29.4 million visitors in year 2022. We publish the latest news and articles on cutting-edge software development topics. Developers share their knowledge and connect via content, forums, and chapters. Thousands of members benefit from our monthly events, webinars, and conferences. All conferences are managed under Global Tech Conferences, a CSharp Inc sister company. We also provide tools for career growth such as career advice, resume writing, training, certifications, books and white-papers, and videos. We also connect developers with their potential employers via our Job board. Visit C# Corner

MORE BOOKS